

Babel

Code

Version 26.9.127548
2026/06/28

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	LaTeX: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	15
4.2	Errors	23
4.3	More on selection	24
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	27
4.8	Shorthands	30
4.9	Language attributes	39
4.10	Support for saving and redefining macros	40
4.11	French spacing	41
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	49
4.15	Making glyphs available	50
4.15.1	Quotation marks	50
4.15.2	Letters	51
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	54
4.17	Load engine specific macros	55
4.18	Creating and modifying languages	55
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	67
4.21	French spacing (again)	73
4.22	Handle language system	74
4.23	Numerals	75
4.24	Casing	76
4.25	Getting info	77
4.26	BCP 47 related commands	78
5	Adjusting the Babel behavior	79
5.1	Cross referencing macros	81
5.2	Layout	84
5.3	Marks	85
5.4	Other packages	86
5.4.1	ifthen	86
5.4.2	varioref	87
5.4.3	hhline	87
5.5	Encoding and fonts	88
5.6	Basic bidi support	90
5.7	Local Language Configuration	93
5.8	Language options	93

6	The kernel of Babel	97
7	Error messages	98
8	Loading hyphenation patterns	102
9	luatex + xetex: common stuff	106
10	Hooks for XeTeX and LuaTeX	109
10.1	XeTeX	109
10.2	Support for interchar	111
10.3	Layout	113
10.4	8-bit TeX	114
10.5	LuaTeX	115
10.6	Southeast Asian scripts	122
10.7	CJK line breaking	123
10.8	Arabic justification	125
10.9	Common stuff	129
10.10	Automatic fonts and ids switching	130
10.11	Bidi	137
10.12	Layout	139
10.13	Lua: transforms	149
10.14	Lua: Auto bidi with basic and basic-r	159
11	Data for CJK	171
12	The ‘nil’ language	171
13	Calendars	172
13.1	Islamic	172
13.2	Hebrew	174
13.3	Persian	178
13.4	Coptic and Ethiopic	178
13.5	Julian	179
13.6	Buddhist	179
14	Support for Plain T_EX (plain.def)	180
14.1	Not renaming hyphen.tex	180
14.2	Emulating some L ^A T _E X features	181
14.3	General tools	182
14.4	Encoding related macros	185
15	Acknowledgements	188

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `<<name=value>>`, or with a series of lines between `<<*name>>` and `<</name>>`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<version=26.9.127548>>
2 <<date=2026/06/28>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<*Basic macros>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8     {\def#1{#2}}%
9     {\expandafter\def\expandafter#1\expandafter{#1#2}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@csarg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@cs#1{\csname bbl@#1\endcsname}
17 \def\bbl@c#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
```

```

20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \< for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35   \let\<\noexpand
36   \let\<\bbl@exp@en
37   \let\[\bbl@exp@ue
38   \edef\bbl@exp@aux{\endgroup#1}%
39   \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{ }%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@\expandafter{\the\toks@##1}%
120     \else
121       \toks@\expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize@undefined\else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax}%
129   \def\bbl@tempa{#1}%
130   \def\bbl@tempb{#2}%
131   \def\bbl@tempe{#3}}
132 \def\bbl@sreplace#1#2#3{%
133   \begingroup
134     \expandafter\bbl@parsedef\meaning#1\relax
135     \def\bbl@tempc{#2}%
136     \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137     \def\bbl@tempd{#3}%
138     \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139     \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140     \ifin@
141       \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142       \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143         \\makeatletter % "internal" macros with @ are assumed
144         \\scantokens{%
145           \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146           \noexpand\noexpand}%
147         \catcode64=\the\catcode64\relax}% Restore @
148     \else
149       \let\bbl@tempc@empty % Not \relax
150     \fi
151     \bbl@exp{% For the 'uplevel' assignments
152   \endgroup
153   \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bbl@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bbl@tempb{#1}%
158   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
159   \protected@edef\bbl@tempc{#2}%
160   \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
161   \ifx\bbl@tempb\bbl@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bbl@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bbl@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bbl@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bbl@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal `\let's` made by `\MakeUppercase` and `\MakeLowercase` between things like `\oe` and `\OE`.

```

184 \def\bbl@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bbl@afterelse\expandafter\MakeUppercase
190   \else
191     \bbl@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with `#`'s. Used to deal with `alph`, `Alph` and frenchspacing when there are already changes (with `\babel@save`).

```

196 \def\bbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
197   \toks@\expandafter\expandafter\expandafter{%
198     \csname extras\language\endcsname}%
199   \bbl@exp{\in{#1}{\the\toks@}}%
200   \ifin@\else
201     \@temptokena{#2}%
202     \edef\bbl@tempc{\the\@temptokena\the\toks@}%
203     \toks@\expandafter{\bbl@tempc#3}%
204     \expandafter\edef\csname extras\language\endcsname{\the\toks@}%
205   \fi}
206 <</Basic macros>>

```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.


```

207 <<*Make sure ProvidesFile is defined>> ≡
208 \ifx\ProvidesFile\@undefined
209   \def\ProvidesFile#1[#2 #3 #4]{%
210     \wlog{File: #1 #4 #3 <#2>}%
211     \let\ProvidesFile\@undefined}
212 \fi
213 <</Make sure ProvidesFile is defined>>

```

3.1. A few core definitions

\language Just for compatibility, for not to touch hyphen.cfg.

```

214 <<*Define core switching macros>> ≡
215 \ifx\language\@undefined
216   \csname newcount\endcsname\language
217 \fi
218 <</Define core switching macros>>

```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```

219 <<*Define core switching macros>> ≡
220 \countdef\last@language=19
221 \def\addlanguage{\csname newlanguage\endcsname}
222 <</Define core switching macros>>

```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```

223 <*package>
224 \NeedsTeXFormat{LaTeX2e}
225 \ProvidesPackage{babel}%
226 [<@date@> v<@version@>
227   The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]

```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

228 \ifpackagewith{babel}{debug}
229   {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
230   \let\bbl@debug\@firstofone
231   \ifx\directlua\@undefined\else
232     \directlua{
233       Babel = Babel or {}
234       Babel.debug = true }%
235     \input{babel-debug.tex}%
236   \fi}
237 {\providecommand\bbl@trace[1]{}%
238 \let\bbl@debug\@gobble
239 \ifx\directlua\@undefined\else
240   \directlua{
241     Babel = Babel or {}
242     Babel.debug = false }%

```

```

243 \fi}
244 % Temporary:
245 \newif\ifBabelDebugGerman
246 \@ifpackagewith{babel}{debug-german}
247 {\BabelDebugGermantrue}
248 {\BabelDebugGermanfalse}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

249 \def\bbl@error#1{% Implicit #2#3#4
250 \begingroup
251 \catcode\==0 \catcode\==12 \catcode\`=12
252 \input errbabel.def
253 \endgroup
254 \bbl@error{#1}}
255 \def\bbl@warning#1{%
256 \begingroup
257 \def\{\MessageBreak}%
258 \PackageWarning{babel}{#1}%
259 \endgroup}
260 \def\bbl@infowarn#1{%
261 \begingroup
262 \def\{\MessageBreak}%
263 \PackageNote{babel}{#1}%
264 \endgroup}
265 \def\bbl@info#1{%
266 \begingroup
267 \def\{\MessageBreak}%
268 \PackageInfo{babel}{#1}%
269 \endgroup}

```

Many of the following options don't do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

270 <@Basic macros>
271 \@ifpackagewith{babel}{silent}
272 {\let\bbl@info\@gobble
273 \let\bbl@infowarn\@gobble
274 \let\bbl@warning\@gobble}
275 {}
276 %
277 \def\AfterBabelLanguage#1{%
278 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

279 \ifx\bbl@languages\undefined\else
280 \begingroup
281 \catcode\^^I=12
282 \@ifpackagewith{babel}{showlanguages}{%
283 \begingroup
284 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%
285 \wlog{<*languages>}%
286 \bbl@languages
287 \wlog{</languages>}%
288 \endgroup}{%
289 \endgroup
290 \def\bbl@elt#1#2#3#4{%
291 \ifnum#2=z@
292 \gdef\bbl@nulllanguage{#1}%
293 \def\bbl@elt##1##2##3##4{%
294 \fi}%
295 \bbl@languages
296 \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```
297 \bbl@trace{Defining option 'base'}
298 \@ifpackagewith{babel}{base}{%
299   \let\bbl@onlyswitch\@empty
300   \let\bbl@provide@locale\relax
301   \input babel.def
302   \let\bbl@onlyswitch\@undefined
303   \ifx\directlua\@undefined
304     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
305   \else
306     \input luababel.def
307     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
308   \fi
309   \DeclareOption{base}{}%
310   \DeclareOption{showlanguages}{}%
311   \ProcessOptions
312   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
313   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
314   \global\let\@ifl@ter@@\@ifl@ter
315   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@@}%
316   \endinput}{}%
```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```
317 \bbl@trace{key=value and another general options}
318 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
319 \def\bbl@tempb#1.#2{% Removes trailing dot
320   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
321 \def\bbl@tempe#1=#2\@@{%
322   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
323 \def\bbl@tempd#1.#2\@nnil{%
324   \ifx\@empty#2%
325     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
326   \else
327     \in@{,provide=}{, #1}%
328     \ifin@
329       \edef\bbl@tempc{%
330         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
331     \else
332       \in@{$modifiers$}{$#1$}%
333       \ifin@
334         \bbl@tempe#2\@@
335       \else
336         \in@{=}{#1}%
337         \ifin@
338           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
339         \else
340           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
341           \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
342         \fi
343       \fi
344     \fi
345   \fi}
346 \let\bbl@tempc\@empty
```

```

347 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
348 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

349 \DeclareOption{KeepShorthandsActive}{}
350 \DeclareOption{activeacute}{}
351 \DeclareOption{activegrave}{}
352 \DeclareOption{debug}{}
353 \DeclareOption{debug-german}{} % Temporary
354 \DeclareOption{noconfigs}{}
355 \DeclareOption{showlanguages}{}
356 \DeclareOption{silent}{}
357 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
358 \chardef\bbl@iniflag\z@
359 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
360 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
361 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
362 \chardef\bbl@ldfflag\z@
363 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
364 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
365 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
366 \DeclareOption{licr=extended}{}
367 \DeclareOption{licr=unextended}{}
368 % Don't use. Experimental.
369 \newif\ifbbl@single
370 \DeclareOption{selectors=off}{\bbl@singletrue}
371 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key main, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

372 \let\bbl@opt@shorthands\@nnil
373 \let\bbl@opt@config\@nnil
374 \let\bbl@opt@main\@nnil
375 \let\bbl@opt@headfoot\@nnil
376 \let\bbl@opt@layout\@nnil
377 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

378 \def\bbl@tempa#1=#2\bbl@tempa{%
379   \bbl@csarg@ifx{opt#1}\@nnil
380     \bbl@csarg\edef{opt#1}{#2}%
381   \else
382     \bbl@error{bad-package-option}{#1}{#2}{}%
383   \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in \bbl@language@opts, because they are language options.

```

384 \let\bbl@language@opts\@empty
385 \DeclareOption*{%
386   \bbl@xin@{\string=}{\CurrentOption}%
387   \ifin@
388     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
389   \else
390     \bbl@add@list\bbl@language@opts{\CurrentOption}%
391   \fi}

```

Now we finish the first pass (and start over).

```

392 \ProcessOptions*

```

3.5. Post-process some options

```
393 \ifx\bbl@opt@provide\@nnil
394 \let\bbl@opt@provide\@empty %%% MOVE above
395 \else
396 \chardef\bbl@iniflag\@ne
397 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
398 \in@{,provide,}{, #1,}%
399 \ifin@
400 \def\bbl@opt@provide{#2}%
401 \fi}
402 \fi
```

If there is no `shorthands=<chars>`, the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```
403 \bbl@trace{Conditional loading of shorthands}
404 \def\bbl@sh@string#1{%
405 \ifx#1\@empty\else
406 \ifx#1t\string~%
407 \else\ifx#1c\string,%
408 \else\string#1%
409 \fi\fi
410 \expandafter\bbl@sh@string
411 \fi}
412 \ifx\bbl@opt@shorthands\@nnil
413 \def\bbl@ifshorthand#1#2#3{#2}%
414 \else\ifx\bbl@opt@shorthands\@empty
415 \def\bbl@ifshorthand#1#2#3{#3}%
416 \else
```

The following macro tests if a shorthand is one of the allowed ones.

```
417 \def\bbl@ifshorthand#1{%
418 \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
419 \ifin@
420 \expandafter\@firstoftwo
421 \else
422 \expandafter\@secondoftwo
423 \fi}
```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```
424 \edef\bbl@opt@shorthands{%
425 \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%
```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```
426 \bbl@ifshorthand{'}%
427 {\PassOptionsToPackage{activeacute}{babel}}{}
428 \bbl@ifshorthand{`}%
429 {\PassOptionsToPackage{activegrave}{babel}}{}
430 \fi\fi
```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\@resetactivechars`, but seems to work.

```
431 \ifx\bbl@opt@headfoot\@nnil\else
432 \g@addto@macro\@resetactivechars{%
433 \set@typeset@protect
434 \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
435 \let\protect\noexpand}
436 \fi
```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to `none`.

```
437 \ifx\bbl@opt@safe\@undefined
```

```

438 \def\bbl@opt@safe{BR}
439 % \let\bbl@opt@safe\@empty % Pending of \cite
440 \fi

For layout an auxiliary macro is provided, available for packages and language styles.
Optimization: if there is no layout, just do nothing.
441 \bbl@trace{Defining IfBabelLayout}
442 \ifx\bbl@opt@layout\@nnil
443 \newcommand\IfBabelLayout[3]{#3}%
444 \else
445 \bbl@exp{\bbl@forkv{\@nameuse{raw@opt@babel.sty}}}{%
446 \in{,layout,}{, #1,}%
447 \ifin@
448 \def\bbl@opt@layout{#2}%
449 \bbl@replace\bbl@opt@layout{ }{.}%
450 \fi}
451 \newcommand\IfBabelLayout[1]{%
452 \@expandtwoargs\in{.#1.}{.\bbl@opt@layout.}%
453 \ifin@
454 \expandafter\@firstoftwo
455 \else
456 \expandafter\@secondoftwo
457 \fi}
458 \fi
459 </package>

```

3.6. Plain: babel.def (start)

Because of the way docstrip works, we need to insert some code for Plain here. However, the tools provided by the babel installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

460 < *core >
461 \ifx\ldf@quit\@undefined\else
462 \endinput\fi % Same line!
463 <@Make sure ProvidesFile is defined@>
464 \ProvidesFile{babel.def}[<@date@> v<@version@> Babel common definitions]
465 \ifx\AtBeginDocument\@undefined
466 <@Emulate LaTeX@>
467 \fi
468 <@Basic macros@>
469 </core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\text{\LaTeX}$. After it, we will resume the $\text{\LaTeX}$ -only stuff.

4. babel.sty and babel.def (common)

```

470 < *package | core >
471 \def\bbl@version{<@version@>}
472 \def\bbl@date{<@date@>}
473 <@Define core switching macros@>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

474 \def\adddialect#1#2{%
475 \global\chardef#1#2\relax
476 \bbl@usehooks{adddialect}{#1}{#2}%
477 \begingroup
478 \count@#1\relax
479 \def\bbl@elt##1##2##3##4{%
480 \ifnum\count@=##2\relax
481 \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
482 \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'

```

```

483             set to \expandafter\string\csname l@##1\endcsname\\%
484             (\string\language\the\count@). Reported}%
485         \def\bbl@elt###1###2###3###4{}%
486         \fi}%
487     \bbl@cs{languages}%
488 \endgroup

```

\bbl@iflanguage executes code only if the language l@ exists. Otherwise raises an error.

The argument of \bbl@fixname has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when \foreignlanguage and the like appear in a \MakeXXXcase. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note l@ is encapsulated, so that its case does not change.

```

489 \def\bbl@fixname#1{%
490     \begingroup
491     \def\bbl@tempe{l@}%
492     \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
493     \bbl@tempd
494     {\lowercase\expandafter{\bbl@tempd}%
495     {\uppercase\expandafter{\bbl@tempd}%
496     \@empty
497     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
498     \uppercase\expandafter{\bbl@tempd}}}%
499     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
500     \lowercase\expandafter{\bbl@tempd}}}%
501     \@empty
502     \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
503 \bbl@tempd
504 \bbl@exp{\bbl@usehooks{language}{\language}{#1}}}%
505 \def\bbl@iflanguage#1{%
506     \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that sr-latn-ba becomes fr-Latn-BA.

\bbl@bcplookup returns \bbl@bcp with either the found ini tag or \relax. Temporary version, to be refactored with the new \text_bcp_parse:n, which is not currently fully expandable with invalid BCP 47 tags.

```

507 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
508 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
509 \def\bbl@bcplookup#1%    This section is temporal
510     \let\bbl@templ\@empty % language
511     \let\bbl@temps\@empty % script
512     \let\bbl@tempr\@empty % regions
513     \let\bbl@tempv\@empty % variant
514     \edef\bbl@tempa{#1}%
515     \bbl@replace\bbl@tempa{-}{,}%
516     \bbl@replace\bbl@tempa_{,}%
517     \bbl@foreach\bbl@tempa{%
518         \ifx\bbl@templ\@empty
519             \lowercase{\def\bbl@templ{##1}}%
520         \else\ifnum\bbl@cntchars{##1}=4
521             \bbl@xin{\@car##1\@nil}{0123456789}%
522             \ifin@
523                 \def\bbl@tempv{##1}% eg, 1901
524             \else
525                 \uppercase{\edef\bbl@tempb{\@car##1\@nil}}%
526                 \lowercase{\edef\bbl@tempb{\bbl@tempb\@gobble##1}}%
527             \fi
528         \else\ifnum\bbl@cntchars{##1}<4
529             \uppercase{\def\bbl@tempr{##1}}%
530         \else\ifnum\bbl@cntchars{##1}>4
531             \lowercase{\def\bbl@tempv{##1}}%

```

```

532 \fi\fi\fi\fi}%
533 \bbl@exp{\bbl@bcpllookup@i{\bbl@templ}{\bbl@temps}{\bbl@tempr}{\bbl@tempv}}
534 \def\bbl@bcpllookup@try#1{%
535 \ifx\bbl@bcp\relax
536 \IfFileExists{babel-#1\bbl@tempe.ini}{\edef\bbl@bcp{#1\bbl@tempe}}{}%
537 \fi}
538 \def\bbl@bcpllookup@i#1#2#3#4{%
539 \let\bbl@bcp\relax
540 \def\bbl@tempe{#4}%
541 \ifx\bbl@tempe\@empty\else
542 \edef\bbl@tempe{-#4}%
543 \fi
544 \edef\bbl@tempa{#2#3\bbl@tempe}% en
545 \ifx\bbl@tempa\@empty
546 \bbl@bcpllookup@try{#1}%
547 \else
548 \edef\bbl@tempa{#3\bbl@tempe}% en-Latn
549 \ifx\bbl@tempa\@empty
550 \bbl@bcpllookup@try{#1-#2}%
551 \bbl@bcpllookup@try{#1}%
552 \else
553 \edef\bbl@tempa{#2\bbl@tempe}% en-US, en-001
554 \ifx\bbl@tempa\@empty
555 \bbl@bcpllookup@try{#1-#3}%
556 \bbl@bcpllookup@try{#1}%
557 \else % en-Latn-US
558 \bbl@bcpllookup@try{#1-#2-#3}%
559 \bbl@bcpllookup@try{#1-#2}%
560 \bbl@bcpllookup@try{#1-#3}%
561 \bbl@bcpllookup@try{#1}%
562 \fi
563 \fi
564 \fi
565 \ifx\bbl@bcp\relax
566 \ifx\bbl@tempe\@empty\else\bbl@bcpllookup@i{#1}{#2}{#3}{\fi}%
567 \fi}
568 %
569 \let\bbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

570 \def\iflanguage#1{%
571 \bbl@iflanguage{#1}{%
572 \ifnum\csname l@#1\endcsname=\language
573 \expandafter\@firstoftwo
574 \else
575 \expandafter\@secondoftwo
576 \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

577 \let\bbl@select@type\z@
578 \edef\selectlanguage{%
579 \noexpand\protect
580 \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguageL`. Therefore, we have to make sure that a macro `\protect` exists. If it

doesn't it is \let to \relax.

```
581 \ifx\@undefined\protect\let\protect\relax\fi
```

The following definition is preserved for backwards compatibility (e.g., arabi, koma). It is related to a trick for 2.09, now discarded.

```
582 \let\xstring\string
```

Since version 3.5 babel writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's *aftergroup* mechanism to help us. The command \aftergroup stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence \bbl@pop@language to be executed at the end of the group. It calls \bbl@set@language with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called \bbl@language@stack and initially empty.

```
583 \def\bbl@language@stack{}
```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```
584 \def\bbl@push@language{%
585   \ifx\language\@undefined\else
586     \ifx\currentgrouplevel\@undefined
587       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
588     \else
589       \ifnum\currentgrouplevel=\z@
590         \xdef\bbl@language@stack{\language+}%
591       \else
592         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
593       \fi
594     \fi
595 }
```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro \language. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the '+'-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
596 \def\bbl@pop@lang#1+#2\@@{%
597   \edef\language{#1}%
598   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed TeX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a '+'-sign (zero language names won't occur as this macro will only be called after something has been pushed on the stack).

```
599 \let\bbl@ifrestoring\@secondoftwo
600 \def\bbl@pop@language{%
601   \expandafter\bbl@pop@lang\bbl@language@stack\@@
602   \let\bbl@ifrestoring\@firstoftwo
603   \expandafter\bbl@set@language\expandafter{\language}%
604   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to `\bbl@set@language` to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of `\localeid`. This means `\l@. . .` will be reserved for hyphenation patterns (so that two locales can share the same rules).

```

605 \chardef\localeid\z@
606 \gdef\bbl@id@last{0} % No real need for a new counter
607 \def\bbl@id@assign{%
608   \bbl@ifunset\bbl@id@\language\language}%
609   {\count@\bbl@id@last\relax
610    \advance\count@\@ne
611    \global\bbl@csarg\chardef{id@\language\language}\count@
612    \xdef\bbl@id@last{\the\count@}%
613    \ifcase\bbl@engine\or
614      \directlua{
615        Babel.locale_props[\bbl@id@last] = {}
616        Babel.locale_props[\bbl@id@last].name = '\language'
617        Babel.locale_props[\bbl@id@last].vars = {}
618      }%
619      \fi}%
620   }%
621   \chardef\localeid\bbl@cl{id@}

```

The unprotected part of `\selectlanguage`. In case it is used as environment, declare `\endselectlanguage`, just for safety.

```

622 \let\bbl@select@opts@empty
623 \expandafter\def\csname selectlanguage \endcsname{%
624   \ifnextchar[\bbl@select@s{\bbl@select@s[]}}
625 \def\bbl@select@s[#1]#2{%
626   \def\bbl@select@opts{#1}%
627   \ifnum\bbl@hymapsel=\ccclv\let\bbl@hymapsel\tw\fi
628   \bbl@push@language
629   \aftergroup\bbl@pop@language
630   \bbl@set@language{#2}}
631 \let\endselectlanguage\relax

```

`\bbl@set@language` The macro `\bbl@set@language` takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of `\language`. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in `\language` are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining `\BabelContentsFiles`, but make sure they are loaded inside a group (as `aux`, `toc`, `lof`, and `lot` do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from hyperref, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

632 \def\BabelContentsFiles{toc,lof,lot}
633 \def\bbl@set@language#1{% from selectlanguage, pop@
634   % The old buggy way. Preserved for compatibility, but simplified
635   \edef\language{\expandafter\string#1\empty}%
636   \select@language{\language}%
637   \bbl@xin@{,main,},{,\bbl@select@opts,}%
638   \ifin@
639     \let\bbl@main@language\localename
640     \let\mainlocalename\localename
641   \fi
642   \let\bbl@select@opts@empty
643   % write to aux files
644   \expandafter\ifx\csname date\language\endcsname\relax\else

```

```

645 \if@filesw
646 \bbl@xin@{,nofiles,}{,\bbl@select@opts,}%
647 \ifin@else
648 \ifx\babel@aux@gobbletwo\else % Set if single in the first, redundant
649 \bbl@savelastskip
650 \protected@write\@auxout{}\string\babel@aux{\bbl@auxname}{}}%
651 \bbl@restorelastskip
652 \fi
653 \bbl@usehooks{write}{}%
654 \fi
655 \fi
656 \fi}
657 %
658 \let\bbl@restorelastskip\relax
659 \let\bbl@savelastskip\relax
660 %
661 \def\select@language#1{% from set@, babel@aux, babel@toc
662 \ifx\bbl@select@name\empty
663 \def\bbl@select@name{select}%
664 \fi
665 % set hmap
666 \ifnum\bbl@hmapsel=\@cclv\chardef\bbl@hmapsel4\relax\fi
667 % set name (when coming from babel@aux)
668 \edef\language{#1}%
669 \bbl@fixname\language
670 % define \localename when coming from set@, with a trick
671 \ifx\scantokens\undefined
672 \def\localename{??}%
673 \else
674 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
675 \fi
676 \bbl@provide@locale
677 \bbl@iflanguage\language{%
678 \let\bbl@select@type\z@
679 \expandafter\bbl@switch\expandafter{\language}}
680 \def\babel@aux#1#2{%
681 \select@language{#1}%
682 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
683 \@writefile{##1}{\babel@toc{#1}{#2}\relax}}}%
684 \def\babel@toc#1#2{%
685 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring $\TeX$ in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

686 \newif\ifbbl@usedategroup
687 \let\bbl@savextras\empty
688 \def\bbl@switch#1{% from select@, foreign@
689 % restore
690 \originalTeX
691 \expandafter\def\expandafter\originalTeX\expandafter{%
692 \csname noextras#1\endcsname

```

```

693 \let\originalTeX\@empty
694 \babel@beginsave}%
695 \bbl@usehooks{afterreset}{}%
696 \languageshorthands{none}%
697 % set the locale id
698 \bbl@id@assign
699 % switch captions, date
700 \bbl@bsphack
701 \ifcase\bbl@select@type
702 \csname captions#1\endcsname\relax
703 \csname date#1\endcsname\relax
704 \else
705 \bbl@xin@{,captions,}{,\bbl@select@opts,}%
706 \ifin@
707 \csname captions#1\endcsname\relax
708 \fi
709 \bbl@xin@{,date,}{,\bbl@select@opts,}%
710 \ifin@ % if \foreign... within \<language>date
711 \csname date#1\endcsname\relax
712 \fi
713 \fi
714 \bbl@esphack
715 % switch extras
716 \csname bbl@preextras@#1\endcsname
717 \bbl@usehooks{beforeextras}{}%
718 \csname extras#1\endcsname\relax
719 \bbl@usehooks{afterextras}{}%
720 % > babel-ensure
721 % > babel-sh-<short>
722 % > babel-bidi
723 % > babel-fontspec
724 \let\bbl@savextras\@empty
725 % hyphenation - case mapping
726 \ifcase\bbl@opt@hyphenmap\or
727 \def\BabelLower##1##2{\lccode##1=##2\relax}%
728 \ifnum\bbl@hymap>4\else
729 \csname\language @bbl@hyphenmap\endcsname
730 \fi
731 \chardef\bbl@opt@hyphenmap\z@
732 \else
733 \ifnum\bbl@hymap>\bbl@opt@hyphenmap\else
734 \csname\language @bbl@hyphenmap\endcsname
735 \fi
736 \fi
737 \let\bbl@hymap\@cclv
738 % hyphenation - select rules
739 \ifnum\csname l@\language\endcsname=\l@unhyphenated
740 \edef\bbl@tempa{u}%
741 \else
742 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
743 \fi
744 % linebreaking - handle u, e, k (v in the future)
745 \bbl@xin@{/u}{/\bbl@tempa}%
746 \ifin@ \else \bbl@xin@{/e}{/\bbl@tempa} \fi % elongated forms
747 \ifin@ \else \bbl@xin@{/k}{/\bbl@tempa} \fi % only kashida
748 \ifin@ \else \bbl@xin@{/p}{/\bbl@tempa} \fi % padding (e.g., Tibetan)
749 \ifin@ \else \bbl@xin@{/v}{/\bbl@tempa} \fi % variable font
750 % hyphenation - save mins
751 \babel@savevariable\lefthyphenmin
752 \babel@savevariable\righthyphenmin
753 \ifnum\bbl@engine=@ne
754 \babel@savevariable\hyphenationmin
755 \fi

```

```

756 \ifin@
757 % unhyphenated/kashida/elongated/padding = allow stretching
758 \language\l@unhyphenated
759 \babel@savevariable\emergencystretch
760 \emergencystretch\maxdimen
761 \babel@savevariable\hbadness
762 \hbadness\@M
763 \else
764 % other = select patterns
765 \bbl@patterns{#1}%
766 \fi
767 % hyphenation - set mins
768 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
769 \set@hyphenmins\tw@\thr@@\relax
770 \@nameuse{bbl@hyphenmins@}%
771 \else
772 \expandafter\expandafter\expandafter\set@hyphenmins
773 \csname #1hyphenmins\endcsname\relax
774 \fi
775 \@nameuse{bbl@hyphenmins@}%
776 \@nameuse{bbl@hyphenmins@\language\language}%
777 \@nameuse{bbl@hyphenatmin@}%
778 \@nameuse{bbl@hyphenatmin@\language\language}%
779 \let\bbl@selectortname\empty}

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

780 \edef\otherlanguage{%
781 \noexpand\protect
782 \expandafter\noexpand\csname otherlanguage \endcsname}
783 \expandafter\def\csname otherlanguage \endcsname{%
784 \@ifstar{\@nameuse{otherlanguage*}}\bbl@otherlanguage}
785 \def\bbl@otherlanguage#1{%
786 \def\bbl@selectortname{other}%
787 \ifnum\bbl@hymapsel=\@ccclv\let\bbl@hymapsel\thr@@\fi
788 \csname selectlanguage \endcsname{#1}%
789 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

790 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of `\foreign@language`.

```

791 \expandafter\def\csname otherlanguage*\endcsname{%
792 \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
793 \def\bbl@otherlanguage@s[#1]#2{%
794 \def\bbl@selectortname{other*}%
795 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
796 \def\bbl@select@opts{#1}%
797 \foreign@language{#2}}

```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```

798 \expandafter\let\csname endotherlanguage*\endcsname\relax

```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the

`\extras<language>` command doesn't make any `\global` changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a 'text' command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```

799 \providecommand\bbl@beforeforeign{}
800 \edef\foreignlanguage{%
801   \noexpand\protect
802   \expandafter\noexpand\csname foreignlanguage \endcsname}
803 \expandafter\def\csname foreignlanguage \endcsname{%
804   \@ifstar\bbl@foreign@s\bbl@foreign@x}
805 \providecommand\bbl@foreign@x[3][]{%
806   \begingroup
807     \def\bbl@select@name{foreign}%
808     \def\bbl@select@opts{#1}%
809     \let\BabelText\@firstofone
810     \bbl@beforeforeign
811     \foreign@language{#2}%
812     \bbl@usehooks{foreign}{}%
813     \BabelText{#3}% Now in horizontal mode!
814   \endgroup}
815 \def\bbl@foreign@s#1#2{%
816   \begingroup
817     {\par}%
818     \def\bbl@select@name{foreign*}%
819     \let\bbl@select@opts\@empty
820     \let\BabelText\@firstofone
821     \foreign@language{#1}%
822     \bbl@usehooks{foreign*}{}%
823     \bbl@dirparastext
824     \BabelText{#2}% Still in vertical mode!
825   {\par}%
826   \endgroup}
827 \providecommand\BabelWrapText[1]{%
828   \def\bbl@tempa{\def\BabelText###1}%
829   \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}
```

`\foreign@language` This macro does the work for `\foreignlanguage` and the `otherlanguage*` environment. First we need to store the name of the language and check that it is a known language. Then it just calls `bbl@switch`.

```

830 \def\foreign@language#1{%
831   % set name
832   \edef\language#1}%
833 \ifbbl@usedategroup
834   \bbl@add\bbl@select@opts{,date,}%
835   \bbl@usedategroupfalse
836 \fi
837 \bbl@fixname\language
838 \let\localname\language
839 \bbl@provide@locale
840 \bbl@iflanguage\language%
```

```

841 \let\bbl@select@type\@ne
842 \expandafter\bbl@switch\expandafter{\language\name}}

```

The following macro executes conditionally some code based on the selector being used.

```

843 \def\IfBabelSelectorTF#1{%
844 \bbl@xin@{\bbl@select@name,}{,\zap@space#1 \@empty,}%
845 \ifin@
846 \expandafter\@firstoftwo
847 \else
848 \expandafter\@secondoftwo
849 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

850 \let\bbl@hyphlist\@empty
851 \let\bbl@hyphenation@\relax
852 \let\bbl@pttnlist\@empty
853 \let\bbl@patterns@\relax
854 \let\bbl@hymapsel=\@cclv
855 \def\bbl@patterns#1{%
856 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
857 \csname l@#1\endcsname
858 \edef\bbl@tempa{#1}%
859 \else
860 \csname l@#1:\f@encoding\endcsname
861 \edef\bbl@tempa{#1:\f@encoding}%
862 \fi
863 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
864 % > luatex
865 \ifundefined\bbl@hyphenation@{}{}% Can be \relax!
866 \begingroup
867 \bbl@xin@{\number\language,}{,\bbl@hyphlist}%
868 \ifin@\else
869 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
870 \hyphenation{%
871 \bbl@hyphenation@
872 \@ifundefined\bbl@hyphenation@#1}%
873 \@empty
874 {\space\csname bbl@hyphenation@#1\endcsname}}%
875 \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
876 \fi
877 \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change \language and when the hyphenation rules specified were not loaded it has no effect. Note however, \lccode's and font encodings are not set at all, so in most cases you should use otherlanguage*.

```

878 \def\hyphenrules#1{%
879 \edef\bbl@tempf{#1}%
880 \bbl@fixname\bbl@tempf
881 \bbl@iflanguage\bbl@tempf{%
882 \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
883 \ifx\languageshortands\@undefined\else
884 \languageshortands{none}%
885 \fi
886 \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
887 \set@hyphenmins\tw@thr@@ \relax

```

```

888 \else
889 \expandafter\expandafter\expandafter\set@hyphenmins
890 \csname\bbl@tempf hyphenmins\endcsname\relax
891 \fi}}
892 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\<language>hyphenmins` is already defined this command has no effect.

```

893 \def\providehyphenmins#1#2{%
894 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
895 \@namedef{#1hyphenmins}{#2}%
896 \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

897 \def\set@hyphenmins#1#2{%
898 \lefthyphenmin#1\relax
899 \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_{\epsilon}$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by babel.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

900 \ifx\ProvidesFile\@undefined
901 \def\ProvidesLanguage#1[#2 #3 #4]{%
902 \wlog{Language: #1 #4 #3 <#2>}%
903 }
904 \else
905 \def\ProvidesLanguage#1{%
906 \begingroup
907 \catcode`\ 10 %
908 \@makeother\/%
909 \@ifnextchar[%]
910 {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}}
911 \def\@provideslanguage#1[#2]{%
912 \wlog{Language: #1 #2}%
913 \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
914 \endgroup}
915 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\text{\TeX}$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```

916 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi

```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```

917 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi

```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```

918 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}
919 \let\uselocale\setlocale
920 \let\locale\setlocale
921 \let\selectlocale\setlocale
922 \let\textlocale\setlocale
923 \let\textlanguage\setlocale
924 \let\languagetext\setlocale

```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn't been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\text{\TeX 2.}\epsilon$, so we can safely use its error handling interface. Otherwise we'll have to 'keep it simple'.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```

925 \edef\bbl@nulllanguage{\string\language=0}
926 \def\bbl@nocaption{\protect\bbl@nocaption@i}
927 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
928   \global\@namedef{#2}{\textbf{?#1?}}%
929   \@nameuse{#2}%
930   \edef\bbl@tempa{#1}%
931   \bbl@sreplace\bbl@tempa{name}{}%
932   \bbl@sreplace\bbl@tempa{NAME}{}%
933   \bbl@warning{%
934     \@backslashchar#1 not set for '\language'. Please,\\%
935     define it after the language has been loaded\\%
936     (typically in the preamble) with:\\%
937     \string\setlocalecaption{\language}{\bbl@tempa}{..}\\%
938     Feel free to contribute on github.com/latex3/babel.\\%
939     Reported}}
940 \def\bbl@tentative{\protect\bbl@tentative@i}
941 \def\bbl@tentative@i#1{%
942   \bbl@warning{%
943     Some functions for '#1' are tentative.\\%
944     They might not work as expected and their behavior\\%
945     could change in the future.\\%
946     Reported}}
947 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}{}}
948 \def\@nopatterns#1{%
949   \bbl@warning
950   {No hyphenation patterns were preloaded for\\%
951    the language '#1' into the format.\\%
952    Please, configure your TeX system to add them and\\%
953    rebuild the format. Now I will use the patterns\\%
954    preloaded for \bbl@nulllanguage\space instead}}
955 \let\bbl@usehooks\@gobbletwo

Here ended the now discarded switch.def.
Here also (currently) ends the base option.

956 \ifx\bbl@onlyswitch\@empty\endinput\fi

```

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\bbl@e@<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\bbl@e@<language>` contains `\bbl@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\bbl@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

957 \bbl@trace{Defining babelensure}
958 \newcommand\babelensure[2][]{%
959   \AddBabelHook{babel-ensure}{afterextras}{%
960     \ifcase\bbl@select@type

```

```

961     \bbl@cl{e}%
962 \fi}%
963 \begingroup
964 \let\bbl@ens@include\@empty
965 \let\bbl@ens@exclude\@empty
966 \def\bbl@ens@fontenc{\relax}%
967 \def\bbl@tempb##1{%
968     \ifx\@empty##1\else\noexpand##1\expandafter\bbl@tempb\fi}%
969 \edef\bbl@tempa{\bbl@tempb#1\@empty}%
970 \def\bbl@tempb##1=##2\@@{\@namedef\bbl@ens@##1}{##2}}%
971 \bbl@foreach\bbl@tempa{\bbl@tempb##1\@@}%
972 \def\bbl@tempc{\bbl@ensure}%
973 \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
974     \expandafter\bbl@ens@include}%
975 \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
976     \expandafter\bbl@ens@exclude}%
977 \toks@{\expandafter\bbl@tempc}%
978 \bbl@exp{%
979 \endgroup
980 \def<\bbl@e@#2>{\the\toks@{\bbl@ens@fontenc}}}%
981 \def\bbl@ensure#1#2#3% 1: include 2: exclude 3: fontenc
982 \def\bbl@tempb##1{% elt for (excluding) \bbl@captionslist list
983     \ifx##1\@undefined % 3.32 - Don't assume the macro exists
984         \edef##1{\noexpand\bbl@nocaption
985             {\bbl@stripslash##1}{\language\bbl@stripslash##1}}%
986     \fi
987     \ifx##1\@empty\else
988         \in@{##1}{#2}%
989         \ifin@else
990             \let\bbl@tempc\language
991             \bbl@replace\bbl@tempc{-}{@}%
992             \bbl@ifunset\bbl@ensure@\bbl@tempc}%
993             {\bbl@exp{%
994                 \\\DeclareRobustCommand\<\bbl@ensure@\bbl@tempc>[1]{%
995                     \\\foreignlanguage{\language}%
996                     {\ifx\relax#3\else
997                         \\\fontencoding{#3}\selectfont
998                         \fi
999                     #####1}}}%
1000             }%
1001             \toks@{\expandafter{##1}%
1002             \edef##1{%
1003                 \bbl@csarg\noexpand{ensure@\bbl@tempc}%
1004                 {\the\toks@}}}%
1005         \fi
1006         \expandafter\bbl@tempb
1007     \fi}%
1008 \expandafter\bbl@tempb\bbl@captionslist\today\@empty
1009 \def\bbl@tempa##1{% elt for include list
1010     \ifx##1\@empty\else
1011         \let\bbl@tempc\language
1012         \bbl@replace\bbl@tempc{-}{@}%
1013         \bbl@csarg\in@{ensure@\bbl@tempc\expandafter}\expandafter{##1}%
1014         \ifin@else
1015             \bbl@tempb##1\@empty
1016         \fi
1017         \expandafter\bbl@tempa
1018     \fi}%
1019 \bbl@tempa#1\@empty}
1020 \def\bbl@captionslist{%
1021 \prefacename\refname\abstractname\bibname\chaptername\appendixname
1022 \contentsname\listfigurename\listtablename\indexname\figurename
1023 \tablename\partname\enclname\ccname\headtoname\pagename\seename

```

```
1024 \alsiname\proofname\glossaryname}
```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text⟨tag⟩` and `\⟨tag⟩`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```
1025 \bbl@trace{Short tags}
1026 \newcommand\babeltags[1]{%
1027   \edef\bbl@tempa{\zap@space#1 \@empty}%
1028   \def\bbl@tempb##1=##2\@{#%
1029     \edef\bbl@tempc{%
1030       \noexpand\newcommand
1031       \expandafter\noexpand\csname ##1\endcsname{%
1032         \noexpand\protect
1033         \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1034       \noexpand\newcommand
1035       \expandafter\noexpand\csname text##1\endcsname{%
1036         \noexpand\foreignlanguage{##2}}}}
1037   \bbl@tempc}%
1038   \bbl@for\bbl@tempa\bbl@tempa{%
1039     \expandafter\bbl@tempb\bbl@tempa\@{}}
```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on `language.dat`, but `babel` can be made compatible with this format easily.

```
1040 \bbl@trace{Compatibility with language.def}
1041 \ifx\directlua\@undefined\else
1042   \ifx\bbl@luapatterns\@undefined
1043     \input luababel.def
1044   \fi
1045 \fi
1046 \ifx\bbl@languages\@undefined
1047   \ifx\directlua\@undefined
1048     \openin1 = language.def
1049     \ifeof1
1050       \closein1
1051       \message{I couldn't find the file language.def}
1052     \else
1053       \closein1
1054       \begingroup
1055         \def\addlanguage#1#2#3#4#5{%
1056           \expandafter\ifx\csname lang@#1\endcsname\relax\else
1057             \global\expandafter\let\csname l@#1\expandafter\endcsname
1058             \csname lang@#1\endcsname
1059           \fi}%
1060         \def\uselanguage#1{%
1061           \input language.def
1062         \endgroup
1063       \fi
1064     \fi
1065     \chardef\l@english\zap
1066 \fi
```

\addto It takes two arguments, a *⟨control sequence⟩* and TeX-code to be added to the *⟨control sequence⟩*.

If the *⟨control sequence⟩* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```
1067 \def\addto#1#2{%
1068   \ifx#1\@undefined
```

```

1069 \def#1{#2}%
1070 \else
1071 \ifx#1\relax
1072 \def#1{#2}%
1073 \else
1074 {\toks@\expandafter{#1#2}%
1075 \xdef#1{\the\toks@}}%
1076 \fi
1077 \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1078 \bbl@trace{Hooks}
1079 \newcommand\AddBabelHook[3][ ]{%
1080 \bbl@iifunset\bbl@hk@#2}{\EnableBabelHook{#2}}}%
1081 \def\bbl@tempa##1,##2,##3\@empty{\def\bbl@tempb{##2}}%
1082 \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1083 \bbl@iifunset\bbl@ev@#2@#3@#1{%
1084 {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1085 {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1086 \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1087 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1088 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1089 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1090 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1091 \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1092 \def\bbl@elth##1{%
1093 \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}}%
1094 \bbl@cs{ev@#2@}%
1095 \ifx\language\@undefined\else % Test required for Plain (?)
1096 \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1097 \def\bbl@elth##1{%
1098 \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1@#3}}}%
1099 \bbl@cs{ev@#2@#1}%
1100 \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1101 \def\bbl@evargs{,% <- don't delete this comma
1102 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1103 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1104 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1105 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1106 beforestart=0,language=2,begindocument=1}
1107 \ifx\NewHook\@undefined\else % Test for Plain (?)
1108 \def\bbl@tempa#1=#2\@@{\NewHook{babel/#1}}
1109 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1110 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1111 \providecommand\PassOptionsToLocale[2]{%
1112 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit `\LdfInit` macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a ‘letter’ during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, ‘=’, because it is sometimes used in constructions with the `\let` primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to `\LdfInit` is a control sequence. We do that by looking at the first token after passing #2 through `string`. When it is equal to `\@backslashchar` we are dealing with a control sequence which we can compare with `\@undefined`.

If so, we call `\ldf@quit` to set the main language, restore the category code of the @-sign and call `\endinput`

When #2 was *not* a control sequence we construct one and compare it with `\relax`.

Finally we check `\originalTeX`.

```

1113 \bbl@trace{Macros for setting language files up}
1114 \def\bbl@ldfinit{%
1115   \let\bbl@screset\@empty
1116   \let\BabelStrings\bbl@opt@string
1117   \let\BabelOptions\@empty
1118   \let\BabelLanguages\relax
1119   \ifx\originalTeX\@undefined
1120     \let\originalTeX\@empty
1121   \else
1122     \originalTeX
1123   \fi}
1124 \def\LdfInit#1#2{%
1125   \chardef\atcatcode=\catcode`\@
1126   \catcode`\@=11\relax
1127   \chardef\eqcatcode=\catcode`\=
1128   \catcode`\==12\relax
1129   \ifpackagewith{babel}{ensureinfo=off}}}%
1130   {\ifx\InputIfFileExists\@undefined\else
1131     \bbl@ifunset{bbl@lname@#1}%
1132     {{\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1133       \def\language#1}%
1134       \bbl@id@assign
1135       \bbl@load@info{#1}}}%
1136     {}%
1137   \fi}%
1138   \expandafter\if\expandafter\@backslashchar
1139     \expandafter\@car\string#2\@nil
1140   \ifx#2\@undefined\else
1141     \ldf@quit{#1}%
1142   \fi
1143 \else
1144   \expandafter\ifx\csname#2\endcsname\relax\else
1145     \ldf@quit{#1}%
1146   \fi
1147 \fi
1148 \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1149 \def\ldf@quit#1{%
1150   \expandafter\main@language\expandafter{#1}%
1151   \catcode`\@=\atcatcode \let\atcatcode\relax
1152   \catcode`\==\eqcatcode \let\eqcatcode\relax
1153   \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the @-sign.

```

1154 \def\bbl@afterldf{%
1155   \bbl@afterlang
1156   \let\bbl@afterlang\relax
1157   \let\BabelModifiers\relax
1158   \let\bbl@screset\relax}%
1159 \def\ldf@finish#1{%
1160   \loadlocalcfg{#1}%
1161   \bbl@afterldf
1162   \expandafter\main@language\expandafter{#1}%
1163   \catcode`\@=\atcatcode \let\atcatcode\relax
1164   \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in `ltxex`.

```

1165 \@onlypreamble\LdfInit
1166 \@onlypreamble\ldf@quit
1167 \@onlypreamble\ldf@finish

```

`\main@language`

`\bbl@main@language` This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1168 \def\main@language#1{%
1169   \def\bbl@main@language{#1}%
1170   \let\language\name\bbl@main@language
1171   \let\localename\bbl@main@language
1172   \let\mainlocalename\bbl@main@language
1173   \bbl@id@assign
1174   \ifcase\bbl@engine\or
1175     \ifx\setattribute\undefined\else
1176       \setattribute\bbl@attr@locale\localeid
1177     \fi
1178   \fi
1179   \bbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1180 \def\bbl@beforestart{%
1181   \def\@nolanerr##1{%
1182     \bbl@carg\chardef{l@##1}\z@
1183     \bbl@warning{Undefined language '##1' in aux.\@Reported}}%
1184   \bbl@usehooks{beforestart}}%
1185   \global\let\bbl@beforestart\relax}
1186 \AtBeginDocument{%
1187   {\@nameuse\bbl@beforestart}}% Group!
1188   \if@filesw
1189     \providecommand\babel@aux[2]{}%
1190     \immediate\write\@mainaux{unexpanded{%
1191       \providecommand\babel@aux[2]{\global\let\babel@toc@gobbletwo}}}%
1192     \immediate\write\@mainaux{string\@nameuse\bbl@beforestart}}%
1193   \fi
1194   \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1195   \ifbbl@single % must go after the line above.
1196     \renewcommand\selectlanguage[1]{}%
1197     \renewcommand\foreignlanguage[2]{#2}%
1198     \global\let\babel@aux@gobbletwo % Also as flag
1199   \fi}

```

```

1200 %
1201 \ifcase\bbl@engine\or
1202   \AtBeginDocument{\pagedir\bodydir}
1203 \fi

A bit of optimization. Select in heads/feet the language only if necessary.

1204 \def\select@language@x#1{%
1205   \ifcase\bbl@select@type
1206     \bbl@ifsamestring\language#1\{\}\select@language{#1}}%
1207   \else
1208     \select@language{#1}%
1209 \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1210 \bbl@trace{Shorhands}
1211 \def\bbl@withactive#1#2{%
1212   \begingroup
1213     \lccode`~=#2\relax
1214     \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1215 \def\bbl@add@special#1{% 1:a macro like "\", \?, etc.
1216   \bbl@add\dospecials{\do#1}% test \@sanitize = \relax, for back. compat.
1217   \bbl@ifunset{\@sanitize}\{\bbl@add\@sanitize{\@makeother#1}}%
1218   \ifx\nfss@catcodes\undefined\else
1219     \begingroup
1220       \catcode`#1\active
1221       \nfss@catcodes
1222       \ifnum\catcode`#1=\active
1223         \endgroup
1224         \bbl@add\nfss@catcodes{\@makeother#1}%
1225       \else
1226         \endgroup
1227     \fi
1228 \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char<char>` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char<char>` by default (`<char>` being the character to be made active). Later its definition can be changed to expand to `\active@char<char>` by calling `\bbl@activate{<char>}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string'ed) character, `\<level>@group`, `\<level>@active` and `\<next-level>@active` (except in system).

```

1229 \def\bbl@active@def#1#2#3#4{%
1230   \@namedef{#3#1}{%
1231     \expandafter\ifx\csname#2@sh@#1@\endcsname\relax
1232       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1233     \else
1234       \bbl@afterfi\csname#2@sh@#1@\endcsname
1235     \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```

1236   \long\@namedef{#3@arg#1}##1{%
1237     \expandafter\ifx\csname#2@sh@#1@\string##1@\endcsname\relax
1238       \bbl@afterelse\csname#4#1\endcsname##1%
1239     \else
1240       \bbl@afterfi\csname#2@sh@#1@\string##1@\endcsname
1241     \fi}}%

```

\initiate@active@char calls \@initiate@active@char with 3 arguments. All of them are the same character with different catcodes: active, other (\string'ed) and the original one. This trick simplifies the code a lot.

```

1242 \def\initiate@active@char#1{%
1243   \bbl@ifunset{active@char\string#1}%
1244   {\bbl@withactive
1245     {\expandafter\@initiate@active@char\expandafter}#1\string#1}%
1246   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them \relax and preserving some degree of protection).

```

1247 \def\@initiate@active@char#1#2#3{%
1248   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1249   \ifx#1\@undefined
1250     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1251   \else
1252     \bbl@csarg\let{oridef@#2}#1%
1253     \bbl@csarg\edef{oridef@#2}{%
1254       \let\noexpand#1%
1255       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1256   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define \normal@char{char} to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example ') the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to "8000 *a posteriori*").

```

1257   \ifx#1#3\relax
1258     \expandafter\let\csname normal@char#2\endcsname#3%
1259   \else
1260     \bbl@info{Making #2 an active character}%
1261     \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1262     \@namedef{normal@char#2}{%
1263       \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1264   \else
1265     \@namedef{normal@char#2}{#3}%
1266   \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at \begin{document}. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of \bibitem for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1267   \bbl@restoreactive{#2}%
1268   \AtBeginDocument{%

```



```

1269 \catcode`#2\active
1270 \if@filesw
1271 \immediate\write\@mainaux{\catcode`\string#2\active}%
1272 \fi}%
1273 \expandafter\bbbl@add@special\csname#2\endcsname
1274 \catcode`#2\active
1275 \fi

```

Now we have set `\normal@char<char>`, we must define `\active@char<char>`, to be executed when the character is activated. We define the first level expansion of `\active@char<char>` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active<char>` to start the search of a definition in the user, language and system levels (or eventually `normal@char<char>`).

```

1276 \let\bbbl@tempa\@firstoftwo
1277 \if\string^#2%
1278 \def\bbbl@tempa{\noexpand\textormath}%
1279 \else
1280 \ifx\bbbl@mathnormal\@undefined\else
1281 \let\bbbl@tempa\bbbl@mathnormal
1282 \fi
1283 \fi
1284 \expandafter\edef\csname active@char#2\endcsname{%
1285 \bbbl@tempa
1286 {\noexpand\if@safe@actives
1287 \noexpand\expandafter
1288 \expandafter\noexpand\csname normal@char#2\endcsname
1289 \noexpand\else
1290 \noexpand\expandafter
1291 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1292 \noexpand\fi}%
1293 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1294 \bbbl@csarg\edef{doactive#2}{%
1295 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

$$\text{\active@prefix}\langle char \rangle \text{\normal@char}\langle char \rangle$$

(where `\active@char<char>` is *one* control sequence!).

```

1296 \bbbl@csarg\edef{active@#2}{%
1297 \noexpand\active@prefix\noexpand#1%
1298 \expandafter\noexpand\csname active@char#2\endcsname}%
1299 \bbbl@csarg\edef{normal@#2}{%
1300 \noexpand\active@prefix\noexpand#1%
1301 \expandafter\noexpand\csname normal@char#2\endcsname}%
1302 \bbbl@ncarg\let#1\bbbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn’t exist we check for a shorthand with an argument.

```

1303 \bbbl@active@def#2\user@group{user@active}{language@active}%
1304 \bbbl@active@def#2\language@group{language@active}{system@active}%
1305 \bbbl@active@def#2\system@group{system@active}{normal@char}%

```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ‘ ’ ends up in a heading `TeX` would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```

1306 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1307 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1308 \expandafter\edef\csname\user@group @sh#2@\string\protect\endcsname
1309 {\expandafter\noexpand\csname user@active#2\endcsname}%

```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (') active we need to change `\pr@ms` as well. Also, make sure that a single ' in math mode 'does the right thing'. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```

1310 \if\string'#2%
1311   \let\prim@s\bbl@prim@s
1312   \let\active@math@prime#1%
1313 \fi
1314 \bbl@usehooks{initiateactive}{\#1}{\#2}{\#3}}

```

The following package options control the behavior of shorthands in math mode.

```

1315 <<{*More package options}>> ≡
1316 \DeclareOption{math=active}{}
1317 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1318 <</More package options>>

```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```

1319 \@ifpackagewith{babel}{KeepShorthandsActive}%
1320 {\let\bbl@restoreactive\@gobble}%
1321 {\def\bbl@restoreactive#1{%
1322   \bbl@exp{%
1323     \\AfterBabelLanguage\\CurrentOption
1324     {\catcode`#1=\the\catcode`#1\relax}%
1325     \\AtEndOfPackage
1326     {\catcode`#1=\the\catcode`#1\relax}}}%
1327   \AtEndOfPackage{\let\bbl@restoreactive\@gobble}}

```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```

1328 \def\bbl@sh@select#1#2{%
1329   \expandafter\ifx\csname#1@sh@#2@sel\endcsname\relax
1330     \bbl@afterelse\bbl@scndcs
1331   \else
1332     \bbl@afterfi\csname#1@sh@#2@sel\endcsname
1333   \fi}

```

\active@prefix Used in the expansion of active characters has a function similar to `\OT1-cmd` in that it `\protects` the active character whenever `\protect` is *not* `\typeset@protect`. The `\@gobble` is needed to remove a token such as `\activechar`: (when the double colon was the active character to be dealt with). There are two definitions, depending of `\ifincsname` is available. If there is, the expansion will be more robust.

```

1334 \begingroup
1335 \bbl@ifunset{ifincsname}
1336 {\gdef\active@prefix#1{%
1337   \ifx\protect\@typeset@protect
1338   \else
1339     \ifx\protect\@unexpandable@protect
1340       \noexpand#1%
1341     \else
1342       \protect#1%
1343     \fi
1344     \expandafter\@gobble
1345   \fi}}
1346 {\gdef\active@prefix#1{%
1347   \ifincsname

```

```

1348     \string#1%
1349     \expandafter\@gobble
1350   \else
1351     \ifx\protect\@typeset@protect
1352     \else
1353       \ifx\protect\@unexpandable@protect
1354         \noexpand#1%
1355       \else
1356         \protect#1%
1357       \fi
1358     \expandafter\expandafter\expandafter\@gobble
1359   \fi
1360 \fi}}
1361 \endgroup

```

\if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch `@safe@actives` is available. The setting of this switch should be checked in the first level expansion of `\active@char⟨char⟩`. When this expansion mode is active (with `\@safe@activetrue`), something like “`13`”₁₃ becomes “`12`”₁₂ in an `\edef` (in other words, shorthands are `\string`’ed). This contrasts with `\protected@edef`, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with `\@safe@activefalse`).

```

1362 \newif\if@safe@actives
1363 \@safe@activefalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1364 \def\bbl@restore@actives{\if@safe@actives\@safe@activefalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like `\initiate@active@char`. The macro is used to change the definition of an active character to expand to `\active@char⟨char⟩` in the case of `\bbl@activate`, or `\normal@char⟨char⟩` in the case of `\bbl@deactivate`.

```

1365 \chardef\bbl@activated\z@
1366 \def\bbl@activate#1{%
1367   \chardef\bbl@activated\@ne
1368   \bbl@withactive{\expandafter\let\expandafter}#1%
1369   \csname bbl@active@string#1\endcsname}
1370 \def\bbl@deactivate#1{%
1371   \chardef\bbl@activated\tw@
1372   \bbl@withactive{\expandafter\let\expandafter}#1%
1373   \csname bbl@normal@string#1\endcsname}

```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```

1374 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1375 \def\bbl@scndcs#1#2{\csname#2\endcsname}

```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., `~` or “a”;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The `TEX` code in text mode, (2) the string for `hyperref`, (3) the `TEX` code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn’t discriminate the mode). This macro may be used in `ldf` files.

```

1376 \def\babel@texpdf#1#2#3#4{%

```

```

1377 \ifx\texorpdfstring\undefined
1378   \textormath{#1}{#3}%
1379 \else
1380   \texorpdfstring{\textormath{#1}{#3}}{#2}%
1381   % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1382 \fi}
1383 %
1384 \def\declare@shorthand#1#2{\@decl@short{#1}#2\@nil}
1385 \def\@decl@short#1#2#3\@nil#4{%
1386   \def\bbl@tempa{#3}%
1387   \ifx\bbl@tempa\@empty
1388     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1389     \bbl@ifunset{#1@sh@\string#2@}{}%
1390     {\def\bbl@tempa{#4}%
1391      \expandafter\ifx\csname#1@sh@\string#2@endcsname\bbl@tempa
1392      \else
1393        \bbl@info
1394          {Redefining #1 shorthand \string#2\\%
1395           in language \CurrentOption}%
1396      \fi}%
1397     \@namedef{#1@sh@\string#2@}{#4}%
1398   \else
1399     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1400     \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1401     {\def\bbl@tempa{#4}%
1402      \expandafter\ifx\csname#1@sh@\string#2@\string#3@endcsname\bbl@tempa
1403      \else
1404        \bbl@info
1405          {Redefining #1 shorthand \string#2\string#3\\%
1406           in language \CurrentOption}%
1407      \fi}%
1408     \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1409   \fi}

```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```

1410 \def\textormath{%
1411   \ifmmode
1412     \expandafter\@secondoftwo
1413   \else
1414     \expandafter\@firstoftwo
1415   \fi}

```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```

1416 \def\user@group{user}
1417 \def\language@group{english}
1418 \def\system@group{system}

```

\useshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```

1419 \def\useshorthands{%
1420   \@ifstar\bbl@usesh@s{\bbl@usesh@x{}}
1421   \def\bbl@usesh@s#1{%
1422     \bbl@usesh@x
1423     {\AddBabelHook{babel-sh-\string#1}{afterextras}}{\bbl@activate{#1}}}%
1424   {#1}}

```

```

1425 \def\bbl@usesh@x#1#2{%
1426   \bbl@ifshorthand{#2}%
1427   {\def\user@group{user}%
1428     \initiate@active@char{#2}%
1429     #1%
1430     \bbl@activate{#2}}%
1431   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\defineshorthand Currently we only support two groups of user level shorthands, named internally `user` and `user@(<language>)` (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of `\defineshorthand`) a new level is inserted for it (`user@generic`, done by `\bbl@set@user@generic`); we make also sure `{}` and `\protect` are taken into account in this new top level.

```

1432 \def\user@language@group{user@\language@group}
1433 \def\bbl@set@user@generic#1#2{%
1434   \bbl@ifunset{user@generic@active#1}%
1435   {\bbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1436     \bbl@active@def#1\user@group{user@generic@active}{\language@active}%
1437     \expandafter\edef\csname#2@sh@#1@\endcsname{%
1438       \expandafter\noexpand\csname normal@char#1\endcsname}%
1439     \expandafter\edef\csname#2@sh@#1@\string\protect@\endcsname{%
1440       \expandafter\noexpand\csname user@active#1\endcsname}%
1441     \@empty}
1442   \newcommand\defineshorthand[3][user]{%
1443     \edef\bbl@tempa{\zap@space#1 \@empty}%
1444     \bbl@for\bbl@tempb\bbl@tempa{%
1445       \if*\expandafter\@car\bbl@tempb\@nil
1446       \edef\bbl@tempb{user@\expandafter\@gobble\bbl@tempb}%
1447       \@expandtwoargs
1448       \bbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbl@tempb
1449     }
1450     \fi
1451     \declare@shorthand{\bbl@tempb}{#2}{#3}}

```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```

1451 \def\languageshorthands#1{%
1452   \bbl@ifsamestring{none}{#1}{}%
1453   \bbl@once{short-\localename-#1}{%
1454     \bbl@info{'\localename' activates '#1' shorthands.\Reported}}}%
1455   \def\language@group{#1}}

```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshorthands{"}{/}` is `\active@prefix /\active@char/`, so we still need to let the latter to `\active@char`.

```

1456 \def\aliasshorthand#1#2{%
1457   \bbl@ifshorthand{#2}%
1458   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1459     \ifx\document@notprerr
1460       \@notshorthand{#2}%
1461     \else
1462       \initiate@active@char{#2}%
1463       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1464       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1465       \bbl@activate{#2}%
1466     \fi
1467   \fi}%
1468   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\@notshorthand

```

1469 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}

```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```
1470 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1471 \DeclareRobustCommand*\shorthandoff{%
1472   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1473 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}
```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char` should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```
1474 \def\bbl@switch@sh#1#2{%
1475   \ifx#2\@nnil\else
1476     \bbl@ifunset{bbl@active@\string#2}%
1477     {\bbl@error{not-a-shorthand-b}{\string#2}}}%
1478     {\ifcase#1%   off, on, off*
1479       \catcode`#2\relax
1480       \or
1481       \catcode`#2\active
1482       \bbl@ifunset{bbl@shdef@\string#2}%
1483       {}%
1484       {\bbl@withactive{\expandafter\let\expandafter}#2%
1485         \csname bbl@shdef@\string#2\endcsname
1486         \bbl@csarg\let{shdef@\string#2}\relax}%
1487       \ifcase\bbl@activated\or
1488       \bbl@activate{#2}%
1489       \else
1490       \bbl@deactivate{#2}%
1491       \fi
1492       \or
1493       \bbl@ifunset{bbl@shdef@\string#2}%
1494       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1495       {}%
1496       \csname bbl@oricat@\string#2\endcsname
1497       \csname bbl@oridef@\string#2\endcsname
1498       \fi}%
1499   \bbl@afterfi\bbl@switch@sh#1%
1500 \fi}
```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```
1501 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1502 \def\bbl@putsh#1{%
1503   \bbl@ifunset{bbl@active@\string#1}%
1504   {\bbl@putsh@i#1\@empty\@nnil}%
1505   {\csname bbl@active@\string#1\endcsname}}
1506 \def\bbl@putsh@i#1#2\@nnil{%
1507   \csname\language@group @sh@\string#1@%
1508     \ifx\@empty#2\else\string#2@\fi\endcsname}
1509 %
1510 \ifx\bbl@opt@shorthands\@nnil\else
1511   \let\bbl@s@initiate@active@char\initiate@active@char
1512   \def\initiate@active@char#1{%
1513     \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1514   \let\bbl@s@switch@sh\bbl@switch@sh
1515   \def\bbl@switch@sh#1#2{%
1516     \ifx#2\@nnil\else
```

```

1517 \bbl@afterfi
1518 \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1519 \fi}
1520 \let\bbl@s@activate\bbl@activate
1521 \def\bbl@activate#1{%
1522 \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1523 \let\bbl@s@deactivate\bbl@deactivate
1524 \def\bbl@deactivate#1{%
1525 \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1526 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1527 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting `\prime` for each right quote in mathmode is `\prim@s`. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1528 \def\bbl@prim@s{%
1529 \prime\futurelet\@let@token\bbl@pr@m@s}
1530 \def\bbl@if@primes#1#2{%
1531 \ifx#1\@let@token
1532 \expandafter\@firstoftwo
1533 \else\ifx#2\@let@token
1534 \bbl@afterelse\expandafter\@firstoftwo
1535 \else
1536 \bbl@afterfi\expandafter\@secondoftwo
1537 \fi\fi}
1538 \begingroup
1539 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^
1540 \catcode`\'=12 \catcode`\"=\active \lccode`\"='\ '
1541 \lowercase{%
1542 \gdef\bbl@pr@m@s{%
1543 \bbl@if@primes" '%
1544 \pr@@@s
1545 {\bbl@if@primes*\^pr@@@t\egroup}}}
1546 \endgroup

```

Usually the `~` is active and expands to `\penalty\@M\_{}`. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character `~` as a start character for a shorthand, it is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when `~` is still a non-break space), and in some cases is inconvenient (if `~` has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```

1547 \initiate@active@char{~}
1548 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1549 \bbl@activate{~}

```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the `\f@encoding` macro. Therefore we define two macros here to store the position of the character in these encodings.

```

1550 \expandafter\def\csname OT1dqpos\endcsname{127}
1551 \expandafter\def\csname T1dqpos\endcsname{4}

```

When the macro `\f@encoding` is undefined (as it is in plain $\TeX$) we define it here to expand to OT1

```

1552 \ifx\f@encoding\undefined
1553 \def\f@encoding{OT1}
1554 \fi

```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro `\languageattribute` checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

```
1555 \bbl@trace{Language attributes}
1556 \newcommand\languageattribute[2]{%
1557   \def\bbl@tempc{#1}%
1558   \bbl@fixname\bbl@tempc
1559   \bbl@iflanguage\bbl@tempc{%
1560     \bbl@vforeach{#2}{%
```

To make sure each attribute is selected only once, we store the already selected attributes in `\bbl@known@attrs`. When that control sequence is not yet defined this attribute is certainly not selected before.

```
1561     \ifx\bbl@known@attrs\undefined
1562       \in@false
1563     \else
1564       \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1565     \fi
1566     \ifin@
1567       \bbl@warning{%
1568         You have more than once selected the attribute '##1'\%
1569         for language #1. Reported}%
1570     \else
```

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated $\TeX$ -code.

```
1571       \bbl@info{Activated '##1' attribute for\%
1572         '\bbl@tempc'. Reported}%
1573       \bbl@exp{%
1574         \\bbl@add@list\\bbl@known@attrs{\bbl@tempc-##1}}%
1575       \edef\bbl@tempa{\bbl@tempc-##1}%
1576       \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes%
1577       {\csname\bbl@tempc @attr##1\endcsname}%
1578       {\@attrerr{\bbl@tempc}{##1}}%
1579     \fi}}
1580 \@onlypreamble\languageattribute
```

The error text to be issued when an unknown attribute is selected.

```
1581 \newcommand*\@attrerr[2]{%
1582   \bbl@error{unknown-attribute}{#1}{#2}{}}
```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```
1583 \def\bbl@declare@ttribute#1#2#3{%
1584   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1585   \ifin@
1586     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1587   \fi
1588   \bbl@add@list\bbl@attributes{#1-#2}%
1589   \expandafter\def\csname#1@attr@#2\endcsname{#3}}
```


\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```

1590 \def\bbl@ifattributeset#1#2#3#4{%
1591   \ifx\bbl@known@attrs\@undefined
1592     \in@false
1593   \else
1594     \bbl@xin@{,#1-#2,}\bbl@known@attrs,%
1595   \fi
1596   \ifin@
1597     \bbl@afterelse#3%
1598   \else
1599     \bbl@afterfi#4%
1600   \fi}

```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```

1601 \def\bbl@ifknown@ttrib#1#2{%
1602   \let\bbl@tempa\@secondoftwo
1603   \bbl@loopx\bbl@tempb{#2}%
1604   \expandafter\in\expandafter{\expandafter\bbl@tempb,}\bbl@tempa,%
1605   \ifin@
1606     \let\bbl@tempa\@firstoftwo
1607   \else
1608     \fi}%
1609   \bbl@tempa}

```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```

1610 \def\bbl@clear@ttribs{%
1611   \ifx\bbl@attributes\@undefined\else
1612     \bbl@loopx\bbl@tempa{\bbl@attributes}%
1613     \expandafter\bbl@clear@ttrib\bbl@tempa.%
1614   \let\bbl@attributes\@undefined
1615   \fi}
1616 \def\bbl@clear@ttrib#1-#2.{%
1617   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1618 \AtBeginDocument{\bbl@clear@ttribs}

```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are *\relax*'ed.

\babel@savecnt

\babel@beginsave The initialization of a new save cycle: reset the counter to zero.

```

1619 \bbl@trace{Macros for saving definitions}
1620 \def\babel@beginsave{\babel@savecnt\z@}

    Before it's forgotten, allocate the counter and initialize all.

1621 \newcount\babel@savecnt
1622 \babel@beginsave

```

\babel@save

\babel@savevariable The macro `\babel@save<csname>` saves the current meaning of the control sequence `<csname>` to `\originalTeX` (which has to be expandable, i.e., you shouldn't let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable<variable>` saves the value of the variable. `<variable>` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```
1623 \def\babel@save#1{%
1624   \def\bbl@tempa{,{#1,}}% Clumsy, for Plain
1625   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1626     \expandafter\expandafter,\bbl@savedextras,}%
1627   \expandafter\in@\bbl@tempa
1628   \ifin@
1629     \bbl@add\bbl@savedextras{,{#1,}}%
1630     \bbl@carg\let\babel@number\babel@savecnt#1\relax
1631     \toks@{\expandafter{\originalTeX\let#1=}}%
1632     \bbl@exp{%
1633       \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1634     \advance\babel@savecnt@one
1635   \fi}
1636 \def\babel@savevariable#1{%
1637   \toks@{\expandafter{\originalTeX #1=}}%
1638   \bbl@exp{\def\\originalTeX{\the\toks@<\the#1\relax}}}
```

\bbl@redefine To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don't want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```
1639 \def\bbl@redefine#1{%
1640   \edef\bbl@tempa{\bbl@stripslash#1}%
1641   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1642   \expandafter\def\csname\bbl@tempa\endcsname}
1643 \@onlypreamble\bbl@redefine
```

\bbl@redefine@long This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```
1644 \def\bbl@redefine@long#1{%
1645   \edef\bbl@tempa{\bbl@stripslash#1}%
1646   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1647   \long\expandafter\def\csname\bbl@tempa\endcsname}
1648 \@onlypreamble\bbl@redefine@long
```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo_`. So it is necessary to check whether `\foo_` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo_`.

```
1649 \def\bbl@redefineroobust#1{%
1650   \edef\bbl@tempa{\bbl@stripslash#1}%
1651   \bbl@ifunset{\bbl@tempa\space}%
1652     {\expandafter\let\csname org@\bbl@tempa\endcsname#1%
1653       \bbl@exp{\def\\#1{\protect<\bbl@tempa\space>}}}%
1654     {\bbl@exp{\let<org@\bbl@tempa><\bbl@tempa\space>}}}%
1655   \@namedef{\bbl@tempa\space}
1656 \@onlypreamble\bbl@redefineroobust
```

4.11. French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have \frenchspacing in effect. Others don't want that. The command \bbl@frenchspacing switches it on when it isn't already in effect and \bbl@nonfrenchspacing switches it off if necessary.

```

1657 \def\bbl@frenchspacing{%
1658   \ifnum\the\sfcode`\.\=@m
1659     \let\bbl@nonfrenchspacing\relax
1660   \else
1661     \frenchspacing
1662     \let\bbl@nonfrenchspacing\nonfrenchspacing
1663   \fi}
1664 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with ini files. Here an auxiliary macro is defined, but the main part is in \babelprovide. This new method should be ideally the default one.

```

1665 \let\bbl@elt\relax
1666 \edef\bbl@fs@chars{%
1667   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1668   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1669   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1670 \def\bbl@pre@fs{%
1671   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1672   \edef\bbl@save@sfcodes{\bbl@fs@chars}}%
1673 \def\bbl@post@fs{%
1674   \bbl@save@sfcodes
1675   \edef\bbl@tempa{\bbl@cl{frspc}}%
1676   \edef\bbl@tempa{\expandafter\@car\bbl@tempa\@nil}%
1677   \if u\bbl@tempa      % do nothing
1678   \else\if n\bbl@tempa % non french
1679     \def\bbl@elt##1##2##3{%
1680       \ifnum\sfcode`##1=##2\relax
1681         \babel@savevariable{\sfcode`##1}%
1682         \sfcode`##1=##3\relax
1683       \fi}%
1684     \bbl@fs@chars
1685   \else\if y\bbl@tempa % french
1686     \def\bbl@elt##1##2##3{%
1687       \ifnum\sfcode`##1=##3\relax
1688         \babel@savevariable{\sfcode`##1}%
1689         \sfcode`##1=##2\relax
1690       \fi}%
1691     \bbl@fs@chars
1692   \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: \bbl@hyphenation@ for the global ones and \bbl@hyphenation@<language> for language ones. See \bbl@patterns above for further details. We make sure there is a space between words when multiple commands are used.

```

1693 \bbl@trace{Hyphens}
1694 \@onlypreamble\babelhyphenation
1695 \AtEndOfPackage{%
1696   \newcommand\babelhyphenation[2][\@empty]{%
1697     \ifx\bbl@hyphenation@\relax
1698       \let\bbl@hyphenation@\@empty
1699     \fi
1700     \ifx\bbl@hyphlist\@empty\else
1701       \bbl@warning{%
1702         You must not intermingle \string\selectlanguage\space and\\%
1703         \string\babelhyphenation\space or some exceptions will not\\%
1704         be taken into account. Reported}%
1705       \fi

```

```

1706 \ifx\@empty#1%
1707 \protected@edef\bb@hyphenation@{\bb@hyphenation@\space#2}%
1708 \else
1709 \bb@vforeach{#1}{%
1710 \def\bb@tempa{##1}%
1711 \bb@fixname\bb@tempa
1712 \bb@iflanguage\bb@tempa{%
1713 \bb@csarg\protected@edef\hyphenation@{\bb@tempa}{%
1714 \bb@ifunset{\bb@hyphenation@\bb@tempa}%
1715 }%
1716 {\csname \bb@hyphenation@\bb@tempa\endcsname\space}%
1717 #2}}}%
1718 \fi}}

```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1719 \ifx\NewDocumentCommand\@undefined\else
1720 \NewDocumentCommand\babelhyphenmins{sommo}{%
1721 \IfNoValueTF{#2}%
1722 {\protected@edef\bb@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1723 \IfValueT{#5}{%
1724 \protected@edef\bb@hyphenatmin@{\hyphenationmin=#5\relax}}%
1725 \IfBooleanT{#1}{%
1726 \leftthyphenmin=#3\relax
1727 \rightthyphenmin=#4\relax
1728 \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1729 {\edef\bb@tempb{\zap@space#2 \@empty}%
1730 \bb@for\bb@tempa\bb@tempb{%
1731 \@namedef{\bb@hyphenmins@\bb@tempa}{\set@hyphenmins{#3}{#4}}}%
1732 \IfValueT{#5}{%
1733 \@namedef{\bb@hyphenatmin@\bb@tempa}{\hyphenationmin=#5\relax}}}%
1734 \IfBooleanT{#1}{\bb@error{hyphenmins-args}{}}}%
1735 \fi

```

\bb@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than `\nobreak\hskip 0pt plus 0pt`. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1736 \def\bb@allowhyphens{\ifvmode\else\nobreak\hskip\zap@space\fi}
1737 \def\bb@t@one{T1}
1738 \def\allowhyphens{\ifx\cf@encoding\bb@t@one\else\bb@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in `\babelhyphen`. Instead of protecting it with `\DeclareRobustCommand`, which could insert a `\relax`, we use the same procedure as shorthands, with `\active@` prefix.

```

1739 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1740 \def\babelhyphen{\active@prefix\babelhyphen\bb@hyphen}
1741 \def\bb@hyphen{%
1742 \@ifstar{\bb@hyphen@i @}{\bb@hyphen@i \@empty}}
1743 \def\bb@hyphen@i#1#2{%
1744 \lowercase{\bb@ifunset{\bb@hy@#1#2\@empty}}}%
1745 {\csname \bb@lusehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1746 {\lowercase{\csname \bb@hy@#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. `\nobreak` is always preceded by `\leavevmode`, in case the shorthand starts a paragraph.

```

1747 \def\bb@usehyphen#1{%
1748 \leavevmode

```

```

1749 \ifdim\lastskip>\z@\mbox{#1}\else\nobreak#1\fi
1750 \nobreak\hskip\z@skip}
1751 \def\bbl@usehyphen#1{%
1752 \leavevmode\ifdim\lastskip>\z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1753 \def\bbl@hyphenchar{%
1754 \ifnum\hyphenchar\font=\m@ne
1755 \babe\nullhyphen
1756 \else
1757 \char\hyphenchar\font
1758 \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in `ldf`’s. After a space, the `\mbox` in `\bbl@hy@nobreak` is redundant.

```

1759 \def\bbl@hy@soft{\bbl@usehyphen\discretionary{\bbl@hyphenchar}{}}{}{}
1760 \def\bbl@hy@@soft{\bbl@usehyphen\discretionary{\bbl@hyphenchar}{}}{}{}
1761 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1762 \def\bbl@hy@@hard{\bbl@usehyphen\bbl@hyphenchar}
1763 \def\bbl@hy@nobreak{\bbl@usehyphen\mbox{\bbl@hyphenchar}}
1764 \def\bbl@hy@@nobreak{\mbox{\bbl@hyphenchar}}
1765 \def\bbl@hy@repeat{%
1766 \bbl@usehyphen{
1767 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1768 \def\bbl@hy@@repeat{%
1769 \bbl@usehyphen{
1770 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1771 \def\bbl@hy@empty{\hskip\z@skip}
1772 \def\bbl@hy@@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro `\bbl@disc` is used to ease the insertion of discretionary for letters that behave ‘abnormally’ at a breakpoint.

```

1773 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by `luatex` and `xetex`. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1774 \bbl@trace{Multiencoding strings}
1775 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated `\SetCase`.

```

1776 <<*More package options>> ≡
1777 \DeclareOption{nocase}{}
1778 <</More package options>>

```

The following package options control the behavior of `\SetString`.

```

1779 <<*More package options>> ≡
1780 \let\bbl@opt@strings\@nnil % accept strings=value
1781 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1782 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1783 \def\BabelStringsDefault{generic}
1784 <</More package options>>

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1785 \@onlypreamble\StartBabelCommands
1786 \def\StartBabelCommands{%
1787   \begingroup
1788   \@tempcnta="7F
1789   \def\bbl@tempa{%
1790     \ifnum\@tempcnta>"FF\else
1791       \catcode\@tempcnta=11
1792       \advance\@tempcnta\@ne
1793       \expandafter\bbl@tempa
1794     \fi}%
1795   \bbl@tempa
1796   <@Macros local to BabelCommands@>
1797   \def\bbl@provstring##1##2{%
1798     \providecommand##1{##2}%
1799     \bbl@tglobal##1}%
1800   \global\let\bbl@scafter\@empty
1801   \let\StartBabelCommands\bbl@startcmds
1802   \ifx\BabelLanguages\relax
1803     \let\BabelLanguages\CurrentOption
1804   \fi
1805   \begingroup
1806   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1807   \StartBabelCommands}
1808 \def\bbl@startcmds{%
1809   \ifx\bbl@screset\@nnil\else
1810     \bbl@usehooks{stopcommands}{}%
1811   \fi
1812   \endgroup
1813   \begingroup
1814   \@ifstar
1815     {\ifx\bbl@opt@strings\@nnil
1816       \let\bbl@opt@strings\BabelStringsDefault
1817     \fi
1818     \bbl@startcmds@i}%
1819   \bbl@startcmds@i}
1820 \def\bbl@startcmds@i##1##2{%
1821   \edef\bbl@L{\zap@space#1 \@empty}%
1822   \edef\bbl@G{\zap@space#2 \@empty}%
1823   \bbl@startcmds@ii}
1824 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of \SetString. There are two main cases, depending of if there is an optional argument: without it and strings=encoded, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and strings=encoded, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in strings=) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1825 \newcommand\bbl@startcmds@ii[[1]][@empty]{%
1826   \let\SetString@gobbletwo
1827   \let\bbl@stringdef@gobbletwo
1828   \let\AfterBabelCommands@gobble
1829   \ifx\@empty#1%
1830     \def\bbl@sc@label{generic}%
1831     \def\bbl@encstring##1##2{%
1832       \ProvideTextCommandDefault##1{##2}%
1833       \bbl@tglobal##1%
1834       \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%

```

```

1835 \let\bbl@sctest\in@true
1836 \else
1837 \let\bbl@sc@charset\space % <- zapped below
1838 \let\bbl@sc@fontenc\space % <- " "
1839 \def\bbl@tempa##1=##2\@nil{%
1840 \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1841 \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1842 \def\bbl@tempa##1 ##2{% space -> comma
1843 ##1%
1844 \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1845 \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1846 \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1847 \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1848 \def\bbl@encstring##1##2{%
1849 \bbl@foreach\bbl@sc@fontenc{%
1850 \bbl@ifunset{T@####1}%
1851 {}%
1852 {\ProvideTextCommand##1{####1}{##2}%
1853 \bbl@tglobal##1%
1854 \expandafter
1855 \bbl@tglobal\csname####1\string##1\endcsname}}}%
1856 \def\bbl@sctest{%
1857 \bbl@xin@{\bbl@opt@strings,}{,\bbl@sc@label,\bbl@sc@fontenc,}%
1858 \fi
1859 \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1860 \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1861 \let\AfterBabelCommands\bbl@aftercmds
1862 \let\SetString\bbl@setstring
1863 \let\bbl@stringdef\bbl@encstring
1864 \else % i.e., strings=value
1865 \bbl@sctest
1866 \ifin@
1867 \let\AfterBabelCommands\bbl@aftercmds
1868 \let\SetString\bbl@setstring
1869 \let\bbl@stringdef\bbl@provstring
1870 \fi\fi\fi
1871 \bbl@scswitch
1872 \ifx\bbl@G\@empty
1873 \def\SetString##1##2{%
1874 \bbl@error{missing-group}{##1}{}}}%
1875 \fi
1876 \ifx\@empty#1%
1877 \bbl@usehooks{defaultcommands}{}%
1878 \else
1879 \@expandtwoargs
1880 \bbl@usehooks{encodedcommands}{\bbl@sc@charset}\bbl@sc@fontenc}%
1881 \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\(group)(language)` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date(language)` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has been loaded).

```

1882 \def\bbl@forlang#1#2{%
1883 \bbl@for#1\bbl@L{%
1884 \bbl@xin@{,##1,}{,\BabelLanguages,}%
1885 \ifin@#2\relax\fi}}
1886 \def\bbl@scswitch{%
1887 \bbl@forlang\bbl@tempa{%
1888 \ifx\bbl@G\@empty\else

```

```

1889 \ifx\SetString@gobbletwo\else
1890 \edef\bbl@GL{\bbl@G\bbl@tempa}%
1891 \bbl@xin@{\bbl@GL,}{\bbl@screset,}%
1892 \ifin@else
1893 \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1894 \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1895 \fi
1896 \fi
1897 \fi}}
1898 \AtEndOfPackage{%
1899 \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1900 \let\bbl@scswitch\relax}
1901 \@onlypreamble\EndBabelCommands
1902 \def\EndBabelCommands{%
1903 \bbl@usehooks{stopcommands}{}%
1904 \endgroup
1905 \endgroup
1906 \bbl@scafter}
1907 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1908 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1909 \bbl@forlang\bbl@tempa{%
1910 \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1911 \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1912 {\bbl@exp{%
1913 \global\bbbl@add\<\bbl@G\bbl@tempa>{\bbbl@scset\#1\<\bbl@LC>}}}%
1914 }%
1915 \def\BabelString{#2}%
1916 \bbl@usehooks{stringprocess}{}%
1917 \expandafter\bbl@stringdef
1918 \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1919 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1920 << *Macros local to BabelCommands >> ≡
1921 \def\SetStringLoop##1##2{%
1922 \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1923 \count@\z@
1924 \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1925 \advance\count@\@ne
1926 \toks@\expandafter{\bbl@tempa}%
1927 \bbl@exp{%
1928 \\\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%
1929 \count@=\the\count@\relax}}}%
1930 << /Macros local to BabelCommands >>

```

Delaying code Now the definition of \AfterBabelCommands when it is activated.

```

1931 \def\bbl@aftercmds#1{%
1932 \toks@\expandafter{\bbl@scafter#1}%
1933 \xdef\bbl@scafter{\the\toks@}}

```


Case mapping The command `\SetCase` is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1934 <<*Macros local to BabelCommands>> ≡
1935   \newcommand\SetCase[3][]{%
1936     \def\bbl@tempa####1####2{%
1937       \ifx####1\@empty\else
1938         \bbl@carg\bbl@add{extras\CurrentOption}{%
1939           \bbl@carg\babel@save{c__text_uppercase\_string####1_tl}%
1940           \bbl@carg\def{c__text_uppercase\_string####1_tl}{####2}%
1941           \bbl@carg\babel@save{c__text_lowercase\_string####2_tl}%
1942           \bbl@carg\def{c__text_lowercase\_string####2_tl}{####1}}%
1943         \expandafter\bbl@tempa
1944       \fi}%
1945     \bbl@tempa##1\@empty\@empty
1946     \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1947 <</Macros local to BabelCommands>>

```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1948 <<*Macros local to BabelCommands>> ≡
1949   \newcommand\SetHyphenMap[1]{%
1950     \bbl@forlang\bbl@tempa{%
1951       \expandafter\bbl@stringdef
1952       \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1953 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1954 \newcommand\BabelLower[2]{% one to one.
1955   \ifnum\lccode#1=#2\else
1956     \babel@savevariable{\lccode#1}%
1957     \lccode#1=#2\relax
1958   \fi}
1959 \newcommand\BabelLowerMM[4]{% many-to-many
1960   \@tempcnta=#1\relax
1961   \@tempcntb=#4\relax
1962   \def\bbl@tempa{%
1963     \ifnum\@tempcnta>#2\else
1964       \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1965       \advance\@tempcnta#3\relax
1966       \advance\@tempcntb#3\relax
1967       \expandafter\bbl@tempa
1968     \fi}%
1969   \bbl@tempa}
1970 \newcommand\BabelLowerM0[4]{% many-to-one
1971   \@tempcnta=#1\relax
1972   \def\bbl@tempa{%
1973     \ifnum\@tempcnta>#2\else
1974       \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1975       \advance\@tempcnta#3
1976       \expandafter\bbl@tempa
1977     \fi}%
1978   \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1979 <<*More package options>> ≡
1980 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1981 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap\@ne}
1982 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1983 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@@}
1984 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1985 <</More package options>>

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1986 \AtEndOfPackage{%
1987   \ifx\bbbl@opt@hyphenmap\@undefined
1988     \bbbl@xin@{,}\bbbl@language@opts}%
1989     \chardef\bbbl@opt@hyphenmap\ifin@4\else\@ne\fi
1990   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1991 \newcommand\setlocalecaption{%
1992   \ifstar\bbbl@setcaption@s\bbbl@setcaption@x}
1993 \def\bbbl@setcaption@x#1#2#3{% language caption-name string
1994   \bbbl@trim@def\bbbl@tempa{#2}%
1995   \bbbl@xin@{.template}\bbbl@tempa}%
1996   \ifin@
1997     \bbbl@ini@captions@template{#3}{#1}%
1998   \else
1999     \edef\bbbl@tempd{%
2000       \expandafter\expandafter\expandafter
2001       \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
2002     \bbbl@xin@
2003       {\expandafter\string\csname #2name\endcsname}%
2004       {\bbbl@tempd}%
2005     \ifin@ % Renew caption
2006       \bbbl@xin@{\string\bbbl@scset}\bbbl@tempd}%
2007     \ifin@
2008       \bbbl@exp{%
2009         \\bbbl@ifsamestring{\bbbl@tempa}\language}%
2010         {\\bbbl@scset\<#2name>\<#1#2name>}%
2011         {}}%
2012       \else % Old way converts to new way
2013         \bbbl@ifunset{#1#2name}%
2014         {\bbbl@exp{%
2015           \\bbbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2016           \\bbbl@ifsamestring{\bbbl@tempa}\language}%
2017           {\def\<#2name>\<#1#2name>}}%
2018           {}}}%
2019       {}%
2020     \fi
2021   \else
2022     \bbbl@xin@{\string\bbbl@scset}\bbbl@tempd}% New
2023     \ifin@ % New way
2024       \bbbl@exp{%
2025         \\bbbl@add\<captions#1>{\\bbbl@scset\<#2name>\<#1#2name>}}%
2026         \\bbbl@ifsamestring{\bbbl@tempa}\language}%
2027         {\\bbbl@scset\<#2name>\<#1#2name>}}%
2028         {}}%
2029       \else % Old way, but defined in the new way
2030       \bbbl@exp{%
2031         \\bbbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2032         \\bbbl@ifsamestring{\bbbl@tempa}\language}%
2033         {\def\<#2name>\<#1#2name>}}%
2034         {}}%
2035       \fi%
2036     \fi
2037     \@namedef{#1#2name}{#3}%
2038     \toks@ \expandafter\bbbl@captionslist}%
2039     \bbbl@exp{\in@{\<#2name>}\the\toks@}%
2040     \ifin@ \else
2041       \bbbl@exp{\\bbbl@add\\bbbl@captionslist{\<#2name>}}%

```

```

2042      \bbl@tglobal\bbl@captionslist
2043      \fi
2044      \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through `Tlenc.def`.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2045 \bbl@trace{Macros related to glyphs}
2046 \def\set@low@box#1{\setbox\tw@hbox{,}\setbox\z@hbox{#1}%
2047      \dimen\z@ht\z@ \advance\dimen\z@ -\ht\tw@%
2048      \setbox\z@hbox{\lower\dimen\z@ \box\z@}\ht\z@ht\tw@ \dp\z@dp\tw@}

```

\save@sf@q The macro `\save@sf@q` is used to save and reset the current space factor.

```

2049 \def\save@sf@q#1{\leavevmode
2050      \begingroup
2051      \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2052      \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via `\quotedblbase`. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2053 \ProvideTextCommand{\quotedblbase}{OT1}{%
2054      \save@sf@q{\set@low@box{\textquotedblright}/}%
2055      \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2056 \ProvideTextCommandDefault{\quotedblbase}{%
2057      \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2058 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2059      \save@sf@q{\set@low@box{\textquoteright}/}%
2060      \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2061 \ProvideTextCommandDefault{\quotesinglbase}{%
2062      \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```

2063 \ProvideTextCommand{\guillemetleft}{OT1}{%
2064      \ifmmode
2065          \ll
2066      \else
2067          \save@sf@q{\nobreak
2068              \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2069          \fi}
2070 \ProvideTextCommand{\guillemetright}{OT1}{%
2071      \ifmmode
2072          \gg
2073      \else
2074          \save@sf@q{\nobreak
2075              \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%

```

```

2076 \fi}
2077 \ProvideTextCommand{\guillemotleft}{OT1}{%
2078 \ifmmode
2079 \ll
2080 \else
2081 \save@sf@q{\nobreak
2082 \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2083 \fi}
2084 \ProvideTextCommand{\guillemotright}{OT1}{%
2085 \ifmmode
2086 \gg
2087 \else
2088 \save@sf@q{\nobreak
2089 \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2090 \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2091 \ProvideTextCommandDefault{\guillemetleft}{%
2092 \UseTextSymbol{OT1}{\guillemetleft}}
2093 \ProvideTextCommandDefault{\guillemetright}{%
2094 \UseTextSymbol{OT1}{\guillemetright}}
2095 \ProvideTextCommandDefault{\guillemotleft}{%
2096 \UseTextSymbol{OT1}{\guillemotleft}}
2097 \ProvideTextCommandDefault{\guillemotright}{%
2098 \UseTextSymbol{OT1}{\guillemotright}}

```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```

2099 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2100 \ifmmode
2101 <%
2102 \else
2103 \save@sf@q{\nobreak
2104 \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2105 \fi}
2106 \ProvideTextCommand{\guilsinglright}{OT1}{%
2107 \ifmmode
2108 >%
2109 \else
2110 \save@sf@q{\nobreak
2111 \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2112 \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2113 \ProvideTextCommandDefault{\guilsinglleft}{%
2114 \UseTextSymbol{OT1}{\guilsinglleft}}
2115 \ProvideTextCommandDefault{\guilsinglright}{%
2116 \UseTextSymbol{OT1}{\guilsinglright}}

```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter ‘ij’. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```

2117 \DeclareTextCommand{\ij}{OT1}{%
2118 i\kern-0.02em\bbl@allowhyphens j}
2119 \DeclareTextCommand{\IJ}{OT1}{%
2120 I\kern-0.02em\bbl@allowhyphens J}
2121 \DeclareTextCommand{\ij}{T1}{\char188}
2122 \DeclareTextCommand{\IJ}{T1}{\char156}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2123 \ProvideTextCommandDefault{\ij}{%
2124   \UseTextSymbol{OT1}{\ij}}
2125 \ProvideTextCommandDefault{\IJ}{%
2126   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2127 \def\crrtic@{\hrule height0.1ex width0.3em}
2128 \def\crttic@{\hrule height0.1ex width0.33em}
2129 \def\ddj@{%
2130   \setbox0\hbox{d}\dimen@=\ht0
2131   \advance\dimen@lex
2132   \dimen@.45\dimen@
2133   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2134   \advance\dimen@ii.5ex
2135   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2136 \def\DDJ@{%
2137   \setbox0\hbox{D}\dimen@=.55\ht0
2138   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2139   \advance\dimen@ii.15ex % correction for the dash position
2140   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2141   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2142   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crttic@}}}}
2143 %
2144 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2145 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2146 \ProvideTextCommandDefault{\dj}{%
2147   \UseTextSymbol{OT1}{\dj}}
2148 \ProvideTextCommandDefault{\DJ}{%
2149   \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2150 \DeclareTextCommand{\SS}{OT1}{SS}
2151 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```
2152 \ProvideTextCommandDefault{\glq}{%
2153   \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2154 \ProvideTextCommand{\grq}{T1}{%
2155   \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}}
2156 \ProvideTextCommand{\grq}{TU}{%
2157   \textormath{\textquoteleft}{\mbox{\textquoteleft}}}}
2158 \ProvideTextCommand{\grq}{OT1}{%
2159   \save@sf@q{\kern-.0125em
2160     \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%

```

```

2161 \kern.07em\relax}}
2162 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{0T1}\grq}

```

\glqq

\grqq The ‘german’ double quotes.

```

2163 \ProvideTextCommandDefault{\glqq}{%
2164 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}

```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```

2165 \ProvideTextCommand{\grqq}{T1}{%
2166 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2167 \ProvideTextCommand{\grqq}{TU}{%
2168 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2169 \ProvideTextCommand{\grqq}{0T1}{%
2170 \save@sf@q{\kern-.07em
2171 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%
2172 \kern.07em\relax}}
2173 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{0T1}\grqq}

```

\flq

\frq The ‘french’ single guillemets.

```

2174 \ProvideTextCommandDefault{\flq}{%
2175 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}
2176 \ProvideTextCommandDefault{\frq}{%
2177 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}

```

\flqq

\frqq The ‘french’ double guillemets.

```

2178 \ProvideTextCommandDefault{\flqq}{%
2179 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}
2180 \ProvideTextCommandDefault{\frqq}{%
2181 \textormath{\guillemetright}{\mbox{\guillemetright}}}

```

4.15.4. Umlauts and tremas

The command \~ needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of \~ we provide two commands to switch the positioning, the default will be \umlauthigh (the normal positioning).

```

2182 \def\umlauthigh{%
2183 \def\bbl@umlauta##1{\leavevmode\bgroup%
2184 \accent\csname\f@encoding dqpos\endcsname
2185 ##1\bbl@allowhyphens\egroup}%
2186 \let\bbl@umlaute\bbl@umlauta}
2187 \def\umlautlow{%
2188 \def\bbl@umlauta{\protect\lower@umlaut}}
2189 \def\umlautelow{%
2190 \def\bbl@umlaute{\protect\lower@umlaut}}
2191 \umlauthigh

```

\lower@umlaut Used to position the \" closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra *(dimen)* register.

```
2192 \expandafter\ifx\csname U@D\endcsname\relax
2193   \csname newdimen\endcsname\U@D
2194 \fi
```

The following code fools $\TeX$'s `make_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```
2195 \def\lower@umlaut#1{%
2196   \leavevmode\bgroup
2197   \U@D lex%
2198   {\setbox\z@\hbox{%
2199     \char\csname f@encoding dqpos\endcsname}%
2200     \dimen@ -.45ex\advance\dimen@\ht\z@
2201     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2202   \accent\csname f@encoding dqpos\endcsname
2203   \fontdimen5\font\U@D #1%
2204   \egroup}
```

For all vowels we declare \" to be a composite command which uses `\bbl@umlauta` or `\bbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbl@umlauta` and/or `\bbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```
2205 \AtBeginDocument{%
2206   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbl@umlauta{a}}%
2207   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbl@umlaute{e}}%
2208   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbl@umlaute{i}}%
2209   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbl@umlaute{\i}}%
2210   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbl@umlauta{o}}%
2211   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbl@umlauta{u}}%
2212   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbl@umlauta{A}}%
2213   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbl@umlaute{E}}%
2214   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbl@umlaute{I}}%
2215   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbl@umlauta{O}}%
2216   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbl@umlauta{U}}}
```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```
2217 \ifx\l@english\@undefined
2218   \chardef\l@english\z@
2219 \fi
2220 % The following is used to cancel rules in ini files (see Amharic).
2221 \ifx\l@unhyphenated\@undefined
2222   \newlanguage\l@unhyphenated
2223 \fi
```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```
2224 \bbl@trace{Bidi layout}
2225 \providecommand\IfBabelLayout[3]{#3}%
```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```
2226 \bbl@trace{Input engine specific macros}
2227 \ifcase\bbl@engine
2228   \input txtbabel.def
2229 \or
2230   \input luababel.def
2231 \or
2232   \input xebabel.def
2233 \fi
2234 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}
2235 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}
2236 \ifx\babelposthyphenation\undefined
2237   \let\babelposthyphenation\babelprehyphenation
2238   \let\babelpatterns\babelprehyphenation
2239   \let\babelcharproperty\babelprehyphenation
2240 \fi
2241 </package | core>
```

4.18. Creating and modifying languages

Continue with $\LaTeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded ldf files.

```
2242 <{*package}
2243 \bbl@trace{Creating languages and reading ini files}
2244 \let\bbl@extend@ini@gobble
2245 \newcommand\babelprovide[2][]{%
2246   \let\bbl@save@langname\language
2247   \edef\bbl@save@localeid{\the\localeid}%
2248   \global\let\bbl@afterload@empty
2249   % Set name and locale id
2250   \edef\language{#2}%
2251   \bbl@id@assign
2252   % Initialize keys
2253   \bbl@vforeach{captions,date,import,main,script,language,%
2254     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2255     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2256     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2257     @import}%
2258     {\bbl@csarg\let{KVP@##1}\@nnil}%
2259   \global\let\bbl@release@transforms@empty
2260   \global\let\bbl@release@casing@empty
2261   \let\bbl@calendars@empty
2262   \global\let\bbl@inidata@empty
2263   \global\let\bbl@extend@ini@gobble
2264   \global\let\bbl@included@inis@empty
2265   \gdef\bbl@key@list{;}%
2266   \bbl@ifunset{bbl@passto@#2}%
2267     {\def\bbl@tempa{#1}%
2268      {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2269      \expandafter\bbl@forkv\expandafter{\bbl@tempa}{%
2270        \in@/{/}{##1}% With /, (re)sets a value in the ini
2271        \ifin@
2272          \bbl@renewinikey##1\@@{##2}%
2273        \else
2274          \bbl@csarg\ifx{KVP@##1}\@nnil\else
2275            \bbl@error{unknown-provide-key}{##1}{}}{}%
2276        \fi
2277        \bbl@csarg\def{KVP@##1}{##2}%
```



```

2278 \fi}%
2279 \chardef\bbl@howloaded=% 0:none; 1:ldf without ini; 2:ini
2280 \bbl@ifunset{date#2}\z@{\bbl@ifunset{\bbl@llevel@#2}\@ne\tw}%
2281 % == init ==
2282 \ifx\bbl@screset\@undefined
2283 \bbl@ldfinit
2284 \fi
2285 % ==
2286 % If there is no import (last wins), use @import (internal, there
2287 % must be just one). To consider any order (because
2288 % \PassOptionsToLocale).
2289 \ifx\bbl@KVP@import\@nnil
2290 \let\bbl@KVP@import\bbl@KVP@@import
2291 \fi
2292 % == date (as option) ==
2293 % \ifx\bbl@KVP@date\@nnil\else
2294 % \fi
2295 % ==
2296 \let\bbl@lbfkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2297 \ifcase\bbl@howloaded
2298 \let\bbl@lbfkflag\@empty % new
2299 \else
2300 \ifx\bbl@KVP@hyphenrules\@nnil\else
2301 \let\bbl@lbfkflag\@empty
2302 \fi
2303 \ifx\bbl@KVP@import\@nnil\else
2304 \let\bbl@lbfkflag\@empty
2305 \fi
2306 \fi
2307 % == import, captions ==
2308 \ifx\bbl@KVP@import\@nnil\else
2309 \bbl@exp{\@bbl@ifblank{\bbl@KVP@import}}%
2310 {\ifx\bbl@initoload\relax
2311 \begingroup
2312 \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2313 \bbl@input@texini{##2}%
2314 \endgroup
2315 \else
2316 \xdef\bbl@KVP@import{\bbl@initoload}%
2317 \fi}%
2318 {}%
2319 \let\bbl@KVP@date\@empty
2320 \fi
2321 \let\bbl@KVP@captions@@\bbl@KVP@captions
2322 \ifx\bbl@KVP@captions\@nnil
2323 \let\bbl@KVP@captions\bbl@KVP@import
2324 \fi
2325 % ==
2326 \ifx\bbl@KVP@transforms\@nnil\else
2327 \bbl@replace\bbl@KVP@transforms{ }{,}%
2328 \fi
2329 % ==
2330 \ifx\bbl@KVP@mapdot\@nnil\else
2331 \def\bbl@tempa{\@empty}%
2332 \ifx\bbl@KVP@mapdot\bbl@tempa\else
2333 \bbl@exp{\gdef<\bbl@map@@.\@{\language\name}>{\bbl@KVP@mapdot}}}%
2334 \fi
2335 \fi
2336 % Load ini
2337 % -----
2338 \ifcase\bbl@howloaded
2339 \bbl@provide@new{#2}%
2340 \else

```

```

2341 \bbl@ifblank{#1}%
2342 {}% With \bbl@load@basic below
2343 {\bbl@provide@renew{#2}}%
2344 \fi
2345 % Post tasks
2346 % -----
2347 % == subsequent calls after the first provide for a locale ==
2348 \ifx\bbl@inidata\@empty\else
2349 \bbl@extend@ini{#2}%
2350 \fi
2351 % == ensure captions ==
2352 \ifx\bbl@KVP@captions\@nnil\else
2353 \bbl@ifunset{bbl@extracaps@#2}%
2354 {\bbl@exp{\bbl@babelensure[exclude=\\today]{#2}}}%
2355 {\bbl@exp{\bbl@babelensure[exclude=\\today,
2356 include=\[bbl@extracaps@#2]]{#2}}}%
2357 \let\bbl@tempc\languagename
2358 \bbl@replace\bbl@tempc{-}{@}%
2359 \bbl@ifunset{bbl@ensure@\bbl@tempc}%
2360 {\bbl@exp{%
2361 \\DeclareRobustCommand\<bbl@ensure@\bbl@tempc>[1]{%
2362 \\foreignlanguage{\languagename}%
2363 {###1}}}%
2364 }%
2365 \bbl@exp{%
2366 \\bbl@tglobal\<bbl@ensure@\bbl@tempc>%
2367 \\bbl@tglobal\<bbl@ensure@\bbl@tempc\space>%
2368 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2369 \bbl@load@basic{#2}%
2370 % == script, language ==
2371 % Override the values from ini or defines them
2372 \ifx\bbl@KVP@script\@nnil\else
2373 \bbl@csarg\edef{sname@#2}{\bbl@KVP@script}%
2374 \fi
2375 \ifx\bbl@KVP@language\@nnil\else
2376 \bbl@csarg\edef{lname@#2}{\bbl@KVP@language}%
2377 \fi
2378 \ifcase\bbl@engine\or
2379 \bbl@ifunset{bbl@chrng@\languagename}{}%
2380 {\directlua{
2381 Babel.set_chrngs_b('\bbl@cl{sbc}', '\bbl@cl{chrng}') }}%
2382 \fi
2383 % == Line breaking: intraspace, intrapenalty ==
2384 % For CJK, East Asian, Southeast Asian, if interspace in ini
2385 \ifx\bbl@KVP@intraspace\@nnil\else % We can override the ini or set
2386 \bbl@csarg\edef{intsp@#2}{\bbl@KVP@intraspace}%
2387 \fi
2388 \bbl@provide@intraspace
2389 % == Line breaking: justification ==
2390 \ifx\bbl@KVP@justification\@nnil\else
2391 \let\bbl@KVP@linebreaking\bbl@KVP@justification
2392 \fi
2393 \ifx\bbl@KVP@linebreaking\@nnil\else
2394 \bbl@xin@{,\bbl@KVP@linebreaking,}%
2395 {,elongated,kashida,cjk,padding,unhyphenated,}%
2396 \ifin@
2397 \bbl@csarg\xdef
2398 {\lnbrk@\languagename}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2399 \fi

```

```

2400 \fi
2401 \bbl@xin@{/e}/{/bbl@cl{lnbrk}}%
2402 \ifin@else\bbl@xin@{/k}/{/bbl@cl{lnbrk}}\fi
2403 \ifin@bbl@arabicjust\fi
2404 \bbl@xin@{/p}/{/bbl@cl{lnbrk}}%
2405 \ifin@AtBeginDocument{@nameuse{bbl@tibetanjust}}\fi
2406 % == Line breaking: hyphenate.other.(locale|script) ==
2407 \ifx\bbl@lbfkflag\empty
2408   \bbl@ifunset{bbl@hyotl@language\language}{}%
2409   {\bbl@csarg\bbl@replace{hyotl@language}{ }{,}%
2410     \bbl@startcommands*{language}{}%
2411     \bbl@csarg\bbl@foreach{hyotl@language}{%
2412       \ifcase\bbl@engine
2413         \ifnum##1<257
2414           \SetHyphenMap{\BabelLower{##1}{##1}}%
2415         \fi
2416       \else
2417         \SetHyphenMap{\BabelLower{##1}{##1}}%
2418       \fi}%
2419   \bbl@endcommands}%
2420 \bbl@ifunset{bbl@hyots@language\language}{}%
2421 {\bbl@csarg\bbl@replace{hyots@language}{ }{,}%
2422   \bbl@csarg\bbl@foreach{hyots@language}{%
2423     \ifcase\bbl@engine
2424       \ifnum##1<257
2425         \global\lccode##1=##1\relax
2426       \fi
2427     \else
2428       \global\lccode##1=##1\relax
2429     \fi}}%
2430 \fi
2431 % == Counters: maparabic ==
2432 % Native digits, if provided in ini (TeX level, xe and lua)
2433 \ifcase\bbl@engine\else
2434   \bbl@ifunset{bbl@dgnat@language\language}{}%
2435   {\expandafter\ifx\csname bbl@dgnat@language\endcsname\empty\else
2436     \expandafter\expandafter\expandafter
2437     \bbl@setdigits\csname bbl@dgnat@language\endcsname
2438     \ifx\bbl@KVP@maparabic\@nnil\else
2439       \ifx\bbl@latinarabic\@undefined
2440         \expandafter\let\expandafter\@arabic
2441         \csname bbl@counter@language\endcsname
2442       \else % i.e., if layout=counters, which redefines \@arabic
2443         \expandafter\let\expandafter\bbl@latinarabic
2444         \csname bbl@counter@language\endcsname
2445       \fi
2446     \fi
2447   \fi}%
2448 \fi
2449 % == Counters: mapdigits ==
2450 % > luababel.def
2451 % == Counters: alph, Alph ==
2452 \ifx\bbl@KVP@alph\@nnil\else
2453   \bbl@exp{%
2454     \\bbl@add<bbl@preextras@language>{%
2455       \\babel@save\\@alph
2456       \let\\@alph<bbl@cntr@bbl@KVP@alph @language>}}%
2457 \fi
2458 \ifx\bbl@KVP@Alph\@nnil\else
2459   \bbl@exp{%
2460     \\bbl@add<bbl@preextras@language>{%
2461       \\babel@save\\@Alph
2462       \let\\@Alph<bbl@cntr@bbl@KVP@Alph @language>}}%

```

```

2463 \fi
2464 % == Counters: mapdot ==
2465 \ifx\bbk@KVP@mapdot\@nnil\else
2466 \bbk@foreach\bbk@list@the{%
2467 \bbk@ifunset{the##1}{}%
2468 {{\bbk@ncarg\let\bbk@tempd{the##1}%
2469 \bbk@carg\bbk@sreplace{the##1}{.}{\bbk@map@lbl{.}}}%
2470 \expandafter\ifx\csname the##1\endcsname\bbk@tempd\else
2471 \bbk@exp{\gdef<the##1>{{\the##1}}}%
2472 \fi}}%
2473 \edef\bbk@tempb{enumi,enumii,enumiii,enumiv}%
2474 \bbk@foreach\bbk@tempb{%
2475 \bbk@ifunset{label##1}{}%
2476 {{\bbk@ncarg\let\bbk@tempd{label##1}%
2477 \bbk@carg\bbk@sreplace{label##1}{.}{\bbk@map@lbl{.}}}%
2478 \expandafter\ifx\csname label##1\endcsname\bbk@tempd\else
2479 \bbk@exp{\gdef<label##1>{{\label##1}}}%
2480 \fi}}%
2481 \fi
2482 % == Casing ==
2483 \bbk@release@casing
2484 \ifx\bbk@KVP@casing\@nnil\else
2485 \bbk@csarg\xdef{casing@{language}}%
2486 {\@nameuse{bbk@casing@{language}}\bbk@maybextx\bbk@KVP@casing}%
2487 \fi
2488 % == Calendars ==
2489 \ifx\bbk@KVP@calendar\@nnil
2490 \edef\bbk@KVP@calendar{\bbk@cl{calpr}}%
2491 \fi
2492 \def\bbk@tempe##1 ##2\@{ % Get first calendar
2493 \def\bbk@tempa{##1}}%
2494 \bbk@exp{\bbk@tempe\bbk@KVP@calendar\space\@}%
2495 \def\bbk@tempe##1.##2.##3\@{ %
2496 \def\bbk@tempc{##1}%
2497 \def\bbk@tempb{##2}}%
2498 \expandafter\bbk@tempe\bbk@tempa.\@
2499 \bbk@csarg\xdef{calpr@{language}}{%
2500 \ifx\bbk@tempc\@empty\else
2501 calendar=\bbk@tempc
2502 \fi
2503 \ifx\bbk@tempb\@empty\else
2504 ,variant=\bbk@tempb
2505 \fi}%
2506 % == engine specific extensions ==
2507 % Defined in XXXbabel.def
2508 \bbk@provide@extra{#2}%
2509 % == require.babel in ini ==
2510 % To load or reload the babel-*.tex, if require.babel in ini
2511 \ifx\bbk@beforestart\relax\else % But not in doc aux or body
2512 \bbk@ifunset{bbk@rqtex@{language}}{%
2513 \expandafter\ifx\csname bbk@rqtex@{language}\endcsname\@empty\else
2514 \let\BabelBeforeIni\gobbletwo
2515 \chardef\atcatcode=\catcode`\@
2516 \catcode`\@=11\relax
2517 \def\CurrentOption{#2}%
2518 \bbk@input@texini{\bbk@cs{rqtex@{language}}}%
2519 \catcode`\@=\atcatcode
2520 \let\atcatcode\relax
2521 \global\bbk@csarg\let{rqtex@{language}}\relax
2522 \fi}%
2523 \bbk@foreach\bbk@calendars{%
2524 \bbk@ifunset{bbk@ca@##1}{%
2525 \chardef\atcatcode=\catcode`\@

```

```

2526      \catcode\@=11\relax
2527      \InputIfFileExists{babel-ca-##1.tex}{}{}%
2528      \catcode\@=\atcatcode
2529      \let\atcatcode\relax}%
2530  {}}%
2531  \fi
2532  % == frenchspacing ==
2533  \ifcase\bbl@howloaded\in@true\else\in@false\fi
2534  \ifin@ \else \bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2535  \ifin@
2536    \bbl@extras@wrap{\bbl@pre@fs}%
2537    {\bbl@pre@fs}%
2538    {\bbl@post@fs}%
2539  \fi
2540  % == transforms ==
2541  % > luababel.def
2542  \def\CurrentOption{#2}%
2543  \@nameuse{\bbl@icsave@#2}%
2544  % == main ==
2545  \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2546    \let\languagename\bbl@savelangname
2547    \chardef\localeid\bbl@savelocaleid\relax
2548  \fi
2549  % == ==
2550  \ifx\ldf@finish\@onlypreamble\else
2551    \bbl@afterload
2552  \fi
2553  % == hyphenrules (apply if current) ==
2554  \ifx\bbl@KVP@hyphenrules\@nnil\else
2555    \ifnum\bbl@savelocaleid=\localeid
2556      \language\@nameuse{l@\languagename}%
2557    \fi
2558  \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2559 \def\bbl@provide@new#1{%
2560   \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2561   \@namedef{extras#1}{}%
2562   \@namedef{noextras#1}{}%
2563   \bbl@startcommands*{#1}{captions}%
2564   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2565     \def\bbl@tempb##1{% elt for \bbl@captionslist
2566       \ifx##1\@nnil\else
2567         \bbl@exp{%
2568           \\SetString\\##1{%
2569             \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2570         \expandafter\bbl@tempb
2571       \fi}%
2572     \expandafter\bbl@tempb\bbl@captionslist\@nnil
2573   \else
2574     \ifx\bbl@initoload\relax
2575       \bbl@read@ini{\bbl@KVP@captions}2% % Here letters cat = 11
2576     \else
2577       \bbl@read@ini{\bbl@initoload}2% % Same
2578     \fi
2579   \fi
2580   \StartBabelCommands*{#1}{date}%
2581   \ifx\bbl@KVP@date\@nnil
2582     \bbl@exp{%
2583       \\SetString\\today{\bbl@nocaption{today}{#1today}}}%
2584   \else
2585     \bbl@savetoday

```

```

2586     \bbl@savestate
2587     \fi
2588 \bbl@endcommands
2589 \bbl@load@basic{#1}%
2590 % == hyphenmins == (only if new)
2591 \bbl@exp{%
2592     \gdef\<#1hyphenmins>{%
2593         {\bbl@ifunset{\bbl@lfthm@#1}{2}{\bbl@cs{lfthm@#1}}}%
2594         {\bbl@ifunset{\bbl@rgthm@#1}{3}{\bbl@cs{rgthm@#1}}}%
2595 % == hyphenrules (also in renew) ==
2596 \bbl@provide@hyphens{#1}%
2597 % == main ==
2598 \ifx\bbl@KVP@main\@nnil\else
2599     \expandafter\main@language\expandafter{#1}%
2600 \fi}
2601 %
2602 \def\bbl@provide@renew#1{%
2603     \ifx\bbl@KVP@captions\@nnil\else
2604         \StartBabelCommands*{#1}{captions}%
2605         \bbl@read@ini{\bbl@KVP@captions}2%    % Here all letters cat = 11
2606         \EndBabelCommands
2607     \fi
2608     \ifx\bbl@KVP@date\@nnil\else
2609         \StartBabelCommands*{#1}{date}%
2610         \bbl@savestate
2611         \bbl@savestate
2612         \EndBabelCommands
2613     \fi
2614 % == hyphenrules (also in new) ==
2615 \ifx\bbl@lbkflag\@empty
2616     \bbl@provide@hyphens{#1}%
2617 \fi
2618 % == main ==
2619 \ifx\bbl@KVP@main\@nnil\else
2620     \expandafter\main@language\expandafter{#1}%
2621 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2622 \def\bbl@load@basic#1{%
2623     \ifcase\bbl@howloaded\or\or
2624         \ifcase\csname bbl@llevel@\language\endcsname
2625             \bbl@csarg\let\lname@\language\relax
2626         \fi
2627     \fi
2628     \bbl@ifunset{\bbl@lname@#1}%
2629     {\def\BabelBeforeIni##1##2{%
2630         \begingroup
2631             \let\bbl@ini@captions@aux\@gobbletwo
2632             \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6}%
2633             \bbl@read@ini{##1}1%
2634             \ifx\bbl@initoload\relax\endinput\fi
2635         \endgroup}%
2636     \begingroup      % boxed, to avoid extra spaces:
2637         \ifx\bbl@initoload\relax
2638             \bbl@input@texini{#1}%
2639         \else
2640             \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}}}%
2641         \fi
2642     \endgroup}%
2643     {}%

```

The following ini reader ignores everything but the identification section. It is called when a

font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2644 \def\bbl@load@info#1{%
2645   \def\BabelBeforeIni##1##2{%
2646     \begingroup
2647       \bbl@read@ini{##1}0%
2648       \endinput           % babel- .tex may contain onlypreamble's
2649       \endgroup}%         boxed, to avoid extra spaces:
2650   {\bbl@input@texini{#1}}}
```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2651 \def\bbl@provide@hyphens#1{%
2652   \@tempcnta\m@ne % a flag
2653   \ifx\bbl@KVP@hyphenrules\@nnil\else
2654     \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2655     \bbl@foreach\bbl@KVP@hyphenrules{%
2656       \ifnum\@tempcnta=\m@ne % if not yet found
2657         \bbl@ifsamestring{##1}{+}%
2658         {\bbl@carg\addlanguage{l@##1}}%
2659         }%
2660         \bbl@ifunset{l@##1}% After a possible +
2661         {}%
2662         {\@tempcnta\@nameuse{l@##1}}%
2663       \fi}%
2664   \ifnum\@tempcnta=\m@ne
2665     \bbl@warning{%
2666       Requested 'hyphenrules' for '\language' not found:\%
2667       \bbl@KVP@hyphenrules.\%
2668       Using the default value. Reported}%
2669   \fi
2670 \fi
2671 \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2672   \ifx\bbl@KVP@captions@\@nnil
2673     \bbl@ifunset\bbl@hyphr@#1{}% use value in ini, if exists
2674     {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2675      {}%
2676      {\bbl@ifunset{l@\bbl@cl{hyphr}}}%
2677      {}% if hyphenrules found:
2678      {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}}}%
2679   \fi
2680 \fi
2681 \bbl@ifunset{l@#1}%
2682   {\ifnum\@tempcnta=\m@ne
2683     \bbl@carg\adddialect{l@#1}\language
2684   \else
2685     \bbl@carg\adddialect{l@#1}\@tempcnta
2686   \fi}%
2687   {\ifnum\@tempcnta=\m@ne\else
2688     \global\bbl@carg\chardef{l@#1}\@tempcnta
2689   \fi}}
```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2690 \def\bbl@input@texini#1{%
2691   \bbl@bsphack
2692   \bbl@exp{%
2693     \catcode`\%%=14 \catcode`\%%=0
2694     \catcode`\{=1 \catcode`\}=2
2695     \lowercase{\InputIfFileExists{babel-#1.tex}{}}%
2696     \catcode`\%%=\the\catcode`\%\relax
2697     \catcode`\%%=\the\catcode`\%\relax
```

```

2698      \catcode`\\={\the\catcode`\}\relax
2699      \catcode`\\}= \the\catcode`\}\relax}%
2700      \bbl@esphack}

The following macros read and store ini files (but don't process them). For each line, there are 3
possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are
used in the first step of \bbl@read@ini.

2701 \def\bbl@iniline#1\bbl@iniline{%
2702   \ifnextchar[\bbl@inisect{\@ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2703 \def\bbl@inisect[#1]#2\@@{\def\bbl@section{#1}}
2704 \def\bbl@iniskip#1\@@{%      if starts with ;
2705 \def\bbl@inistore#1=#2\@@{%   full (default)
2706   \bbl@trim@def\bbl@tempa{#1}%
2707   \bbl@trim\toks@{#2}%
2708   \bbl@ifsamestring{\bbl@tempa}{\include}%
2709   {\bbl@read@subini{\the\toks@}}%
2710   {\bbl@xin@{;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2711    \ifin@ \else
2712      \bbl@xin@{,identification/include.}%
2713      {,\bbl@section/\bbl@tempa}%
2714      \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2715      \bbl@exp{%
2716        \\g@addto@macro\\bbl@inidata{%
2717          \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2718      \fi}}
2719 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2720   \bbl@trim@def\bbl@tempa{#1}%
2721   \bbl@trim\toks@{#2}%
2722   \bbl@xin@{.identification.}{.\bbl@section.}%
2723   \ifin@
2724     \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2725       \\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2726   \fi}

```

4.19. Main loop in ‘provide’

Now, the ‘main loop’, \bbl@read@ini, which **must be executed inside a group**. At this point, \bbl@inidata may contain data declared in \babelprovide, with ‘slashed’ keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, ‘export’ some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```

2727 \def\bbl@loop@ini#1{%
2728   \loop
2729     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2730     \endlinechar\m@ne
2731     \read#1 to \bbl@line
2732     \endlinechar`\^^M
2733     \ifx\bbl@line\empty\else
2734       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2735     \fi
2736   \repeat}
2737 %
2738 \def\bbl@read@subini#1{%
2739   \ifx\bbl@readsubstream\undefined
2740     \csname newread\endcsname\bbl@readsubstream
2741   \fi

```



```

2742 \openin\bbl@readsubstream=babel-#1.ini
2743 \ifeof\bbl@readsubstream
2744   \bbl@error{no-ini-file}{#1}{}{}%
2745 \else
2746   {\bbl@loop@ini\bbl@readsubstream}%
2747 \fi
2748 \closein\bbl@readsubstream}
2749 %
2750 \ifx\bbl@readstream\undefined
2751   \csname newread\endcsname\bbl@readstream
2752 \fi
2753 \def\bbl@read@ini#1#2{%
2754   \global\let\bbl@extend@ini@gobble
2755   \openin\bbl@readstream=babel-#1.ini
2756   \ifeof\bbl@readstream
2757     \bbl@error{no-ini-file}{#1}{}{}%
2758   \else
2759     % == Store ini data in \bbl@inidata ==
2760     \catcode\ =10 \catcode\ =12
2761     \catcode\[ =12 \catcode\] =12 \catcode\ =12 \catcode\& =12
2762     \catcode\; =12 \catcode\| =12 \catcode\% =14 \catcode\ - =12
2763     \ifnum#2=\m@ne % Just for the info
2764       \edef\language\{tag \bbl@metalang}%
2765     \fi
2766     \ifnum#2=\m@ne
2767       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.\\%
2768         Fetching the locale name from babel-#1.ini@gobble}%
2769     \else
2770       \bbl@info{Importing
2771         \ifcase#2font and identification \or basic \fi
2772         data for '\language'\\%
2773         from babel-#1.ini. Reported}%
2774     \fi
2775     \ifnum#2<\@ne
2776       \global\let\bbl@inidata\@empty
2777       \let\bbl@inistore\bbl@inistore@min % Remember it's local
2778     \fi
2779     \def\bbl@section{identification}%
2780     \bbl@exp{%
2781       \\ \bbl@inistore tag.ini=#1\\ \@
2782       \\ \bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\ \@}%
2783     \bbl@loop@ini\bbl@readstream
2784     % == Process stored data ==
2785     \ifnum#2=\m@ne
2786       \def\bbl@tempa##1 ##2\@{##1}% Get first name
2787       \def\bbl@elt###1##2###3{%
2788         \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2789         {\edef\language{\bbl@tempa##3 \@}%
2790          \let\localename\language
2791          \bbl@id@assign
2792          \def\bbl@elt####1####2####3{}}%
2793         {}}%
2794       \bbl@inidata
2795     \fi
2796     \bbl@csarg\xdef{lini@\language}{#1}%
2797     \bbl@read@ini@aux
2798     % == 'Export' data ==
2799     \bbl@ini@exports{#2}%
2800     \global\bbl@csarg\let{inidata@\language}\bbl@inidata
2801     \global\let\bbl@inidata\@empty
2802     \bbl@exp{\\ \bbl@add@list\\ \bbl@ini@loaded{\language}}%
2803     \bbl@tglobal\bbl@ini@loaded
2804     \@ifpackagewith{babel}{licr=unextended}{}%

```

```

2805     {\ifcase\bbl@engine\else % Find a better place
2806       \bbl@xin@{\,\bbl@cl{sbcpr},}{,CyrL,}%
2807       \ifin@
2808         \bbl@once{licr-cryl}{\g@addto@macro\bbl@afterload{\input{cyrLuni.def}}}%
2809       \fi
2810     \fi}%
2811 \fi
2812 \closein\bbl@readstream}
2813 \def\bbl@read@ini@aux{%
2814   \let\bbl@savestrings\@empty
2815   \let\bbl@savetoday\@empty
2816   \let\bbl@savestate\@empty
2817   \def\bbl@elt##1##2##3{%
2818     \def\bbl@section{##1}%
2819     \in@{=date.}{=##1}% Find a better place
2820     \ifin@
2821       \bbl@ifunset{bbl@inikv@##1}%
2822       {\bbl@ini@calendar{##1}}%
2823     }%
2824   \fi
2825   \bbl@ifunset{bbl@inikv@##1}{}%
2826   {\csname bbl@inikv@##1\endcsname{##2}{##3}}%
2827   \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2828 \def\bbl@extend@ini@aux#1{%
2829   \bbl@startcommands*{#1}{captions}%
2830   % Activate captions/... and modify exports
2831   \bbl@csarg\def{inikv@captions.licr}##1##2{%
2832     \setlocalecaption{#1}{##1}{##2}}%
2833   \def\bbl@inikv@captions##1##2{%
2834     \bbl@ini@captions@aux{##1}{##2}}%
2835   \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2836   \def\bbl@exportkey##1##2##3{%
2837     \bbl@ifunset{bbl@kv@##2}{}%
2838     {\expandafter\ifx\csname bbl@kv@##2\endcsname\@empty\else
2839       \bbl@exp{\global\let<bbl@##1\language>\<bbl@kv@##2>}}%
2840     \fi}%
2841   % As with \bbl@read@ini, but with some changes
2842   \bbl@read@ini@aux
2843   \bbl@ini@exports\tw@
2844   % Update inidata@lang by pretending the ini is read.
2845   \def\bbl@elt##1##2##3{%
2846     \def\bbl@section{##1}%
2847     \bbl@iniline##2=##3\bbl@iniline}%
2848     \csname bbl@inidata@#1\endcsname
2849     \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2850   \StartBabelCommands*{#1}{date}% And from the import stuff
2851   \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2852   \bbl@savetoday
2853   \bbl@savestate
2854   \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2855 \def\bbl@ini@calendar#1{%
2856   \lowercase{\def\bbl@tempa{=##1=}}%
2857   \bbl@replace\bbl@tempa{=date.gregorian}{}%
2858   \bbl@replace\bbl@tempa{=date.}{}%
2859   \in@{.licr=}{#1}%
2860   \ifin@
2861     \ifcase\bbl@engine
2862       \bbl@replace\bbl@tempa{.licr=}{}%
2863     \else

```

```

2864 \let\bbl@tempa\relax
2865 \fi
2866 \fi
2867 \ifx\bbl@tempa\relax\else
2868 \bbl@replace\bbl@tempa{=}{}%
2869 \ifx\bbl@tempa\@empty\else
2870 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2871 \fi
2872 \bbl@exp{%
2873 \def\<bbl@inikv@#1>####1####2{%
2874 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2875 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2876 \def\bbl@renewinikey#1/#2\@#3{%
2877 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2878 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2879 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2880 \bbl@trim\toks@{#3}% value
2881 \bbl@exp{%
2882 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2883 \\g@addto@macro\\bbl@inidata{%
2884 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2885 \def\bbl@exportkey#1#2#3{%
2886 \bbl@ifunset{\bbl@kv@#2}%
2887 {\bbl@csarg\gdef{#1@\language}\@empty}%
2888 {\expandafter\ifx\csname\bbl@kv@#2\endcsname\@empty
2889 \bbl@csarg\gdef{#1@\language}\@empty}%
2890 \else
2891 \bbl@exp{\global\let\<bbl@#1@\language>\<bbl@kv@#2>}%
2892 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bbl@ini@exports is called always (via \bbl@inisec), while \bbl@after@ini must be called explicitly after \bbl@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdftex to select a font encoding valid (and preloaded) for a language loaded on the fly.

```

2893 \def\bbl@iniwarning#1{%
2894 \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2895 {\bbl@warning{%
2896 From babel-\bbl@cs{lini@\language}.ini:\\%
2897 \bbl@cs{@kv@identification.warning#1}\\%
2898 Reported}}}
2899 %
2900 \let\bbl@release@transforms\@empty
2901 \let\bbl@release@casing\@empty

```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```

2902 \def\bbl@ini@exports#1{%
2903   % Identification always exported
2904   \bbl@iniwarning{}%
2905   \ifcase\bbl@engine
2906     \bbl@iniwarning{.pdflatex}%
2907   \or
2908     \bbl@iniwarning{.lualatex}%
2909   \or
2910     \bbl@iniwarning{.xelatex}%
2911   \fi%
2912   \bbl@exportkey{lllevel}{identification.load.level}{}%
2913   \bbl@exportkey{elname}{identification.name.english}{}%
2914   \bbl@exp{\bbl@exportkey{lname}{identification.name.opentype}%
2915     {\csname bbl@elname@\language\endcsname}}%
2916   \bbl@exportkey{tbcpr}{identification.tag.bcp47}{}%
2917   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2918   \bbl@exportkey{lbcpr}{identification.language.tag.bcp47}{}%
2919   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2920   \bbl@exportkey{esname}{identification.script.name}{}%
2921   \bbl@exp{\bbl@exportkey{sname}{identification.script.name.opentype}%
2922     {\csname bbl@esname@\language\endcsname}}%
2923   \bbl@exportkey{sbcpr}{identification.script.tag.bcp47}{}%
2924   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2925   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2926   \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2927   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2928   \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2929   \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2930   % Also maps bcp47 -> language
2931   \bbl@csarg\xdef{bcp@map@\bbl@cl{tbcpr}}{\language}%
2932   \ifcase\bbl@engine\or
2933     \directlua{%
2934       Babel.locale_props[\the\bbl@cs{id@\language}].script
2935       = '\bbl@cl{sbcpr}'}%
2936   \fi
2937   % Conditional
2938   \ifnum#1>\z@      % -1 or 0 = only info, 1 = basic, 2 = (re)new
2939     \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2940     \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2941     \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2942     \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%
2943     \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2944     \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2945     \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2946     \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2947     \bbl@exportkey{intsp}{typography.intraspaces}{}%
2948     \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2949     \bbl@exportkey{chrng}{characters.ranges}{}%
2950     \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2951     \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2952     \ifnum#1=\tw@      % only (re)new
2953       \bbl@exportkey{rqtex}{identification.require.babel}{}%
2954       \bbl@tglobal\bbl@savetoday
2955       \bbl@tglobal\bbl@savestate
2956       \bbl@savestrings
2957     \fi
2958   \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in `\bbl@kv@<section>.<key>`.

```

2959 \def\bbl@inikv#1#2{%      key=value
2960   \toks@{#2}%             This hides #'s from ini values

```

```
2961 \bbl@csarg\edef{@kv@bbl@section.#1}{\the\toks@}}
```

By default, the following sections are just read. Actions are taken later.

```
2962 \let\bbl@inikv@identification\bbl@inikv
2963 \let\bbl@inikv@date\bbl@inikv
2964 \let\bbl@inikv@typography\bbl@inikv
2965 \let\bbl@inikv@numbers\bbl@inikv
```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```
2966 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\languagename}\@empty x-\fi}
2967 \def\bbl@inikv@characters#1#2{%
2968   \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2969   {\bbl@exp{%
2970     \\g@addto@macro\\bbl@release@casing{%
2971       \\bbl@casemapping{ }\languagename}{\unexpanded{#2}}}%
2972   {\in@{ $casing. }{ $#1 }% e.g., casing.Uv = uV
2973     \ifin@
2974       \lowercase{\def\bbl@tempb{#1}}%
2975       \bbl@replace\bbl@tempb{casing.}{ }%
2976       \bbl@exp{ \\g@addto@macro\\bbl@release@casing{%
2977         \\bbl@casemapping
2978         { \\bbl@maybextx\bbl@tempb}{\languagename}{\unexpanded{#2}}}%
2979       \else
2980         \bbl@inikv{#1}{#2}%
2981       \fi}}
```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’.

```
2982 \def\bbl@inikv@counters#1#2{%
2983   \bbl@ifsamestring{#1}{digits}%
2984   {\bbl@error{digits-is-reserved}{ }{ }{ }%
2985     }%
2986   \def\bbl@tempc{#1}%
2987   \bbl@trim@def{\bbl@tempb*}{#2}%
2988   \in@{.1$}{#1$}%
2989   \ifin@
2990     \bbl@replace\bbl@tempc{.1}{ }%
2991     \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\languagename}{%
2992       \noexpand\bbl@alphanumeric{\bbl@tempc}}%
2993   \fi
2994   \in@{.F.}{#1}%
2995   \ifin@\else\in@{.S.}{#1}\fi
2996   \ifin@
2997     \bbl@csarg\protected@xdef{cntr@#1@\languagename}{\bbl@tempb*}%
2998   \else
2999     \toks@{% Required by \bbl@buildifcase, which returns \bbl@tempa
3000     \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \ \
3001     \bbl@csarg\global\expandafter\let}{cntr@#1@\languagename}\bbl@tempa
3002   \fi}
```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```
3003 \ifcase\bbl@engine
3004   \bbl@csarg\def{inikv@captions.licr}#1#2{%
3005     \bbl@ini@captions@aux{#1}{#2}}
3006 \else
3007   \def\bbl@inikv@captions#1#2{%
3008     \bbl@ini@captions@aux{#1}{#2}}
3009 \fi
```



```

3070 \def\bbl@inikv@labels#1#2{%
3071   \in@{.map}{#1}%
3072   \ifin@
3073     \in@{,dot.map,}{, #1,}%
3074     \ifin@
3075       \global\@namedef{bbl@map@.@@\language}{#2}%
3076       \fi
3077   \ifx\bbl@KVP@labels\@nnil\else
3078     \bbl@xin@{ map }{ \bbl@KVP@labels\space}%
3079     \ifin@
3080       \def\bbl@tempc{#1}%
3081       \bbl@replace\bbl@tempc{.map}{}%
3082       \in@{, #2,}{, arabic, roman, Roman, alph, Alph, fnsymbol,}%
3083       \bbl@exp{%
3084         \gdef\<bbl@map@\bbl@tempc @\language>%
3085           {\ifin@<#2>\else\\localecounter{#2}\fi}}%
3086       \bbl@foreach\bbl@list@the{%
3087         \bbl@ifunset{the##1}{}%
3088         {\bbl@ncarg\let\bbl@tempd{the##1}%
3089          \bbl@exp{%
3090            \\bbl@sreplace\<the##1>%
3091            {\<\bbl@tempc>{##1}}%
3092            {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3093            \\bbl@sreplace\<the##1>%
3094            {\<\@empty @\bbl@tempc>\<c@##1>%
3095             {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3096             \\bbl@sreplace\<the##1>%
3097             {\\\csname @\bbl@tempc\\endcsname\<c@##1>%
3098              {\\\bbl@map@cnt{\bbl@tempc}{##1}}}}%
3099          \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3100            \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3101            \fi}}%
3102       \fi
3103     \fi
3104 %
3105 \else
3106   % The following code is still under study. You can test it and make
3107   % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3108   % language dependent.
3109   \in@{enumerate.}{#1}%
3110   \ifin@
3111     \def\bbl@tempa{#1}%
3112     \bbl@replace\bbl@tempa{enumerate.}{}%
3113     \def\bbl@toreplace{#2}%
3114     \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}%
3115     \bbl@replace\bbl@toreplace{[]}{\csname the}%
3116     \bbl@replace\bbl@toreplace{[]}{\endcsname}}%
3117     \toks@{\expandafter{\bbl@toreplace}%
3118     \bbl@exp{%
3119       \\bbl@add\<extras\language>{%
3120         \\babel@save\<labelenum\romannumeral\bbl@tempa>%
3121         \def\<labelenum\romannumeral\bbl@tempa>\[the\toks@]}%
3122       \\bbl@tglobal\<extras\language>}%
3123     \fi
3124   \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3125 \def\bbl@chapttype{chapter}
3126 \ifx\@makechapterhead\undefined
3127   \let\bbl@patchchapter\relax

```

```

3128 \else\ifx\thechapter\@undefined
3129 \let\bbl@patchchapter\relax
3130 \else\ifx\ps@headings\@undefined
3131 \let\bbl@patchchapter\relax
3132 \else
3133 \def\bbl@patchchapter{%
3134 \global\let\bbl@patchchapter\relax
3135 \gdef\bbl@chfmt{%
3136 \bbl@ifunset\bbl@\bbl@chapttype fmt@\language}%
3137 {\@chapapp\space\thechapter}%
3138 {\@nameuse\bbl@\bbl@chapttype fmt@\language}}}%
3139 \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3140 \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3141 \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3142 \bbl@sreplace\makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3143 \bbl@tglobal\appendix
3144 \bbl@tglobal\ps@headings
3145 \bbl@tglobal\chaptermark
3146 \bbl@tglobal\makechapterhead}
3147 \let\bbl@patchappendix\bbl@patchchapter
3148 \fi\fi\fi
3149 \ifx\@part\@undefined
3150 \let\bbl@patchpart\relax
3151 \else
3152 \def\bbl@patchpart{%
3153 \global\let\bbl@patchpart\relax
3154 \gdef\bbl@partformat{%
3155 \bbl@ifunset\bbl@partfmt@\language}%
3156 {\partname\nobreakspace\thepart}%
3157 {\@nameuse\bbl@partfmt@\language}}}%
3158 \bbl@sreplace\@part{\partname\nobreakspace\thepart}{\bbl@partformat}%
3159 \bbl@tglobal\@part}
3160 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3161 \let\bbl@calendar\@empty
3162 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3163 \def\bbl@localedate#1#2#3#4{%
3164 \begingroup
3165 \edef\bbl@they{#2}%
3166 \edef\bbl@them{#3}%
3167 \edef\bbl@thed{#4}%
3168 \edef\bbl@tempe{%
3169 \bbl@ifunset\bbl@calpr@\language}{\bbl@cl{calpr}},%
3170 #1}%
3171 \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3172 \bbl@replace\bbl@tempe{ }{}%
3173 \bbl@replace\bbl@tempe{convert}{convert=}%
3174 \let\bbl@ld@calendar\@empty
3175 \let\bbl@ld@variant\@empty
3176 \let\bbl@ld@convert\relax
3177 \def\bbl@tempb##1=##2\@{\@namedef\bbl@ld@##1}{##2}}%
3178 \bbl@foreach\bbl@tempe{\bbl@tempb##1\@}%
3179 \bbl@replace\bbl@ld@calendar{gregorian}{}%
3180 \ifx\bbl@ld@calendar\@empty\else
3181 \ifx\bbl@ld@convert\relax\else
3182 \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3183 {\bbl@ld@calendar}\bbl@they\bbl@them\bbl@thed
3184 \fi
3185 \fi
3186 \@nameuse\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3187 \edef\bbl@calendar{% Used in \month..., too

```



```

3188 \bbl@ld@calendar
3189 \ifx\bbl@ld@variant\@empty\else
3190 .\bbl@ld@variant
3191 \fi}%
3192 \bbl@cased
3193 {\@nameuse\bbl@date@\language\name @\bbl@calendar}%
3194 \bbl@they\bbl@them\bbl@thed}%
3195 \endgroup}
3196 %
3197 \def\bbl@printdate#1{%
3198 \ifnextchar[{\bbl@printdate@i{#1}}{\bbl@printdate@i{#1}[]}}
3199 \def\bbl@printdate@i#1[#2]#3#4#5{%
3200 \bbl@usedategroupttrue
3201 \def\bbl@tempc{#1}%
3202 \bbl@replace\bbl@tempc{-}{@}%
3203 \@nameuse\bbl@ensure@\bbl@tempc{\@localedate[#2]{#3}{#4}{#5}}
3204 %
3205 % e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3206 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3207 \bbl@trim@def\bbl@tempa{#1.#2}%
3208 \bbl@ifsamestring{\bbl@tempa}{months.wide}% to savedate
3209 {\bbl@trim@def\bbl@tempa{#3}%
3210 \bbl@trim\toks@{#5}%
3211 \@temptokena\expandafter{\bbl@savedate}%
3212 \bbl@exp{% Reverse order - in ini last wins
3213 \def\\bbl@savedate{%
3214 \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3215 \the@temptokena}}}%
3216 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3217 {\lowercase{\def\bbl@tempb{#6}}}%
3218 \bbl@trim@def\bbl@toreplace{#5}%
3219 \bbl@TG@@date
3220 \global\bbl@csarg\let{date@\language\name @\bbl@tempb}\bbl@toreplace
3221 \ifx\bbl@savetoday\@empty
3222 \bbl@exp{%
3223 \\AfterBabelCommands{%
3224 \gdef\<\language\name date>{\protect\<\language\name date >}%
3225 \gdef\<\language\name date >{\bbl@printdate{\language\name}}}%
3226 \def\\bbl@savetoday{%
3227 \\SetString\\today{%
3228 \<\language\name date>[convert]%
3229 {\the\year}{\the\month}{\the\day}}}%
3230 \fi}%
3231 {}}}

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after \bbl@replace \toks@ contains the resulting string, which is used by \bbl@replace@finish@iii (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3232 \let\bbl@calendar\@empty
3233 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3234 \@nameuse\bbl@ca@#2#1\@}
3235 \newcommand\babelDateSpace{\nobreakspace}
3236 \newcommand\babelDateDot{.\@}
3237 \newcommand\babelDated[1][\number#1]}
3238 \newcommand\babelDatedd[1][\ifnum#1<10 0\fi\number#1]}
3239 \newcommand\babelDateM[1][\number#1]}
3240 \newcommand\babelDateMM[1][\ifnum#1<10 0\fi\number#1]}
3241 \newcommand\babelDateMMMM[1][{%
3242 \csname month\romannumeral#1\bbl@calendar name\endcsname}}%
3243 \newcommand\babelDatey[1][\number#1]}%
3244 \newcommand\babelDateyy[1][{%

```

```

3245 \ifnum#1<10 0\number#1 %
3246 \else\ifnum#1<100 \number#1 %
3247 \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3248 \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3249 \else
3250 \bbl@error{limit-two-digits}{\the\toks@}%
3251 \fi\fi\fi\fi}}
3252 \newcommand\BabelDateyyy[1]{\number#1}
3253 \newcommand\BabelDateU[1]{\number#1}%
3254 \def\bbl@replace@finish@iii#1{%
3255 \bbl@exp{\def\#1####1####2####3{\the\toks@}}
3256 \def\bbl@TG@@date{%
3257 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace{}}%
3258 \bbl@replace\bbl@toreplace{[.]}{\BabelDateDot{}}%
3259 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{####1}}%
3260 \bbl@replace\bbl@toreplace{[y|]}{\bbl@datecctr[####1]}%
3261 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{####1}}%
3262 \bbl@replace\bbl@toreplace{[yyyy]}{\BabelDateyyy{####1}}%
3263 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{####2}}%
3264 \bbl@replace\bbl@toreplace{[M|]}{\bbl@datecctr[####2]}%
3265 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{####2}}%
3266 \bbl@replace\bbl@toreplace{[MMMM]}{\BabelDateMMMM{####2}}%
3267 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{####3}}%
3268 \bbl@replace\bbl@toreplace{[d|]}{\bbl@datecctr[####3]}%
3269 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{####3}}%
3270 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{####1}}%
3271 \bbl@replace\bbl@toreplace{[U|]}{\bbl@datecctr[####1]}%
3272 \bbl@replace@finish@iii\bbl@toreplace}
3273 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3274 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3275 \AddToHook{begindocument/before}{%
3276 \let\bbl@normalsf\normalsfcodes
3277 \let\normalsfcodes\relax}
3278 \AtBeginDocument{%
3279 \ifx\bbl@normalsf\@empty
3280 \ifnum\scode`\.=\@m
3281 \let\normalsfcodes\frenchspacing
3282 \else
3283 \let\normalsfcodes\nonfrenchspacing
3284 \fi
3285 \else
3286 \let\normalsfcodes\bbl@normalsf
3287 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3288 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3289 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3290 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3291 #1[#2]{#3}{#4}{#5}}
3292 \begingroup
3293 \catcode`\%=12
3294 \catcode`\&=14
3295 \gdef\bbl@transforms#1#2#3{\&%

```

```

3296 \directlua{
3297     local str = [==[#2]==]
3298     str = str:gsub('%.%d+%.%d+$', '')
3299     token.set_macro('babeltempa', str)
3300 }&%
3301 \def\babeltempc{}&%
3302 \bbl@xin@{,\babeltempa,}{,\bbl@KVP@transforms,}&%
3303 \ifin@else
3304     \bbl@xin@{:,\babeltempa,}{,\bbl@KVP@transforms,}&%
3305 \fi
3306 \ifin@
3307     \bbl@foreach\bbl@KVP@transforms{&%
3308         \bbl@xin@{:,\babeltempa,}{,##1,}&%
3309         \ifin@ &% font:font:transform syntax
3310             \directlua{
3311                 local t = {}
3312                 for m in string.gmatch('##1'..' ':'(.)') do
3313                     table.insert(t, m)
3314                 end
3315                 table.remove(t)
3316                 token.set_macro('babeltempc', ',fonts=' .. table.concat(t, ' '))
3317             }&%
3318         \fi}&%
3319     \in@{.0$}{#2$}&%
3320     \ifin@
3321         \directlua{&% (\attribute) syntax
3322             local str = string.match([[ \bbl@KVP@transforms]],
3323                 '%(([^%(-)%][^%)]-\babeltempa)')
3324             if str == nil then
3325                 token.set_macro('babeltempb', '')
3326             else
3327                 token.set_macro('babeltempb', ',attribute=' .. str)
3328             end
3329         }&%
3330     \toks@{#3}&%
3331     \bbl@exp{&%
3332         \\g@addto@macro\\bbl@release@transforms{&%
3333             \relax &% Closes previous \bbl@transforms@aux
3334             \\bbl@transforms@aux
3335             \\#1{label=\babeltempa\babeltempb\babeltempc}&%
3336             {\language\the\toks@}}&%
3337     \else
3338         \g@addto@macro\bbl@release@transforms{, {#3}}&%
3339     \fi
3340 \fi}
3341 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3342 \def\bbl@provide@lsys#1{%
3343     \bbl@ifunset{bbl@lname@#1}%
3344     {\bbl@load@info{#1}}%
3345     {}%
3346     \bbl@csarg\let{lsys@#1}\@empty
3347     \bbl@ifunset{bbl@sname@#1}{\bbl@csarg\gdef{sname@#1}{Default}}{}%
3348     \bbl@ifunset{bbl@sotf@#1}{\bbl@csarg\gdef{sotf@#1}{DFLT}}{}%
3349     \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3350     \bbl@ifunset{bbl@lname@#1}{%
3351         {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%

```

```

3352 \ifcase\bbl@engine\or\or
3353 \bbl@ifunset{\bbl@prehc@#1}{}%
3354 {\bbl@exp{\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3355 }%
3356 {\ifx\bbl@xenohyph\undefined
3357 \global\let\bbl@xenohyph\bbl@xenohyph@
3358 \ifx\AtBeginDocument\@notprerr
3359 \expandafter\@secondoftwo % to execute right now
3360 \fi
3361 \AtBeginDocument{%
3362 \bbl@patchfont{\bbl@xenohyph}%
3363 {\expandafter\select@language\expandafter{\language}%
3364 \fi}}%
3365 \fi
3366 \bbl@csarg\bbl@toglobal{\sys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the ini file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\TeX$. Non-digits characters are kept. The first macro is the generic “localized” command.

```

3367 \def\bbl@setdigits#1#2#3#4#5{%
3368 \bbl@exp{%
3369 \def\<\language name digits>####1{% i.e., \langdigits
3370 \<\bbl@digits@\language name>####1\\\@nil}%
3371 \let\<\bbl@cnt@digits@\language name>\<\language name digits>%
3372 \def\<\language name counter>####1{% i.e., \langcounter
3373 \\\expandafter\<\bbl@counter@\language name>%
3374 \\\csname c@####1\endcsname}%
3375 \def\<\bbl@counter@\language name>####1{% i.e., \bbl@counter@lang
3376 \\\expandafter\<\bbl@digits@\language name>%
3377 \\\number####1\\\@nil}}%
3378 \def\bbl@tempa##1##2##3##4##5{%
3379 \bbl@exp{% Wow, quite a lot of hashes! :- (
3380 \def\<\bbl@digits@\language name>#####1{%
3381 \\\ifx#####1\\\@nil % i.e., \bbl@digits@lang
3382 \\\else
3383 \\\ifx0#####1#1%
3384 \\\else\\\ifx1#####1#2%
3385 \\\else\\\ifx2#####1#3%
3386 \\\else\\\ifx3#####1#4%
3387 \\\else\\\ifx4#####1#5%
3388 \\\else\\\ifx5#####1#1%
3389 \\\else\\\ifx6#####1#2%
3390 \\\else\\\ifx7#####1#3%
3391 \\\else\\\ifx8#####1#4%
3392 \\\else\\\ifx9#####1#5%
3393 \\\else#####1%
3394 \\\fi\\\fi\\\fi\\\fi\\\fi\\\fi\\\fi\\\fi\\\fi\\\fi
3395 \\\expandafter\<\bbl@digits@\language name>%
3396 \\\fi}}}%
3397 \bbl@tempa}

```

Alphabetic counters must be converted from a space separated list to an `\ifcase` structure.

```

3398 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={ }
3399 \ifx\#1% % \ before, in case #1 is multiletter
3400 \bbl@exp{%
3401 \def\<\bbl@tempa####1{%
3402 \<\ifcase>####1\space\the\toks@\<\else>\\\@ctrerr\<\fi>}}%
3403 \else
3404 \toks@\expandafter{\the\toks@\or #1}%
3405 \expandafter\bbl@buildifcase
3406 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before @@ collect digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210. Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as a special case, for a fixed form (see babel-he.ini, for example).

```

3407 \newcommand\localenumberal[2]{%
3408   \bbl@ifunset{bbl@cntr@#1@\language}%
3409   {#2}%
3410   {\bbl@cs{cntr@#1@\language}{#2}}}
3411 \def\bbl@localecntr#1#2{\localenumberal{#2}{#1}}
3412 \newcommand\localecounter[2]{%
3413   \expandafter\bbl@localecntr
3414   \expandafter\number\csname c@#2\endcsname}{#1}}
3415 \def\bbl@alphnumer#1#2{%
3416   \expandafter\bbl@alphnumer@i\number#2 76543210\@@{#1}}
3417 \def\bbl@alphnumer@i#1#2#3#4#5#6#7#8\@@#9{%
3418   \ifcase\@car#8\@nil\or    % Currently <10000, but prepared for bigger
3419     \bbl@alphnumer@ii{#9}00000#1\or
3420     \bbl@alphnumer@ii{#9}00000#1#2\or
3421     \bbl@alphnumer@ii{#9}0000#1#2#3\or
3422     \bbl@alphnumer@ii{#9}000#1#2#3#4\else
3423     \bbl@alphnum@invalid{>9999}%
3424   \fi}
3425 \def\bbl@alphnumer@ii#1#2#3#4#5#6#7#8{%
3426   \bbl@ifunset{bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3427   {\bbl@cs{cntr@#1.4@\language}#5%
3428     \bbl@cs{cntr@#1.3@\language}#6%
3429     \bbl@cs{cntr@#1.2@\language}#7%
3430     \bbl@cs{cntr@#1.1@\language}#8%
3431     \ifnum#6#7#8>\z@
3432       \bbl@ifunset{bbl@cntr@#1.S.321@\language}{}%
3433       {\bbl@cs{cntr@#1.S.321@\language}}%
3434     \fi}%
3435   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
3436 \def\bbl@alphnum@invalid#1{%
3437   \bbl@error{alphabetic-too-large}{#1}{}}

```

4.24. Casing

```

3438 \newcommand\BabelUppercaseMapping[3]{%
3439   \DeclareUppercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3440 \newcommand\BabelTitlecaseMapping[3]{%
3441   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3442 \newcommand\BabelLowercaseMapping[3]{%
3443   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

```

The parser for casing and casing. (variant).

```

3444 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3445   \def\bbl@utftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3446 \else
3447   \def\bbl@utftocode#1{\expandafter`\string#1}
3448 \fi
3449 \def\bbl@casemapping#1#2#3{% 1:variant
3450   \def\bbl@tempa##1 ##2{% Loop
3451     \bbl@casemapping@i{##1}%
3452     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3453   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3454   \def\bbl@tempe{0}% Mode (upper/lower...)
3455   \def\bbl@tempc{#3}% Casing list
3456   \expandafter\bbl@tempa\bbl@tempc\@empty}
3457 \def\bbl@casemapping@i#1{%
3458   \def\bbl@tempb{#1}%
3459   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}

```

```

3460 \nameuse{regex_replace_all:nnN}%
3461 {\x{c0}-\x{ff}}[\x{80}-\x{bf}]*}{\0}}\bbl@tempb
3462 \else
3463 \nameuse{regex_replace_all:nnN}{.}{\0}}\bbl@tempb
3464 \fi
3465 \expandafter\bbl@casemapping@ii\bbl@tempb\@@}
3466 \def\bbl@casemapping@ii#1#2#3\@@{%
3467 \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3468 \ifin@
3469 \edef\bbl@tempe{%
3470 \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3471 \else
3472 \ifcase\bbl@tempe\relax
3473 \DeclareUppercaseMapping[\bbl@templ]{\bbl@utfocode{#1}}{#2}%
3474 \DeclareLowercaseMapping[\bbl@templ]{\bbl@utfocode{#2}}{#1}%
3475 \or
3476 \DeclareUppercaseMapping[\bbl@templ]{\bbl@utfocode{#1}}{#2}%
3477 \or
3478 \DeclareLowercaseMapping[\bbl@templ]{\bbl@utfocode{#1}}{#2}%
3479 \or
3480 \DeclareTitlecaseMapping[\bbl@templ]{\bbl@utfocode{#1}}{#2}%
3481 \fi
3482 \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3483 \def\bbl@localeinfo#1#2{%
3484 \bbl@ifunset{bbl@info@#2}{#1}%
3485 {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3486 {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3487 \newcommand\localeinfo[1]{%
3488 \ifx*#1\@empty
3489 \bbl@afterelse\bbl@localeinfo}%
3490 \else
3491 \bbl@localeinfo
3492 {\bbl@error{no-ini-info}{}}{}%
3493 {#1}%
3494 \fi}
3495 % \@namedef{bbl@info@name.locale}{lcname}
3496 \@namedef{bbl@info@tag.ini}{lini}
3497 \@namedef{bbl@info@name.english}{elname}
3498 \@namedef{bbl@info@name.opentype}{lname}
3499 \@namedef{bbl@info@tag.bcp47}{tbc}
3500 \@namedef{bbl@info@language.tag.bcp47}{lbcp}
3501 \@namedef{bbl@info@tag.opentype}{lotf}
3502 \@namedef{bbl@info@script.name}{esname}
3503 \@namedef{bbl@info@script.name.opentype}{sname}
3504 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3505 \@namedef{bbl@info@script.tag.opentype}{sotf}
3506 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3507 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3508 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3509 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3510 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3511 <<*<More package options>> \equiv
3512 \DeclareOption{ensureinfo=off}{}
3513 <</<More package options>>
3514 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is `\getlocaleproperty`.

```

3515 \newcommand\getlocaleproperty{%
3516   \ifstar\bbl@getproperty@s\bbl@getproperty@x%
3517   \def\bbl@getproperty@s#1#2#3{%
3518     \let#1\relax
3519     \def\bbl@elt##1##2##3{%
3520       \bbl@ifsamestring{##1/##2}{#3}%
3521       {\providecommand#1{##3}%
3522       \def\bbl@elt###1####2####3{}}%
3523     }%
3524     \bbl@cs{inidata@#2}}%
3525   \def\bbl@getproperty@x#1#2#3{%
3526     \bbl@getproperty@s{#1}{#2}{#3}%
3527     \ifx#1\relax
3528       \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3529     \fi}

```

To inspect every possible loaded ini, we define `\LocaleForEach`, where `\bbl@ini@loaded` is a comma-separated list of locales, built by `\bbl@read@ini`.

```

3530 \let\bbl@ini@loaded\@empty
3531 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3532 \def\ShowLocaleProperties#1{%
3533   \typeout{}%
3534   \typeout{*** Properties for language '#1' ***}%
3535   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3536   \@nameuse{\bbl@inidata@#1}%
3537   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 27 tag to the actual language, if `bcp47.toname` is enabled (i.e., if `bbl@bcptoname` is true), and (2) lazy loading. With `autoload.bcp47` enabled *and* lazy loading, we must first build a name for the language, with the help of `autoload.bcp47.prefix`. Then we use `\provideprovide` passing the options set with `autoload.bcp47.options` (by default import). Finally, and if the locale has not been loaded before, we use `\provideprovide` with the language name as passed to the selector.

```

3538 \newif\ifbbl@bcppallowed
3539 \bbl@bcppallowedfalse
3540 \def\bbl@autoload@options{@import}
3541 \def\bbl@provide@locale{%
3542   \ifx\babelprovide\@undefined
3543     \bbl@error{base-on-the-fly}{}{}%
3544   \fi
3545   \let\bbl@auxname\language
3546   \ifbbl@bcptoname
3547     \bbl@ifunset{\bbl@bcp@map@\language}{}% Move uplevel??
3548     {\edef\language{\@nameuse{\bbl@bcp@map@\language}}}%
3549     \let\localename\language}%
3550   \fi
3551   \ifbbl@bcppallowed
3552     \expandafter\ifx\csname date\language\endcsname\relax
3553       \expandafter
3554       \bbl@bcplookup{\language}%-\@empty-\@empty-\@empty@@
3555       \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3556         \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3557         \let\localename\language
3558         \expandafter\ifx\csname date\language\endcsname\relax
3559           \let\bbl@initoload\bbl@bcp
3560           \bbl@exp{\babelprovide[\bbl@autoload@bcppoptions]{\language}}%
3561           \let\bbl@initoload\relax
3562         \fi

```

```

3563      \bbl@csarg\xdef{bcp@map@{bbl@bcp}{\localename}%
3564      \fi
3565      \fi
3566      \fi
3567      \expandafter\ifx\csname date\language\endcsname\relax
3568      \IfFileExists{babel-\language.tex}%
3569      {\bbl@exp{\babelprovide{bbl@autoload@options}{\language}}}%
3570      {}}%
3571      \fi}

```

$\TeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension.`(s)` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to `tag.bcp47`.

```

3572 \providecommand\BCPdata{}
3573 \ifx\renewcommand\@undefined\else
3574   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3575   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3576     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3577     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3578     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3579   \def\bbl@bcpdata@ii#1#2{%
3580     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3581     {\bbl@error{unknown-ini-field}{#1}{}}}%
3582     {\bbl@ifunset{bbl@csname bbl@info@#1.tag.bcp47\endcsname @#2}{}}%
3583     {\bbl@cs{csname bbl@info@#1.tag.bcp47\endcsname @#2}}}%
3584   \fi
3585   \@namedef{bbl@info@casing.tag.bcp47}{casing}
3586   \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3587 \newcommand\babeladjust[1]{%
3588   \bbl@forkv{#1}{%
3589     \bbl@ifunset{bbl@ADJ@##1@##2}%
3590     {\bbl@cs{ADJ@##1}{##2}}%
3591     {\bbl@cs{ADJ@##1@##2}}}
3592 %
3593 \def\bbl@adjust@lua#1#2{%
3594   \ifvmode
3595     \ifnum\currentgrouplevel=\z@
3596       \directlua{ Babel.#2 }%
3597       \expandafter\expandafter\expandafter\@gobble
3598     \fi
3599   \fi
3600   {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3601 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3602   \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3603 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3604   \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3605 \@namedef{bbl@ADJ@bidi.text@on}{%
3606   \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3607 \@namedef{bbl@ADJ@bidi.text@off}{%
3608   \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3609 \@namedef{bbl@ADJ@bidi.math@on}{%
3610   \let\bbl@noamsmath\@empty}
3611 \@namedef{bbl@ADJ@bidi.math@off}{%
3612   \let\bbl@noamsmath\relax}
3613 %
3614 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%

```



```

3615 \bbl@adjust@lua{bidi}{digits_mapped=true}}
3616 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3617 \bbl@adjust@lua{bidi}{digits_mapped=false}}
3618 %
3619 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3620 \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3621 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3622 \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3623 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3624 \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3625 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3626 \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3627 \@namedef{bbl@ADJ@justify.arabic@on}{%
3628 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3629 \@namedef{bbl@ADJ@justify.arabic@off}{%
3630 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3631 %
3632 \def\bbl@adjust@layout#1{%
3633 \ifvmode
3634 #1%
3635 \expandafter\@gobble
3636 \fi
3637 {\bbl@error{layout-only-vertical}}{}}}% Gobbled if everything went ok.
3638 \@namedef{bbl@ADJ@layout.tabular@on}{%
3639 \ifnum\bbl@tabular@mode=\tw@
3640 \bbl@adjust@layout{\let\@tabular\bbl@NL@tabular}%
3641 \else
3642 \chardef\bbl@tabular@mode\@ne
3643 \fi}
3644 \@namedef{bbl@ADJ@layout.tabular@off}{%
3645 \ifnum\bbl@tabular@mode=\tw@
3646 \bbl@adjust@layout{\let\@tabular\bbl@OL@tabular}%
3647 \else
3648 \chardef\bbl@tabular@mode\z@
3649 \fi}
3650 \@namedef{bbl@ADJ@layout.lists@on}{%
3651 \bbl@adjust@layout{\let\list\bbl@NL@list}}
3652 \@namedef{bbl@ADJ@layout.lists@off}{%
3653 \bbl@adjust@layout{\let\list\bbl@OL@list}}
3654 %
3655 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3656 \bbl@bcpallowedtrue}
3657 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3658 \bbl@bcpallowedfalse}
3659 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3660 \def\bbl@bcp@prefix{#1}}
3661 \def\bbl@bcp@prefix{bcp47-}
3662 \@namedef{bbl@ADJ@autoload.options}#1{%
3663 \def\bbl@autoload@options{#1}}
3664 \def\bbl@autoload@bcptoptions{import}
3665 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3666 \def\bbl@autoload@bcptoptions{#1}}
3667 \newif\ifbbl@bcptname
3668 %
3669 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3670 \bbl@bcptnametrue}
3671 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3672 \bbl@bcptnamefalse}
3673 %
3674 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3675 \directlua{ Babel.ignore_pre_char = function(node)
3676 return (node.lang == \the\csname l@nohyphenation\endcsname)
3677 end }}

```

```

3678 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3679   \directlua{ Babel.ignore_pre_char = function(node)
3680     return false
3681   end }}
3682 %
3683 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3684   \def\bbl@ignoreinterchar{%
3685     \ifnum\language=\l@nohyphenation
3686       \expandafter\@gobble
3687     \else
3688       \expandafter\@firstofone
3689     \fi}}
3690 \@namedef{bbl@ADJ@interchar.disable@off}{%
3691   \let\bbl@ignoreinterchar\@firstofone}
3692 %
3693 \@namedef{bbl@ADJ@select.write@shift}{%
3694   \let\bbl@restorelastskip\relax
3695   \def\bbl@savelastskip{%
3696     \let\bbl@restorelastskip\relax
3697     \ifvmode
3698       \ifdim\lastskip=\z@
3699         \let\bbl@restorelastskip\nobreak
3700       \else
3701         \bbl@exp{%
3702           \def\\bbl@restorelastskip{%
3703             \skip@=\the\lastskip
3704             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3705         \fi
3706       \fi}}
3707 \@namedef{bbl@ADJ@select.write@keep}{%
3708   \let\bbl@restorelastskip\relax
3709   \let\bbl@savelastskip\relax}
3710 \@namedef{bbl@ADJ@select.write@omit}{%
3711   \AddBabelHook{babel-select}{beforestart}{%
3712     \expandafter\babel@aux\expandafter{\bbl@main@language}{}}%
3713   \let\bbl@restorelastskip\relax
3714   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3715 \@namedef{bbl@ADJ@select.encoding@off}{%
3716   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The key argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```

3717 <<{*More package options}>> ≡
3718 \DeclareOption{safe=none}{\let\bbl@opt@safe\@empty}
3719 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3720 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3721 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3722 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3723 <</More package options>>

```

@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```

3724 \bbl@trace{Cross referencing macros}
3725 \ifx\bbl@opt@safe\@empty\else % i.e., if 'ref' and/or 'bib'
3726   \def\@newl@bel#1#2#3{%
3727     {\@safe@activestrue
3728       \bbl@ifunset{#1@#2}%
3729       \relax
3730       {\gdef\@multiplelabels{%
3731         \@latex@warning@no@line{There were multiply-defined labels}}}%
3732       \@latex@warning@no@line{Label `#2' multiply defined}}}%
3733   \global\@namedef{#1@#2}{#3}}

```

\@testdef An internal $\TeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```

3734 \CheckCommand*\@testdef[3]{%
3735   \def\reserved@a{#3}%
3736   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3737   \else
3738     \@tempswatrue
3739   \fi}

```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```

3740 \def\@testdef#1#2#3{%
3741   \@safe@activestrue
3742   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3743   \def\bbl@tempb{#3}%
3744   \@safe@activestrue
3745   \ifx\bbl@tempa\relax
3746   \else
3747     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3748   \fi
3749   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3750   \ifx\bbl@tempa\bbl@tempb
3751   \else
3752     \@tempswatrue
3753   \fi}
3754 \fi

```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren’t already) to prevent problems if they should become expanded at the wrong moment.

```

3755 \bbl@xin@{R}\bbl@opt@safe
3756 \ifin@
3757   \edef\bbl@tempc{\expandafter\string\csname ref code\endcsname}%
3758   \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3759   {\expandafter\strip@prefix\meaning\ref}%
3760 \ifin@
3761   \bbl@redefine\@kernel@ref#1{%
3762     \@safe@activestrue\org@@kernel@ref{#1}\@safe@activestrue}
3763   \bbl@redefine\@kernel@pageref#1{%
3764     \@safe@activestrue\org@@kernel@pageref{#1}\@safe@activestrue}
3765   \bbl@redefine\@kernel@sref#1{%
3766     \@safe@activestrue\org@@kernel@sref{#1}\@safe@activestrue}
3767   \bbl@redefine\@kernel@spageref#1{%
3768     \@safe@activestrue\org@@kernel@spageref{#1}\@safe@activestrue}
3769 \else
3770   \bbl@redefineroquest\ref#1{%
3771     \@safe@activestrue\org@ref{#1}\@safe@activestrue}

```

```

3772 \bbl@redefineroast\pageref#1{%
3773 \@safe@activetrue\org@pageref{#1}\@safe@activesfalse}
3774 \fi
3775 \else
3776 \let\org@ref\ref
3777 \let\org@pageref\pageref
3778 \fi

```

\@citex The macro used to cite from a bibliography, \cite, uses an internal macro, \@citex. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave \cite alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```

3779 \bbl@xin@{B}\bbl@opt@safe
3780 \ifin@
3781 \bbl@redefine\@citex[#1]#2{%
3782 \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activesfalse
3783 \org@citex[#1]{\bbl@tempa}}

```

Unfortunately, the packages natbib and cite need a different definition of \@citex... To begin with, natbib has a definition for \@citex with *three* arguments... We only know that a package is loaded when \begin{document} is executed, so we need to postpone the different redefinition.

Notice that we use \def here instead of \bbl@redefine because \org@citex is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of natbib change dynamically \@citex, so PR4087 doesn't seem fixable in a simple way. Just load natbib before.)

```

3784 \AtBeginDocument{%
3785 \ifpackageloaded{natbib}{%
3786 \def\@citex[#1][#2]#3{%
3787 \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activesfalse
3788 \org@citex[#1][#2]{\bbl@tempa}}%
3789 }{}}

```

The package cite has a definition of \@citex where the shorthands need to be turned off in both arguments.

```

3790 \AtBeginDocument{%
3791 \ifpackageloaded{cite}{%
3792 \def\@citex[#1]#2{%
3793 \@safe@activetrue\org@citex[#1]{#2}\@safe@activesfalse}%
3794 }{}}

```

\nocite The macro \nocite which is used to instruct BiBTeX to extract uncited references from the database.

```

3795 \bbl@redefine\nocite#1{%
3796 \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}

```

\babcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \babcite is needed we define \babcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \babcite. This new definition is then activated.

```

3797 \bbl@redefine\babcite{%
3798 \bbl@cite@choice
3799 \babcite}

```

\bbl@babcite The macro \bbl@babcite holds the definition of \babcite needed when neither natbib nor cite is loaded.

```

3800 \def\bbl@babcite#1#2{%
3801 \org@babcite{#1}{\@safe@activesfalse#2}}

```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3802 \def\bbl@cite@choice{%
3803   \global\let\bibcite\bbl\bibcite
3804   \@ifpackageloaded{natbib}{\global\let\bibcite\org\bibcite}{}%
3805   \@ifpackageloaded{cite}{\global\let\bibcite\org\bibcite}{}%
3806   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3807 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal $\TeX$ macros called by \bibitem that write the citation label on the aux file.

```
3808 \bbl@redefine\@bibitem#1{%
3809   \@safe@activestruerorg@bibitem{#1}\@safe@activesfalse}
3810 \else
3811   \let\org@nocite\nocite
3812   \let\org@citex\@citex
3813   \let\org@bibcite\bibcite
3814   \let\org@bibitem\@bibitem
3815 \fi
```

5.2. Layout

```
3816 \newcommand\BabelPatchSection[1]{%
3817   \ifundefined{#1}{}%
3818   \bbl@exp{\let\<bbl@ss@#1>\<#1>}%
3819   \@namedef{#1}{%
3820     \ifstar{\bbl@presec@#1}%
3821     {\@dblarg{\bbl@presec@x{#1}}}}%
3822 \def\bbl@presec@x#1[#2]#3{%
3823   \bbl@exp{%
3824     \\\select@language@x{\bbl@main@language}%
3825     \\\bbl@cs{sspre@#1}%
3826     \\\bbl@cs{ss@#1}%
3827     [\\foreignlanguage{\language@name}{\unexpanded{#2}}}%
3828     {\\foreignlanguage{\language@name}{\unexpanded{#3}}}%
3829     \\\select@language@x{\language@name}}%
3830 \def\bbl@presec@s#1#2{%
3831   \bbl@exp{%
3832     \\\select@language@x{\bbl@main@language}%
3833     \\\bbl@cs{sspre@#1}%
3834     \\\bbl@cs{ss@#1}*%
3835     {\\foreignlanguage{\language@name}{\unexpanded{#2}}}%
3836     \\\select@language@x{\language@name}}%
3837 %
3838 \IfBabelLayout{sectioning}%
3839   {\BabelPatchSection{part}%
3840    \BabelPatchSection{chapter}%
3841    \BabelPatchSection{section}%
3842    \BabelPatchSection{subsection}%
3843    \BabelPatchSection{subsubsection}%
3844    \BabelPatchSection{paragraph}%
3845    \BabelPatchSection{subparagraph}%
3846    \def\babel@toc#1{%
3847      \select@language@x{\bbl@main@language}}}%
3848 \IfBabelLayout{captions}%
3849   {\BabelPatchSection{caption}}}
```

\BabelFootnote Footnotes.

```
3850 \bbl@trace{Footnotes}
```

```

3851 \def\bbl@footnote#1#2#3{%
3852   \@ifnextchar[%
3853     {\bbl@footnote@o{#1}{#2}{#3}}%
3854     {\bbl@footnote@x{#1}{#2}{#3}}}
3855 \long\def\bbl@footnote@x#1#2#3#4{%
3856   \bgroup
3857     \select@language@x{\bbl@main@language}%
3858     \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3859   \egroup}
3860 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3861   \bgroup
3862     \select@language@x{\bbl@main@language}%
3863     \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3864   \egroup}
3865 \def\bbl@footnotetext#1#2#3{%
3866   \@ifnextchar[%
3867     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3868     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3869 \long\def\bbl@footnotetext@x#1#2#3#4{%
3870   \bgroup
3871     \select@language@x{\bbl@main@language}%
3872     \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3873   \egroup}
3874 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3875   \bgroup
3876     \select@language@x{\bbl@main@language}%
3877     \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3878   \egroup}
3879 \def\BabelFootnote#1#2#3#4{%
3880   \ifx\bbl@fn@footnote\undefined
3881     \let\bbl@fn@footnote\footnote
3882   \fi
3883   \ifx\bbl@fn@footnotetext\undefined
3884     \let\bbl@fn@footnotetext\footnotetext
3885   \fi
3886   \bbl@ifblank{#2}%
3887     {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3888     \@namedef{\bbl@stripslash#1text}%
3889     {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3890     {\def#1{\bbl@exp{\bbl@footnote{\foreignlanguage{#2}}}{#3}{#4}}%
3891     \@namedef{\bbl@stripslash#1text}%
3892     {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}}{#3}{#4}}}%
3893 \IfBabelLayout{footnotes}%
3894 {\let\bbl@OL@footnote\footnote
3895  \BabelFootnote\footnote\language\name{}}}%
3896 {\BabelFootnote\localfootnote\language\name{}}}%
3897 {\BabelFootnote\mainfootnote{}}}%
3898 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3899 \bbl@trace{Marks}
3900 \IfBabelLayout{sectioning}
3901 {\ifx\bbl@opt@headfoot\@nnil
3902   \g@addto@macro\resetactivechars{%
3903     \set@typeset@protect
3904     \expandafter\select@language@x\expandafter{\bbl@main@language}%

```

```

3905     \let\protect\noexpand
3906     \ifcase\bbbl@bidimode\else % Only with bidi. See also above
3907         \edef\thepage{%
3908             \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3909     \fi}%
3910 \fi}
3911 {\ifbbbl@single\else
3912     \bbl@ifunset{markright }{\bbl@redefine\bbl@redefineroobust
3913         \markright#1{%
3914             \bbl@ifblank{#1}%
3915                 {\org@markright{}}}%
3916                 {\toks@{#1}%
3917                     \bbl@exp{%
3918                         \\org@markright{\\protect\\foreignlanguage{\language}\language}%
3919                         {\\protect\\bbl@restore@actives\the\toks@}}}}}%

```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, $\TeX$ stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3920     \ifx\@mkboth\markboth
3921         \def\bbl@tempc{\let\@mkboth\markboth}%
3922     \else
3923         \def\bbl@tempc{}%
3924     \fi
3925     \bbl@ifunset{markboth }{\bbl@redefine\bbl@redefineroobust
3926         \markboth#1#2{%
3927             \protected@edef\bbl@tempb##1{%
3928                 \protect\foreignlanguage
3929                 {\language}\protect\bbl@restore@actives##1}%
3930             \bbl@ifblank{#1}%
3931                 {\toks@{}}%
3932                 {\toks@\expandafter{\bbl@tempb{#1}}}%
3933             \bbl@ifblank{#2}%
3934                 {\@temptokena{}}%
3935                 {\@temptokena\expandafter{\bbl@tempb{#2}}}%
3936             \bbl@exp{\\org@markboth{\the\toks@}{\the\@temptokena}}}%
3937         \bbl@tempc
3938     \fi} % end ifbbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}
%         {code for odd pages}
%         {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

3939 \bbl@trace{Preventing clashes with other packages}
3940 \ifx\org@ref\@undefined\else
3941   \bbl@xin@{R}\bbl@opt@safe
3942   \ifin@
3943     \AtBeginDocument{%
3944       \ifpackageloaded{ifthen}{%
3945         \bbl@redefine@long\ifthenelse#1#2#3{%
3946           \let\bbl@temp@pref\pageref
3947           \let\pageref\org@pageref
3948           \let\bbl@temp@ref\ref
3949           \let\ref\org@ref
3950           \@safe@activestrue
3951           \org@ifthenelse{#1}%
3952             {\let\pageref\bbl@temp@pref
3953              \let\ref\bbl@temp@ref
3954              \@safe@activesfalse
3955              #2}%
3956             {\let\pageref\bbl@temp@pref
3957              \let\ref\bbl@temp@ref
3958              \@safe@activesfalse
3959              #3}%
3960           }%
3961         }{}%
3962       }
3963 \fi

```

5.4.2. varioref

\@@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

3964 \AtBeginDocument{%
3965   \ifpackageloaded{varioref}{%
3966     \bbl@redefine\@@vpageref#1[#2]#3{%
3967       \@safe@activestrue
3968       \org@@@vpageref{#1}[#2]#3}%
3969     \@safe@activesfalse}%
3970   \bbl@redefine\vrefpagemum#1#2{%
3971     \@safe@activestrue
3972     \org@vrefpagemum{#1}#2}%
3973   \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

3974   \expandafter\def\csname Ref \endcsname#1{%
3975     \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
3976   }{}%
3977 }
3978 \fi

```

5.4.3. hhline

\hhline Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the ‘`’` character which is made active by the french support

in babel. Therefore we need to *reload* the package when the ‘:’ is an active character. Note that this happens *after* the category code of the @-sign has been changed to other, so we need to temporarily change it to letter again.

```

3979 \AtEndOfPackage{%
3980   \AtBeginDocument{%
3981     \@ifpackageloaded{hhline}%
3982       {\expandafter\ifx\csname normal@char\string\endcsname\relax
3983         \else
3984           \makeatletter
3985           \def\@currname{hhline}\input{hhline.sty}\makeatother
3986           \fi}%
3987     {}}}
```

\substitutefontfamily *Deprecated.* It creates an fd file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by $\TeX$ (`\DeclareFontFamilySubstitution`).

```

3988 \def\substitutefontfamily#1#2#3{%
3989   \lowercase{\immediate\openout15=#1#2.fd\relax}%
3990   \immediate\write15{%
3991     \string\ProvidesFile{#1#2.fd}%
3992     [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
3993     \space generated font description file]^J
3994     \string\DeclareFontFamily{#1}{#2}{}}^J
3995     \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
3996     \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
3997     \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
3998     \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
3999     \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
4000     \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
4001     \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
4002     \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
4003   }%
4004   \closeout15
4005 }
4006 \@onlypreamble\substitutefontfamily
```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of $\TeX$ and $\LaTeX$ for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```

4007 \bbl@trace{Encoding and fonts}
4008 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4009 \newcommand\BabelNonText{TS1,T3,TS3}
4010 \let\org@TeX\TeX
4011 \let\org@LaTeX\LaTeX
4012 \let\ensureascii@firstofone
4013 \let\asciienencoding\empty
4014 \AtBeginDocument{%
4015   \def\@elt#1{, #1,}%
4016   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4017   \let\@elt\relax
4018   \let\bbl@tempb\empty
4019   \def\bbl@tempc{OT1}%
4020   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4021     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4022   \bbl@foreach\bbl@tempa{%
```

```

4023 \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4024 \ifin@
4025 \def\bbl@tempb{#1}% Store last non-ascii
4026 \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4027 \ifin@else
4028 \def\bbl@tempc{#1}% Store last ascii
4029 \fi
4030 \fi}%
4031 \ifx\bbl@tempb\@empty\else
4032 \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4033 \ifin@else
4034 \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4035 \fi
4036 \let\asciencoding\bbl@tempc
4037 \renewcommand\ensureascii[1]{%
4038 {\fontencoding{\asciencoding}\selectfont#1}}%
4039 \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4040 \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4041 \fi}

```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

Latinencoding When text is being typeset in an encoding other than ‘latin’ (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```

4042 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}

```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\@ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```

4043 \AtBeginDocument{%
4044 \ifpackageloaded{fontspec}%
4045 {\xdef\latinencoding{%
4046 \ifx\UTFencname\@undefined
4047 EU\ifcase\bbl@engine\or2\or1\fi
4048 \else
4049 \UTFencname
4050 \fi}}%
4051 {\gdef\latinencoding{OT1}%
4052 \ifx\cf@encoding\bbl@t@one
4053 \xdef\latinencoding{\bbl@t@one}%
4054 \else
4055 \def\@elt#1{,#1,}%
4056 \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4057 \let\@elt\relax
4058 \bbl@xin@{,T1,}\bbl@tempa
4059 \ifin@
4060 \xdef\latinencoding{\bbl@t@one}%
4061 \fi
4062 \fi}}

```

Latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4063 \DeclareRobustCommand{\latintext}{%
4064 \fontencoding{\latinencoding}\selectfont
4065 \def\encodingdefault{\latinencoding}}

```

Textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4066 \ifx\@undefined\DeclareTextFontCommand

```

```

4067 \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4068 \else
4069 \DeclareTextFontCommand{\textlatin}{\latintext}
4070 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\TeX$ 2021-06-01, there is a hook for this purpose.

```

4071 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This `babel` module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at `ARABI` (by Youssef Jabri), which is compatible with `babel`.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdftex` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\TeX$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\TeX$ -ja` shows, vertical typesetting is possible, too.

```

4072 \bbl@trace{Loading basic (internal) bidi support}
4073 \ifodd\bbl@engine
4074 \else % Any xe+lua bidi
4075   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4076     \bbl@error{bidi-only-lua}{-}{-}%
4077     \let\bbl@beforeforeign\leavevmode
4078     \AtEndOfPackage{%
4079       \EnableBabelHook{babel-bidi}%
4080       \bbl@xebidipar}
4081   \fi\fi
4082   \def\bbl@loadxebidi#1{%
4083     \ifx\RTLfootnotetext\@undefined
4084       \AtEndOfPackage{%
4085         \EnableBabelHook{babel-bidi}%
4086         \ifx\fontspec\@undefined
4087           \usepackage{fontspec}% bidi needs fontspec
4088         \fi
4089         \usepackage#1{bidi}%
4090         \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4091         \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4092           \ifnum\@nameuse{\bbl@wdir\@languagename}=\tw@ % 'AL' bidi
4093             \bbl@digitsdotdash % So ignore in 'R' bidi
4094           \fi}}%
4095     \fi}
4096   \ifnum\bbl@bidimode>200 % Any xe bidi=
4097     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4098       \bbl@tentative{bidi=bidi}
4099       \bbl@loadxebidi{}
4100     \or
4101       \bbl@loadxebidi{[rldocument]}
4102     \or
4103       \bbl@loadxebidi{}

```

```

4104 \fi
4105 \fi
4106 \fi
4107 \ifnum\bbl@bidimode=\@ne % bidi=default
4108 \let\bbl@beforeforeign\leavevmode
4109 \ifodd\bbl@engine % lua
4110 \newattribute\bbl@attr@dir
4111 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4112 \bbl@exp{\output{\bodydir\pagedir\the\output}}
4113 \fi
4114 \AtEndOfPackage{%
4115 \EnableBabelHook{babel-bidi}% pdf/lua/x
4116 \ifodd\bbl@engine\else % pdf/x
4117 \bbl@xebidipar
4118 \fi}
4119 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4120 \bbl@trace{Macros to switch the text direction}
4121 \def\bbl@alscripts{%
4122 ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4123 \def\bbl@rscripts{%
4124 Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4125 Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4126 Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4127 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4128 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4129 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4130 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4131 Meroitic,N'Ko,Orkhon,Todhri}
4132 %
4133 \def\bbl@provide@dirs#1{%
4134 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4135 \ifin@
4136 \global\bbl@csarg\chardef{wdir@#1}\@ne
4137 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4138 \ifin@
4139 \global\bbl@csarg\chardef{wdir@#1}\tw@
4140 \fi
4141 \else
4142 \global\bbl@csarg\chardef{wdir@#1}\z@
4143 \fi
4144 \ifodd\bbl@engine
4145 \bbl@csarg\ifcase{wdir@#1}%
4146 \directlua{ Babel.locale_props[\the\localeid].texdir = 'l' }%
4147 \or
4148 \directlua{ Babel.locale_props[\the\localeid].texdir = 'r' }%
4149 \or
4150 \directlua{ Babel.locale_props[\the\localeid].texdir = 'al' }%
4151 \fi
4152 \fi}
4153 %
4154 \def\bbl@switchdir{%
4155 \bbl@ifunset\bbl@lsys@\languagename}{\bbl@provide@lsys{\languagename}}{%
4156 \bbl@ifunset\bbl@wdir@\languagename}{\bbl@provide@dirs{\languagename}}{%
4157 \bbl@exp{\bbl@setdirs\bbl@cl{wdir}}}%
4158 \def\bbl@setdirs#1{%
4159 \ifcase\bbl@select@type
4160 \bbl@bodydir{#1}%
4161 \bbl@pardir{#1}% <- Must precede \bbl@texdir
4162 \fi

```

```

4163 \bbl@textdir{#1}}
4164 \ifnum\bbl@bidimode>\z@
4165 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4166 \DisableBabelHook{babel-bidi}
4167 \fi

Now the engine-dependent macros.

4168 \ifodd\bbl@engine % luatex=1
4169 \else % pdftex=0, xetex=2
4170 \newcount\bbl@dirlevel
4171 \chardef\bbl@thetextdir\z@
4172 \chardef\bbl@thepardir\z@
4173 \def\bbl@textdir#1{%
4174 \ifcase#1\relax
4175 \chardef\bbl@thetextdir\z@
4176 \@nameuse{setlatin}%
4177 \bbl@textdir@i\beginL\endL
4178 \else
4179 \chardef\bbl@thetextdir\@ne
4180 \@nameuse{setnonlatin}%
4181 \bbl@textdir@i\beginR\endR
4182 \fi}
4183 \def\bbl@textdir@i#1#2{%
4184 \ifhmode
4185 \ifnum\currentgrouplevel>\z@
4186 \ifnum\currentgrouplevel=\bbl@dirlevel
4187 \bbl@error{multiple-bidi}{\}\}\}%
4188 \bgroup\aftergroup#2\aftergroup\egroup
4189 \else
4190 \ifcase\currentgrouptype\or % 0 bottom
4191 \aftergroup#2% 1 simple {}
4192 \or
4193 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4194 \or
4195 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4196 \or\or\or % vbox vtop align
4197 \or
4198 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4199 \or\or\or\or\or\or % output math disc insert vcent mathchoice
4200 \or
4201 \aftergroup#2% 14 \begingroup
4202 \else
4203 \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4204 \fi
4205 \fi
4206 \bbl@dirlevel\currentgrouplevel
4207 \fi
4208 #1%
4209 \fi}
4210 \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4211 \let\bbl@bodydir\@gobble
4212 \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4213 \def\bbl@xebidipar{%
4214 \let\bbl@xebidipar\relax
4215 \TeXXeTstate\@ne
4216 \def\bbl@xeeverypar{%
4217 \ifcase\bbl@thepardir
4218 \ifcase\bbl@thetextdir\else\beginR\fi
4219 \else
4220 {\setbox\z@\lastbox\beginR\box\z@}%

```

```

4221     \fi}%
4222     \AddToHook{para/begin}{\bbl@xeeverypar}}
4223     \ifnum\bbl@bidimode>200 % Any xe bidi=
4224     \let\bbl@textdir@i@gobbletwo
4225     \let\bbl@xebidipar@empty
4226     \AddBabelHook{bidi}{foreign}{%
4227         \ifcase\bbl@thetextdir
4228             \BabelWrapText{\LR{##1}}%
4229         \else
4230             \BabelWrapText{\RL{##1}}%
4231         \fi}
4232     \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4233     \fi
4234 \fi

A tool for weak L (mainly digits). We also disable warnings with hyperref.

4235 \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4236 \AtBeginDocument{%
4237     \ifx\pdfstringdefDisableCommands\undefined\else
4238     \ifx\pdfstringdefDisableCommands\relax\else
4239         \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4240     \fi
4241     \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension .cfg. For instance the file norsk.cfg will be loaded when the language definition file norsk.ldf is loaded.

For plain-based formats we don't want to override the definition of \loadlocalcfg from plain.def.

```

4242 \bbl@trace{Local Language Configuration}
4243 \ifx\loadlocalcfg\undefined
4244     \@ifpackagewith{babel}{noconfigs}%
4245     {\let\loadlocalcfg@gobble}%
4246     {\def\loadlocalcfg#1{%
4247         \InputIfFileExists{#1.cfg}%
4248         {\typeout{*****^J%
4249             * Local config file #1.cfg used^^J%
4250             *}}%
4251         \@empty}}
4252 \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```

4253 \bbl@trace{Language options}
4254 \def\BabelDefinitionFile#1#2#3{}
4255 \let\bbl@afterlang\relax
4256 \let\BabelModifiers\relax
4257 \let\bbl@loaded@empty
4258 \def\bbl@load@language#1{%
4259     \InputIfFileExists{#1.ldf}%
4260     {\edef\bbl@loaded{\CurrentOption
4261         \ifx\bbl@loaded@empty\else,\bbl@loaded\fi}%
4262         \expandafter\let\expandafter\bbl@afterlang
4263         \csname\CurrentOption.ldf-h@k\endcsname
4264         \expandafter\let\expandafter\BabelModifiers
4265         \csname\bbl@mod@\CurrentOption\endcsname

```

```

4266 \bbl@exp{\AtBeginDocument{%
4267   \bbl@usehooks@lang{\CurrentOption}{\beginDocument}{\CurrentOption}}}%
4268 {\bbl@error{unknown-package-option}}{}{}{}%

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new `ldf` file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language has been set as metadata, read the info from the corresponding `ini` file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With `DocumentMetadata` we also force it with `\foreignlanguage` (this is also done in `bidi` texts).

```

4269 \ifx\GetDocumentProperties\undefined\else
4270   \let\bbl@beforeforeign\leavevmode
4271   \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4272   \let\bbl@collect@other\@empty
4273   \bbl@foreach\bbl@tempa{%
4274     \edef\bbl@metalang{#1}%
4275     \ifx\bbl@metalang\@empty\else
4276       \begingroup
4277         \expandafter
4278         \bbl@bcplookup{\bbl@metalang}%-\@empty-\@empty-\@empty\@@
4279         \ifx\bbl@bcp\relax
4280           \ifx\bbl@opt@main\@nnil
4281             \bbl@error{no-locale-for-meta}{\bbl@metalang}{}{}%
4282           \fi
4283         \else
4284           \bbl@read@ini{\bbl@bcp}\m@ne
4285           \xdef\bbl@collect@other{\bbl@collect@other,\language}%
4286           \GenericInfo{}{(babel) \spaces\@spaces\@spaces
4287             Passing '\language' to babel\@gobble}%
4288           \fi
4289         \endgroup
4290       \fi
4291     \ifx\bbl@collect@other\@empty\else
4292       \xdef\bbl@language@opts{\bbl@collect@other,\bbl@language@opts}%
4293     \fi
4294     \edef\bbl@metalang{\GetDocumentProperties{document/language}}%
4295     \ifx\bbl@metalang\@empty\else
4296       \begingroup
4297         \expandafter
4298         \bbl@bcplookup{\bbl@metalang}%-\@empty-\@empty-\@empty\@@
4299         \ifx\bbl@bcp\relax
4300           \ifx\bbl@opt@main\@nnil
4301             \bbl@error{no-locale-for-meta}{\bbl@metalang}{}{}%
4302           \fi
4303         \else
4304           \bbl@read@ini{\bbl@bcp}\m@ne
4305           \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4306           \ifx\bbl@opt@main\@nnil
4307             \global\let\bbl@opt@main\language
4308           \fi
4309           \GenericInfo{}{(babel) \spaces\@spaces\@spaces
4310             Passing '\language' to babel\@gobble}%
4311           \fi
4312         \endgroup
4313       \fi
4314     \fi
4315     \ifx\bbl@opt@config\@nnil
4316       \ifpackagewith{babel}{noconfigs}{}%
4317       {\InputIfFileExists{bblopts.cfg}%

```

```

4318      {\bbl@info{Configuration files are deprecated, as\\%
4319              they can break document portability.\\%
4320              Reported}%
4321      \typeout{*****^J%
4322              * Local config file bblopts.cfg used^^J%
4323              *}}%
4324      {}}%
4325 \else
4326 \InputIfFileExists{\bbl@opt@config.cfg}%
4327 {\bbl@info{Configuration files are deprecated, as\\%
4328         they can break document portability.\\%
4329         Reported}%
4330 \typeout{*****^J%
4331         * Local config file \bbl@opt@config.cfg used^^J%
4332         *}}%
4333 {\bbl@error{config-not-found}}{}{}}%
4334 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-(name).tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `@import`).

```

4335 \def\BabelBeforeIni#1#2{%
4336 \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4337 \let\bbl@tempb\@empty
4338 #2%
4339 \edef\bbl@toload{%
4340 \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4341 \bbl@toload@last}%
4342 \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}
4343 \def\BabelDefinitionFile#1#2#3{%
4344 \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4345 \@namedef{\bbl@preldf@\CurrentOption}{#3}%
4346 \endinput}%

```

For efficiency, first preprocess the class options to remove those with `=`, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\TeX$ layer.

```

4347 \def\bbl@tempf{,}
4348 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4349 \in@{=}{#1}%
4350 \ifin@ \else
4351 \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4352 \fi}

```

Store the class/package options in a list. If there is an explicit main, it’s placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4353 \let\bbl@toload\@empty
4354 \let\bbl@toload@last\@empty
4355 \let\bbl@unkopt\@gobble %% <- Ugly
4356 \edef\bbl@tempc{%
4357 \bbl@tempf,@,\bbl@language@opts

```



```

4358 \ifx\bbl@opt@main@nnil\else,\bbl@opt@main\fi}
4359 \let\BabelLocalesTentative\bbl@tempc
4360 %
4361 \bbl@foreach\bbl@tempc{%
4362   \in{@}{#1}% <- Ugly
4363   \ifin@
4364     \def\bbl@unkopt##1{%
4365       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}{}}%
4366   \else
4367     \def\CurrentOption{#1}%
4368     \bbl@xin@{/#1//}{\bbl@toload@last}% Collapse consecutive
4369     \ifin@else
4370       \lowercase{\InputIfFileExists{babel-#1.tex}}{}}{%
4371       \IfFileExists{#1.ldf}%
4372       {\edef\bbl@toload{%
4373         \ifx\bbl@toload@empty\else\bbl@toload,\fi
4374         \bbl@toload@last}%
4375       \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4376       {\bbl@unkopt{#1}}}%
4377     \fi
4378   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4379 \ifx\bbl@opt@main@nnil
4380   \ifx\bbl@toload@last@empty
4381     \def\bbl@toload@last{0/0//nil//und-x-nil-nil}
4382     \bbl@info{%
4383       You haven't specified a language as a class or package\\%
4384       option. I'll load 'nil'. Reported}
4385   \fi
4386 \else
4387   \let\bbl@tempc@empty
4388   \bbl@foreach\bbl@toload{%
4389     \bbl@xin@{/#1//}{\bbl@opt@main//}{#1}%
4390     \ifin@else
4391       \bbl@add@list\bbl@tempc{#1}%
4392     \fi}
4393   \let\bbl@toload\bbl@tempc
4394 \fi
4395 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the 'ini' file or the pair 'ini'/'ldf' file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if 'ini', 1 if 'ldf'.

```

4396 \def\AfterBabelLanguage#1{%
4397   \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang}{}}
4398 \NewHook{babel/presets}
4399 \UseHook{babel/presets}
4400 %
4401 \let\bbl@tempb@empty
4402 \def\bbl@tempc#1/#2//#3//#4/#5\@@{%
4403   \count@z@
4404   \ifnum#2=\@m % if no \BabelDefinitionFile
4405     \ifnum#1=z@ % not main. -- % if provide=, provide*=!
4406       \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@@
4407       \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4408     \fi
4409   \else % 10 = main -- % if provide=, provide*=!
4410     \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@@
4411     \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4412   \fi

```

```

4413 \fi
4414 \else
4415 \ifnum#1=\z@ % not main
4416 \ifnum\bb@iniflag>\@ne\else % if ø, provide
4417 \ifcase#2\count@\@ne\else\ifcase\bb@engine\count@\@ne\fi\fi
4418 \fi
4419 \else % l0 = main
4420 \ifodd\bb@iniflag\else % if provide+, provide*
4421 \ifcase#2\count@\@ne\else\ifcase\bb@engine\count@\@ne\fi\fi
4422 \fi
4423 \fi
4424 \bb@tempd{#1}{#2}{#3}{#4}{#5}%
4425 \fi}

Based on the value of \count@, do the actual loading. If 'ldf', we load the basic info from the 'ini' file
before.

4426 \def\bb@tempd#1#2#3#4#5{%
4427 \DeclareOption{#3}{}%
4428 \ifcase\count@
4429 \bb@exp{\bb@add\bb@tempb{%
4430 \global\let\bb@afterload\@empty
4431 \\\@nameuse{bb@preini#3}%
4432 \\\bb@ldfinit
4433 \def\CurrentOption{#3}%
4434 \\\babelprovide[import=#4,\ifnum#1=\z@\else\bb@opt@provide,main\fi]{#3}%
4435 \\\bb@afterldf
4436 \\\bb@afterload}}%
4437 \else
4438 \bb@add\bb@tempb{%
4439 \global\let\bb@afterload\@empty
4440 \def\CurrentOption{#3}%
4441 \let\localename\CurrentOption
4442 \let\language\localename
4443 \def\BabelIniTag{#4}%
4444 \@nameuse{bb@preldf#3}%
4445 \begingroup
4446 \bb@id@assign
4447 \bb@read@ini{\BabelIniTag}0%
4448 \endgroup
4449 \bb@load@language{#5}%
4450 \bb@afterload}%
4451 \fi}
4452 %
4453 \bb@foreach\bb@toload{\bb@tempc#1\@@}
4454 \bb@tempb
4455 \DeclareOption*{}
4456 \ProcessOptions
4457 %
4458 \bb@exp{%
4459 \\\AtBeginDocument{\bb@usehooks@lang{/}{begindocument}{}}}%
4460 \def\AfterBabelLanguage{\bb@error{late-after-babel}{}}
4461 \AddToHook{begindocument/end}{%
4462 \bb@info{Main language set to '\mainlocalename'\@gobble}}
4463 </package>

```

6. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\text{T}_{\text{E}}\text{X}$ users might want to use some of the features of the babel system too, care has to be taken that plain $\text{T}_{\text{E}}\text{X}$ can process the files. For this reason the current format will have to be

checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the `babel` names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```
4464 \kernel
4465 \let\bbl@onlyswitch\@empty
4466 \input babel.def
4467 \let\bbl@onlyswitch\@undefined
4468 \kernel
```

7. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^`, `M`, `%` and `=` are reset before loading the file.

```
4469 \kernel
4470 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4471 \catcode`\:=12 \catcode`\.,=12 \catcode`\.=12 \catcode`\-=12
4472 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4473 \catcode`\@=11 \catcode`\^=7
4474 %
4475 \ifx\MessageBreak\@undefined
4476 \gdef\bbl@error@i#1#2{%
4477 \begingroup
4478 \newlinechar=`^^J
4479 \def\{^^J(babel) }%
4480 \errhelp{#2}\errmessage{\{#1}%
4481 \endgroup}
4482 \else
4483 \gdef\bbl@error@i#1#2{%
4484 \begingroup
4485 \def\{\MessageBreak}%
4486 \PackageError{babel}{#1}{#2}%
4487 \endgroup}
4488 \fi
4489 \def\bbl@errmessage#1#2#3{%
4490 \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4491 \bbl@error@i{#2}{#3}}
4492 % Implicit #2#3#4:
4493 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4494 %
4495 \bbl@errmessage{not-yet-available}
4496 {Not yet available}%
4497 {Find an armchair, sit down and wait}
4498 \bbl@errmessage{bad-package-option}%
4499 {Bad option '#1=#2'. Either you have misspelled the\\%
4500 key or there is a previous setting of '#1'. Valid\\%
4501 keys are, among others, 'shorthands', 'main', 'bidi',\\%
4502 'strings', 'config', 'headfoot', 'safe', 'math'.}%
4503 {See the manual for further details.}
4504 \bbl@errmessage{base-on-the-fly}
4505 {For a language to be defined on the fly 'base'\\%
4506 is not enough, and the whole package must be\\%
4507 loaded. Either delete the 'base' option or\\%
4508 request the languages explicitly}%
4509 {See the manual for further details.}
4510 \bbl@errmessage{undefined-language}
4511 {You haven't defined the language '#1' yet.\\%
```

```

4512     Perhaps you misspelled it or your installation\\%
4513     is not complete}%
4514     {Your command will be ignored, type <return> to proceed}
4515 \bbl@errmessage{invalid-ini-name}
4516     {'#1' not valid with the 'ini' mechanism.\\%
4517     I think you want '#2' instead. You may continue,\\%
4518     but you should fix the name. See the babel manual\\%
4519     for the available locales with 'provide'}%
4520     {See the manual for further details.}
4521 \bbl@errmessage{shorthand-is-off}
4522     {I can't declare a shorthand turned off (\string#2)}
4523     {Sorry, but you can't use shorthands which have been\\%
4524     turned off in the package options}
4525 \bbl@errmessage{not-a-shorthand}
4526     {The character '\string #1' should be made a shorthand character;\\%
4527     add the command \string\usesshorthands\string{#1\string} to
4528     the preamble.\\%
4529     I will ignore your instruction}%
4530     {You may proceed, but expect unexpected results}
4531 \bbl@errmessage{not-a-shorthand-b}
4532     {I can't switch '\string#2' on or off--not a shorthand\\%
4533     This character is not a shorthand. Maybe you made\\%
4534     a typing mistake?}%
4535     {I will ignore your instruction.}
4536 \bbl@errmessage{unknown-attribute}
4537     {The attribute #2 is unknown for language #1.}%
4538     {Your command will be ignored, type <return> to proceed}
4539 \bbl@errmessage{missing-group}
4540     {Missing group for string \string#1}%
4541     {You must assign strings to some category, typically\\%
4542     captions or extras, but you set none}
4543 \bbl@errmessage{only-lua-xe}
4544     {This macro is available only in LuaLaTeX and XeLaTeX.}%
4545     {Consider switching to these engines.}
4546 \bbl@errmessage{only-lua}
4547     {This macro is available only in LuaLaTeX}%
4548     {Consider switching to that engine.}
4549 \bbl@errmessage{unknown-provide-key}
4550     {Unknown key '#1' in \string\babelprovide}%
4551     {See the manual for valid keys}%
4552 \bbl@errmessage{unknown-mapfont}
4553     {Option '\bbl@KVP@mapfont' unknown for\\%
4554     mapfont. Use 'direction'}%
4555     {See the manual for details.}
4556 \bbl@errmessage{no-ini-file}
4557     {There is no ini file for the requested language\\%
4558     (#1: \language). Perhaps you misspelled it or your\\%
4559     installation is not complete}%
4560     {Fix the name or reinstall babel.}
4561 \bbl@errmessage{digits-is-reserved}
4562     {The counter name 'digits' is reserved for mapping\\%
4563     decimal digits}%
4564     {Use another name.}
4565 \bbl@errmessage{limit-two-digits}
4566     {Currently two-digit years are restricted to the\\%
4567     range 0-9999}%
4568     {There is little you can do. Sorry.}
4569 \bbl@errmessage{alphabetic-too-large}
4570     {Alphabetic numeral too large (#1)}%
4571     {Currently this is the limit.}
4572 \bbl@errmessage{no-ini-info}
4573     {I've found no info for the current locale.\\%
4574     The corresponding ini file has not been loaded\\%

```

```

4575     Perhaps it doesn't exist}%
4576     {See the manual for details.}
4577 \bbl@errmessage{unknown-ini-field}
4578     {Unknown field '#1' in \string\BCPdata.\\%
4579     Perhaps you misspelled it}%
4580     {See the manual for details.}
4581 \bbl@errmessage{unknown-locale-key}
4582     {Unknown key for locale '#2':\\%
4583     #3\\%
4584     \string#1 will be set to \string\relax}%
4585     {Perhaps you misspelled it.}%
4586 \bbl@errmessage{adjust-only-vertical}
4587     {Currently, #1 related features can be adjusted only\\%
4588     in the main vertical list}%
4589     {Maybe things change in the future, but this is what it is.}
4590 \bbl@errmessage{layout-only-vertical}
4591     {Currently, layout related features can be adjusted only\\%
4592     in vertical mode}%
4593     {Maybe things change in the future, but this is what it is.}
4594 \bbl@errmessage{bidi-only-lua}
4595     {The bidi method 'basic' is available only in\\%
4596     luatex. I'll continue with 'bidi=default', so\\%
4597     expect wrong results.\\%
4598     Suggested actions:\\%
4599     * If possible, switch to luatex, as xetex is not\\%
4600     recommend anymore.\\
4601     * If you can't, try 'bidi=bidi' with xetex.\\%
4602     * With pdftex, only 'bidi=default' is available.}%
4603     {See the manual for further details.}
4604 \bbl@errmessage{multiple-bidi}
4605     {Multiple bidi settings inside a group\\%
4606     I'll insert a new group, but expect wrong results.\\%
4607     Suggested action:\\%
4608     * Add a new group where appropriate.}
4609     {See the manual for further details.}
4610 \bbl@errmessage{unknown-package-option}
4611     {Unknown option '\CurrentOption'.\\%
4612     Suggested actions:\\%
4613     * Make sure you haven't misspelled it\\%
4614     * Check in the babel manual that it's supported\\%
4615     * If supported and it's a language, you may\\%
4616     \space\space need in some distributions a separate\\%
4617     \space\space installation\\%
4618     * If installed, check there isn't an old\\%
4619     \space\space version of the required files in your system\\%
4620     * If it's an unsupported language, create it with\\%
4621     \string\babelprovide (see the manual)}
4622     {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4623     activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4624     headfoot=, strings=, config=, hyphenmap=, or a language name.}
4625 \bbl@errmessage{config-not-found}
4626     {Local config file '\bbl@opt@config.cfg' not found.\\%
4627     Suggested actions:\\%
4628     * Make sure you haven't misspelled it in config=\\%
4629     * Check it exists and it's in the correct path}%
4630     {Perhaps you misspelled it.}
4631 \bbl@errmessage{late-after-babel}
4632     {Too late for \string\AfterBabelLanguage}%
4633     {Languages have been loaded, so I can do nothing}
4634 \bbl@errmessage{double-hyphens-class}
4635     {Double hyphens aren't allowed in \string\babelcharclass\\%
4636     because it's potentially ambiguous}%
4637     {See the manual for further info}

```

```

4638 \bbl@errmessage{unknown-interchar}
4639   {'#1' for '\language' cannot be enabled.\\%
4640   Maybe there is a typo}%
4641   {See the manual for further details.}
4642 \bbl@errmessage{unknown-interchar-b}
4643   {'#1' for '\language' cannot be disabled.\\%
4644   Maybe there is a typo}%
4645   {See the manual for further details.}
4646 \bbl@errmessage{charproperty-only-vertical}
4647   {\string\babelcharproperty\space can be used only in\\%
4648   vertical mode (preamble or between paragraphs)}%
4649   {See the manual for further info}
4650 \bbl@errmessage{unknown-char-property}
4651   {No property named '#2'. Allowed values are\\%
4652   direction (bc), mirror (bmg), and linebreak (lb)}%
4653   {See the manual for further info}
4654 \bbl@errmessage{bad-transform-option}
4655   {Bad option '#1' in a transform.\\%
4656   I'll ignore it but expect more errors}%
4657   {See the manual for further info.}
4658 \bbl@errmessage{font-conflict-transforms}
4659   {Transforms cannot be re-assigned to different\\%
4660   fonts. The conflict is in '\bbl@kv@label'.\\%
4661   Apply the same fonts or use a different label}%
4662   {See the manual for further details.}
4663 \bbl@errmessage{transform-not-available}
4664   {'#1' for '\language' cannot be enabled.\\%
4665   Maybe there is a typo or it's a font-dependent transform}%
4666   {See the manual for further details.}
4667 \bbl@errmessage{transform-not-available-b}
4668   {'#1' for '\language' cannot be disabled.\\%
4669   Maybe there is a typo or it's a font-dependent transform}%
4670   {See the manual for further details.}
4671 \bbl@errmessage{year-out-range}
4672   {Year out of range.\\%
4673   The allowed range is #1}%
4674   {See the manual for further details.}
4675 \bbl@errmessage{only-pdftex-lang}
4676   {The '#1' ldf style doesn't work with #2,\\%
4677   but you can use the ini locale instead.\\%
4678   Try adding 'provide=*' to the option list. You may\\%
4679   also want to set 'bidi=' to some value}%
4680   {See the manual for further details.}
4681 \bbl@errmessage{hyphenmins-args}
4682   {\string\babelhyphenmins\ accepts either the optional\\%
4683   argument or the star, but not both at the same time}%
4684   {See the manual for further details.}
4685 \bbl@errmessage{no-locale-for-meta}
4686   {There isn't currently a locale for the 'lang' requested\\%
4687   in the PDF metadata ('#1'). To fix it, you can\\%
4688   set explicitly a similar language (using the same\\%
4689   script) with the key main= when loading babel. If you\\%
4690   continue, I'll fallback to the 'nil' language, with\\%
4691   tag 'und' and script 'Latn', but expect a bad font\\%
4692   rendering with other scripts. You may also need set\\%
4693   explicitly captions and date, too}%
4694   {See the manual for further details.}
4695 </errors>
4696 <*patterns>

```

8. Loading hyphenation patterns

The following code is meant to be read by $\text{\texttt{iniT\TeX}}$ because it should instruct $\text{\texttt{T\TeX}}$ to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```
4697 <@Make sure ProvidesFile is defined>
4698 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4699 \xdef\bbl@format{\jobname}
4700 \def\bbl@version{<@version@>}
4701 \def\bbl@date{<@date@>}
4702 \ifx\AtBeginDocument\undefined
4703   \def\@empty{}
4704 \fi
4705 <@Define core switching macros@>
```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```
4706 \def\process@line#1#2 #3 #4 {%
4707   \ifx=#1%
4708     \process@synonym{#2}%
4709   \else
4710     \process@language{#1#2}{#3}{#4}%
4711   \fi
4712   \ignorespaces}
```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```
4713 \toks@{}
4714 \def\bbl@languages{}
```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmin` parameters for the synonym.

```
4715 \def\process@synonym#1{%
4716   \ifnum\last@language=\m@ne
4717     \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4718   \else
4719     \expandafter\chardef\csname l@#1\endcsname\last@language
4720     \wlog{\string\l@#1=\string\language\the\last@language}%
4721     \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4722       \csname\language\hyphenmins\endcsname
4723     \let\bbl@elt\relax
4724     \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%
4725   \fi}
```

\process@language The macro `\process@language` is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call `\addlanguage` to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file `language.dat` by adding for instance ‘:T1’ to the name of the language. The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to `\lefthyphenmin` and `\righthyphenmin`. $\text{\texttt{T\TeX}}$ does not keep track of these assignments. Therefore we try to detect such assignments and store them in the `\<language>hyphenmins` macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the `\lccode` en `\uccode` arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the `\patterns` command acts globally so its effect will be remembered.

Then we globally store the settings of `\lefthyphenmin` and `\righthyphenmin` and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

`\bbl@languages` saves a snapshot of the loaded languages in the form `\bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}`. Note the last 2 arguments are empty in ‘dialects’ defined in `language.dat` with `=`. Note also the language name can have encoding info.

Finally, if the counter `\language` is equal to zero we execute the synonyms stored.

```

4726 \def\process@language#1#2#3{%
4727   \expandafter\addlanguage\csname l@#1\endcsname
4728   \expandafter\language\csname l@#1\endcsname
4729   \edef\language#1{%
4730     \bbl@hook@everylanguage{#1}%
4731     % > luatex
4732     \bbl@get@enc#1::\@@@
4733   \begingroup
4734     \lefthyphenmin\m@ne
4735     \bbl@hook@loadpatterns{#2}%
4736     % > luatex
4737     \ifnum\lefthyphenmin=\m@ne
4738     \else
4739       \expandafter\xdef\csname #1hyphenmins\endcsname{%
4740         \the\lefthyphenmin\the\righthyphenmin}%
4741     \fi
4742   \endgroup
4743   \def\bbl@tempa{#3}%
4744   \ifx\bbl@tempa\@empty\else
4745     \bbl@hook@loadexceptions{#3}%
4746     % > luatex
4747   \fi
4748   \let\bbl@elt\relax
4749   \edef\bbl@languages{%
4750     \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4751   \ifnum\the\language=\z@
4752     \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4753       \set@hyphenmins\tw@\thr@@\relax
4754     \else
4755       \expandafter\expandafter\expandafter\set@hyphenmins
4756       \csname #1hyphenmins\endcsname
4757     \fi
4758     \the\toks@
4759     \toks@{}%
4760   \fi}

```

`\bbl@get@enc`

`\bbl@hyph@enc` The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. It uses delimited arguments to achieve this.

```

4761 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides `luatex`, format-specific configuration files are taken into account. `loadkernel` currently loads nothing, but define some basic macros instead.

```

4762 \def\bbl@hook@everylanguage#1{}
4763 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4764 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4765 \def\bbl@hook@loadkernel#1{%
4766   \def\addlanguage{\csname newlanguage\endcsname}%

```



```

4767 \def\adddialect##1##2{%
4768   \global\chardef##1##2\relax
4769   \log{\string##1 = a dialect from \string\language##2}}%
4770 \def\iflanguage##1{%
4771   \expandafter\ifx\csname l@##1\endcsname\relax
4772     \@nolanner{##1}%
4773   \else
4774     \ifnum\csname l@##1\endcsname=\language
4775       \expandafter\expandafter\expandafter\@firstoftwo
4776     \else
4777       \expandafter\expandafter\expandafter\@secondoftwo
4778     \fi
4779   \fi}%
4780 \def\providehyphenmins##1##2{%
4781   \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4782     \@namedef{##1hyphenmins}{##2}%
4783   \fi}%
4784 \def\set@hyphenmins##1##2{%
4785   \lefthyphenmin##1\relax
4786   \righthyphenmin##2\relax}%
4787 \def\selectlanguage{%
4788   \errhelp{Selecting a language requires a package supporting it}%
4789   \errmessage{No multilingual package has been loaded}}%
4790 \let\foreignlanguage\selectlanguage
4791 \let\otherlanguage\selectlanguage
4792 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4793 \def\bbl@usehooks##1##2{%
4794   \def\setlocale{%
4795     \errhelp{Find an armchair, sit down and wait}%
4796     \errmessage{(babel) Not yet available}}%
4797   \let\uselocale\setlocale
4798   \let\locale\setlocale
4799   \let\selectlocale\setlocale
4800   \let\localename\setlocale
4801   \let\textlocale\setlocale
4802   \let\textlanguage\setlocale
4803   \let\languagetext\setlocale}
4804 \begingroup
4805 \def\AddBabelHook#1#2{%
4806   \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4807     \def\next{\toks1}%
4808   \else
4809     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname####1}%
4810   \fi
4811   \next}
4812 \ifx\directlua\@undefined
4813   \ifx\XeTeXinputencoding\@undefined\else
4814     \input xebabel.def
4815   \fi
4816 \else
4817   \input luababel.def
4818 \fi
4819 \openin1 = babel-\bbl@format.cfg
4820 \ifeof1
4821 \else
4822   \input babel-\bbl@format.cfg\relax
4823 \fi
4824 \closein1
4825 \endgroup
4826 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4827 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```
4828 \def\language{english}%
4829 \ifeof1
4830   \message{I couldn't find the file language.dat,\space
4831           I will try the file hyphen.tex}
4832   \input hyphen.tex\relax
4833   \chardef\l@english\z@
4834 \else
```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```
4835 \last@language@m@ne
```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```
4836 \loop
4837   \endlinechar@m@ne
4838   \read1 to \bbl@line
4839   \endlinechar`^^M
```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```
4840   \if T\ifeof1F\fi T\relax
4841   \ifx\bbl@line\empty\else
4842     \edef\bbl@line{\bbl@line\space\space\space}%
4843     \expandafter\process@line\bbl@line\relax
4844   \fi
4845 \repeat
```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```
4846 \begingroup
4847   \def\bbl@elt#1#2#3#4{%
4848     \global\language=#2\relax
4849     \gdef\language{#1}%
4850     \def\bbl@elt##1##2##3##4{}}%
4851   \bbl@languages
4852 \endgroup
4853 \fi
4854 \closein1
```

We add a message about the fact that `babel` is loaded in the format and with which language patterns to the `\everyjob` register.

```
4855 \if\the\toks@\else
4856   \errhelp{language.dat loads no language, only synonyms}
4857   \errmessage{Orphan language synonym}
4858 \fi
```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```
4859 \let\bbl@line\@undefined
4860 \let\process@line\@undefined
4861 \let\process@synonym\@undefined
4862 \let\process@language\@undefined
4863 \let\bbl@get@enc\@undefined
4864 \let\bbl@hyph@enc\@undefined
4865 \let\bbl@tempa\@undefined
4866 \let\bbl@hook@loadkernel\@undefined
4867 \let\bbl@hook@everylanguage\@undefined
```

```

4868 \let\bbl@hook@loadpatterns\@undefined
4869 \let\bbl@hook@loadexceptions\@undefined
4870 </patterns>

```

Here the code for `iniTeX` ends.

9. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdftex`).

```

4871 <<*More package options>> ≡
4872 \chardef\bbl@bidimode\z@
4873 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4874 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4875 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4876 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4877 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4878 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4879 <</More package options>>

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `bbl@font` replaces hardcoded font names inside `\.family` by the corresponding macro `\.default`.

```

4880 <<*Font selection>> ≡
4881 \bbl@trace{Font handling with fontspec}
4882 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4883 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@cckstdfont}
4884 \DisableBabelHook{babel-fontspec}
4885 \@onlypreamble\babelfont
4886 \ifx\NewDocumentCommand\@undefined\else % Not plain
4887   \NewDocumentCommand\babelfont{0}{m0}{m0}{}%
4888     \bbl@bblfont@o{#1}{#2}{#3,#5}{#4}}
4889 \fi
4890 \newcommand\bbl@bblfont@o[2][]{% 1=langs/scripts 2=fam
4891   \ifx\fontspec\@undefined
4892     \usepackage{fontspec}%
4893   \fi
4894   \EnableBabelHook{babel-fontspec}%
4895   \edef\bbl@tempa{#1}%
4896   \def\bbl@tempb{#2}% Used by \bbl@bblfont
4897   \bbl@bblfont}
4898 \newcommand\bbl@bblfont[2][]{% 1=features 2=fontname, @font=rm|sf|tt
4899   \bbl@ifunset{\bbl@tempb family}%
4900     {\bbl@providfam{\bbl@tempb}}%
4901     {}%
4902   % For the default font, just in case:
4903   \bbl@ifunset{\bbl@sys\languagename}{\bbl@provid@sys{\languagename}}{%
4904     \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
4905     {\bbl@csarg\edef{\bbl@tempb dflt@}{<#1>{#2}}% save bbl@rmdflt@
4906     \bbl@exp{%
4907       \let<\bbl@tempb dflt@\languagename>\<\bbl@tempb dflt@>%
4908       \bbl@font@set<\bbl@tempb dflt@\languagename>%
4909       \<\bbl@tempb default>\<\bbl@tempb family>}}%
4910     {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
4911       \bbl@csarg\def{\bbl@tempb dflt@##1}{<#1>{#2}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4912 \def\bbl@providfam#1{%
4913   \bbl@exp{%
4914     \newcommand\<#1default>{}% Just define it
4915     \bbl@add@list\bbl@font@fams{#1}%

```

```

4916   \\NewHook{#1family}%
4917   \\DeclareRobustCommand\<#1family>{%
4918     \\not@math@alphabet\<#1family>\relax
4919     % \\prepare@family@series@update{#1}\<#1default>% TODO. Fails
4920     \\fontfamily\<#1default>%
4921     \\UseHook{#1family}%
4922     \\selectfont}%
4923   \\DeclareTextFontCommand{\<text#1>}\<#1family>}}

```

The following macro is activated when the hook babel - fonts spec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4924 \def\bbl@nostdfont#1{%
4925   \bbl@once{nostdfam-\f@family}%
4926   {\bbl@infowarn{The current font is not a babel standard family:\\%
4927     #1%
4928     \fontname\font\\%
4929     There is nothing intrinsically wrong, and you can\\%,
4930     ignore this message altogether if you do not need\\%
4931     this font. If they are used in the document, be aware\\%
4932     'babel' will not set Script and Language for it, so\\%
4933     you may consider defining a new family with \string\babelfont.\\%
4934     See the manual for further details about \string\babelfont.
4935     Reported}}%
4936   }%
4937 \gdef\bbl@switchfont{%
4938   \bbl@ifunset{bbl@lsys@\language}\bbl@provide@lsys{\language}}}%
4939   \bbl@exp{% e.g., Arabic -> arabic
4940     \lowercase{\edef\bbl@tempa{\bbl@c{sname}}}%
4941   \bbl@foreach\bbl@font@fams{%
4942     \bbl@ifunset{bbl@##1dflt@\language}% (1) language?
4943     {\bbl@ifunset{bbl@##1dflt*\bbl@tempa}% (2) from script?
4944       {\bbl@ifunset{bbl@##1dflt@}% 2=F - (3) from generic?
4945         {}% 123=F - nothing!
4946         {\bbl@exp{% 3=T - from generic
4947           \global\let\bbl@##1dflt@\language}%
4948           \<bbl@##1dflt@>}}}%
4949       {\bbl@exp{% 2=T - from script
4950         \global\let\bbl@##1dflt@\language}%
4951         \<bbl@##1dflt*\bbl@tempa>}}}%
4952     }% 1=T - language, already defined
4953   \def\bbl@tempa{\bbl@nostdfont}}}%
4954   \bbl@foreach\bbl@font@fams{% don't gather with prev for
4955     \bbl@ifunset{bbl@##1dflt@\language}%
4956     {\bbl@cs{famrst@##1}%
4957     \global\bbl@csarg\let{famrst@##1}\relax}%
4958     {\bbl@exp{% order is relevant.
4959       \\bbl@add\\originalTeX%
4960       \\bbl@font@rst{\bbl@c{sname}}}%
4961       \<##1default>\<##1family>{##1}}%
4962       \\bbl@font@set\<bbl@##1dflt@\language>% the main part!
4963       \<##1default>\<##1family>}}}%
4964   \bbl@ifrestoring{\bbl@tempa}}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with \babelfont.

```

4965 \ifx\f@family\undefined\else % if latex
4966   \ifcase\bbl@engine % if pdftex
4967     \let\bbl@ckeckstdfonts\relax
4968   \else
4969     \def\bbl@ckeckstdfonts{%
4970       \begingroup
4971       \global\let\bbl@ckeckstdfonts\relax
4972       \let\bbl@tempa\empty
4973       \bbl@foreach\bbl@font@fams{%

```

```

4974 \bbl@ifunset{bbl@##1dflt@}%
4975 {\@nameuse{##1family}}%
4976 \bbl@csarg\gdef{WFF@f@family}{}% Flag
4977 \bbl@exp{\\bbl@add\\bbl@tempa{* \<##1family>= \f@family\\}%
4978 \space\space\fontname\font\\}%
4979 \bbl@csarg\xdef{##1dflt@}{\f@family}%
4980 \expandafter\xdef\csname ##1default\endcsname{\f@family}%
4981 {}}%
4982 \ifx\bbl@tempa\empty\else
4983 \bbl@infowarn{The following font families will use the default\\%
4984 settings for all or some languages:\\%
4985 \bbl@tempa
4986 There is nothing intrinsically wrong with it, but\\%
4987 'babel' will no set Script and Language, which could\\%
4988 be relevant in some languages. If your document uses\\%
4989 these families, consider redefining them with \string\babelfont.\\%
4990 Reported}%
4991 \fi
4992 \endgroup}
4993 \fi
4994 \fi

```

Now the macros defining the font with fontspec.

When there are repeated keys in fontspec, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes bx/sc is the correct font, but sometimes points to b/n, even if b/sc exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

4995 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
4996 \bbl@xin@{<>}{#1}%
4997 \ifin@
4998 \bbl@exp{\\bbl@fontspec@set\\#1\expandafter\@gobbletwo#1\\#3}%
4999 \fi
5000 \bbl@exp{%
5001 \def\\#2#1% e.g., \rmdefault{\bbl@rmdflt@lang}
5002 \\bbl@ifsamestring{#2}{\f@family}%
5003 {\\#3%
5004 \\bbl@ifsamestring{\f@series}{\bfdefault}{\\bfseries}}}%
5005 \let\\bbl@tempa\relax}%
5006 {}}}

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5007 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5008 \let\bbl@tempa\bbl@mapselect
5009 \edef\bbl@tempb{\bbl@stripslash#4}% Catcodes hack (better pass it).
5010 \bbl@exp{\\bbl@replace\\bbl@tempb{\bbl@stripslash\family/}}}%
5011 \let\bbl@mapselect\relax
5012 \let\bbl@temp@fam#4% e.g., '\rmfamily', to be restored below
5013 \let#4\empty % Make sure \renewfontfamily is valid
5014 \bbl@set@renderer
5015 \bbl@exp{%
5016 \let\\bbl@temp@pfam<\bbl@stripslash#4\space>% e.g., '\rmfamily '
5017 <\keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}}%
5018 {\\newfontscript{\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5019 <\keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}}%
5020 {\\newfontlanguage{\bbl@cl{lname}}{\bbl@cl{lotf}}}%

```

```

5021   \\renewfontfamily\\#4%
5022   [\bbl@cl{lsys},% xetex removes unknown features :-(
5023   \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5024   #2]}{#3}% i.e., \bbl@exp{..}{#3}
5025   \bbl@unset@renderer
5026   \begingroup
5027   #4%
5028   \xdef#1{\f@family}%      e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5029   \endgroup
5030   \bbl@xin@{\string>\string s\string s\string u\string b\string*}%
5031   {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5032   \ifin@
5033   \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5034   \fi
5035   \bbl@xin@{\string>\string s\string s\string u\string b\string*}%
5036   {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5037   \ifin@
5038   \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5039   \fi
5040   \let#4\bbl@temp@fam
5041   \bbl@exp{\let<\bbl@stripslash#4\space>}\bbl@temp@pfam
5042   \let\bbl@mapselect\bbl@tempe}%

```

font@rst and famrst are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5043 \def\bbl@font@rst#1#2#3#4{%
5044   \bbl@csarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with \babelfont.

```

5045 \def\bbl@font@fams{rm,sf,tt}
5046 <</Font selection>>

```

10. Hooks for XeTeX and LuaTeX

10.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to utf8, which seems a sensible default.

Now, the code.

```

5047 <*<xtex>
5048 \def\BabelStringsDefault{unicode}
5049 \let\xebbl@stop\relax
5050 \AddBabelHook{xetex}{encodedcommands}{%
5051   \def\bbl@tempa{#1}%
5052   \ifx\bbl@tempa\@empty
5053     \XeTeXinputencoding"bytes"%
5054   \else
5055     \XeTeXinputencoding"#1"%
5056   \fi
5057   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5058 \AddBabelHook{xetex}{stopcommands}{%
5059   \xebbl@stop
5060   \let\xebbl@stop\relax}
5061 \def\bbl@input@classes{% Used in CJK intraspaces
5062   \input{load-unicode-xetex-classes.tex}%
5063   \let\bbl@input@classes\relax}
5064 \def\bbl@intraspace#1 #2 #3\@@{%
5065   \bbl@csarg\gdef{xeisp@{\languagename}%
5066     {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5067 \def\bbl@intrapenalty#1\@@{%
5068   \bbl@csarg\gdef{xeipn@{\languagename}%

```

```

5069 {\XeTeXlinebreakpenalty #1\relax}}
5070 \def\bbl@provide@intraspace{%
5071 \bbl@xin@{/s}{/\bbl@cl{lnbrk}}%
5072 \ifin@else\bbl@xin@{/c}{/\bbl@cl{lnbrk}}\fi
5073 \ifin@
5074 \bbl@ifunset{\bbl@intsp@{\language\language}}{%
5075 {\expandafter\ifx\csname bbl@intsp@{\language\language}\endcsname\@empty\else
5076 \ifx\bbl@KVP@intraspace\@nnil
5077 \bbl@exp{%
5078 \\\bbl@intraspace\bbl@cl{intsp}\\\@}%
5079 \fi
5080 \ifx\bbl@KVP@intrapenalty\@nnil
5081 \bbl@intrapenalty0\@@
5082 \fi
5083 \fi
5084 \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5085 \expandafter\bbl@intraspace\bbl@KVP@intraspace\@@
5086 \fi
5087 \ifx\bbl@KVP@intrapenalty\@nnil\else
5088 \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5089 \fi
5090 \bbl@exp{%
5091 \\\bbl@add\<extras\language>{%
5092 \XeTeXlinebreaklocale "\bbl@cl{tbcpr}"%
5093 \<bbl@xeisp@\language>%
5094 \<bbl@xeipn@\language>%
5095 \\\bbl@toglobal\<extras\language>%
5096 \\\bbl@add\<noextras\language>{%
5097 \XeTeXlinebreaklocale ""}%
5098 \\\bbl@toglobal\<noextras\language>%
5099 \ifx\bbl@ispace\undefined
5100 \gdef\bbl@ispace{\bbl@cl{xeisp}}%
5101 \ifx\AtBeginDocument\@notprerr
5102 \expandafter\@secondoftwo % to execute right now
5103 \fi
5104 \AtBeginDocument{\bbl@patchfont{\bbl@ispace}}%
5105 \fi}%
5106 \fi}
5107 \ifx\DisableBabelHook\@undefined\endinput\fi
5108 \let\bbl@set@renderer\relax
5109 \let\bbl@unset@renderer\relax
5110 <@Font selection@>
5111 \def\bbl@provide@extra#1{}

```

Hack for unhyphenated line breaking. See \bbl@provide@lsys in the common code.

```

5112 \def\bbl@xenohyph@{%
5113 \bbl@ifset{\bbl@prehc@\language}%
5114 {\ifnum\hyphenchar\font=\defaultthyphenchar
5115 \iffontchar\font\bbl@cl{prehc}\relax
5116 \hyphenchar\font\bbl@cl{prehc}\relax
5117 \else\iffontchar\font"200B
5118 \hyphenchar\font"200B
5119 \else
5120 \bbl@warning
5121 {Neither 0 nor ZERO WIDTH SPACE are available\\%
5122 in the current font, and therefore the hyphen\\%
5123 will be printed. Try changing the fontspec's\\%
5124 'HyphenChar' to another value, but be aware\\%
5125 this setting is not safe (see the manual).\\%
5126 Reported}%
5127 \hyphenchar\font\defaultthyphenchar
5128 \fi\fi
5129 \fi}%

```

```
5130 {\hyphenchar\font\defaultthyphenchar}}
```

10.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```
5131 \ifnum\Xe@alloc@intercharclass<\thr@@
5132 \Xe@alloc@intercharclass\thr@@
5133 \fi
5134 \chardef\bbl@xe@class@default@=\z@
5135 \chardef\bbl@xe@class@cjkideogram@=\@ne
5136 \chardef\bbl@xe@class@cjkleftpunctuation@=\tw@
5137 \chardef\bbl@xe@class@cjkrightpunctuation@=\thr@@
5138 \chardef\bbl@xe@class@boundary@=4095
5139 \chardef\bbl@xe@class@ignore@=4096
```

The machinery is activated with a hook (enabled only if actually used). Here \bbl@tempc is pre-set with \bbl@usingxe@class, defined below. The standard mechanism based on \originalTeX to save, set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```
5140 \AddBabelHook{babel-interchar}{beforeextras}{%
5141 \nameuse{bbl@xechars@\language@}}
5142 \DisableBabelHook{babel-interchar}
5143 \protected\def\bbl@charclass#1{%
5144 \ifnum\count@<\z@
5145 \count@-\count@
5146 \loop
5147 \bbl@exp{%
5148 \\\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5149 \XeTeXcharclass\count@ \bbl@tempc
5150 \ifnum\count@<`#1\relax
5151 \advance\count@ \@ne
5152 \repeat
5153 \else
5154 \babel@savevariable{\XeTeXcharclass`#1}%
5155 \XeTeXcharclass`#1 \bbl@tempc
5156 \fi
5157 \count@`#1\relax}
```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above has internally the form \bbl@usingxe@class\bbl@xe@class@punct@english\bbl@charclass{.} \bbl@charclass{,} (etc.), where \bbl@usingxe@class stores the class to be applied to the subsequent characters. The \ifcat part deals with the alternative way to enter characters as macros (e.g., \}). As a special case, hyphens are stored as \bbl@upto, to deal with ranges.

```
5158 \newcommand\bbl@ifinterchar[1]{%
5159 \let\bbl@tempa\@gobble % Assume to ignore
5160 \edef\bbl@tempb{\zap@space#1 \@empty}%
5161 \ifx\bbl@KVP@interchar\@nnil\else
5162 \bbl@replace\bbl@KVP@interchar{ }{,}%
5163 \bbl@foreach\bbl@tempb{%
5164 \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5165 \ifin@
5166 \let\bbl@tempa\@firstofone
5167 \fi}%
5168 \fi
5169 \bbl@tempa}
5170 \newcommand\IfBabelIntercharT[2]{%
5171 \bbl@carg\bbl@add{bbl@icsave\CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5172 \newcommand\babelcharclass[3]{%
5173 \EnableBabelHook{babel-interchar}%
5174 \bbl@carg\newXeTeXintercharclass{xe@class@#2@#1}%
5175 \def\bbl@tempb##1{%
```



```

5176 \ifx##1\@empty\else
5177 \ifx##1-%
5178 \bbl@upto
5179 \else
5180 \bbl@class{%-
5181 \ifcat\@noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5182 \fi
5183 \expandafter\bbl@tempb
5184 \fi}%
5185 \bbl@ifunset{\bbl@xechars@#1}%
5186 {\toks@{%-
5187 \babel@savevariable\XeTeXinterchartokenstate
5188 \XeTeXinterchartokenstate\@ne
5189 }}%
5190 {\toks@\expandafter\expandafter\expandafter{%-
5191 \csname bbl@xechars@#1\endcsname}}%
5192 \bbl@csarg\edef{\xechars@#1}{%-
5193 \the\toks@
5194 \bbl@usingxeclasse\csname bbl@xeclasse@#2@#1\endcsname
5195 \bbl@tempb#3\@empty}}
5196 \protected\def\bbl@usingxeclasse#1{\count@ \z@ \let\bbl@tempc#1}
5197 \protected\def\bbl@upto{%-
5198 \ifnum\count@>\z@
5199 \advance\count@\@ne
5200 \count@\count@
5201 \else\ifnum\count@=\z@
5202 \bbl@class{-}%
5203 \else
5204 \bbl@error{double-hyphens-class}{\count@}{\count@}%
5205 \fi\fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is a intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5206 \def\bbl@ignoreinterchar{%-
5207 \ifnum\language=\l@nohyphenation
5208 \expandafter\@gobble
5209 \else
5210 \expandafter\@firstofone
5211 \fi}
5212 \newcommand\babelinterchar[5][{}]{%-
5213 \let\bbl@kv@label\@empty
5214 \bbl@forkv{#1}{\bbl@csarg\edef{\kv@##1}{##2}}%
5215 \@namedef{\zap@space bbl@xeinter@\bbl@kv@label @#3@#4@#2 \@empty}%
5216 {\bbl@ignoreinterchar{#5}}%
5217 \bbl@csarg\let{\ic@\bbl@kv@label @#2}\@firstofone
5218 \bbl@expf{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%-
5219 \bbl@expf{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%-
5220 \XeTeXinterchartoks
5221 \@nameuse{\bbl@xeclasse@\bbl@tempa @#2}%
5222 \bbl@ifunset{\bbl@xeclasse@\bbl@tempa @#2}{\bbl@tempa @#2}%
5223 \@nameuse{\bbl@xeclasse@\bbl@tempb @#2}%
5224 \bbl@ifunset{\bbl@xeclasse@\bbl@tempb @#2}{\bbl@tempb @#2}%
5225 = \expandafter{%-
5226 \csname bbl@ic@\bbl@kv@label @#2\expandafter\endcsname
5227 \csname\zap@space bbl@xeinter@\bbl@kv@label
5228 @#3@#4@#2 \@empty\endcsname}}}}
5229 \DeclareRobustCommand\enablelocaleinterchar[1]{%-
5230 \bbl@ifunset{\bbl@ic@#1@language}%
5231 {\bbl@error{unknown-interchar}{#1}{\count@}}%
5232 {\bbl@csarg\let{\ic@#1@language}\@firstofone}}
5233 \DeclareRobustCommand\disablelocaleinterchar[1]{%-
5234 \bbl@ifunset{\bbl@ic@#1@language}%

```

```

5235     {\bbl@error{unknown-interchar-b}{#1}{}}}%
5236     {\bbl@csarg\let{ic@#1@\language\@gobble}}
5237 </xetex>

```

10.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titles, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdfTeX and xetex.

```

5238 <*xetex | texxet>
5239 \providecommand\bbl@provide@intraspace{}
5240 \bbl@trace{Redefinitions for bidi layout}

    Finish here if there in no layout.

5241 \ifx\bbl@opt@layout\@nnil\else % if layout=..
5242   \ifx\bbl@opt@layout\@undefined\else % Plain
5243     \IfBabelLayout{nopars}
5244     {}
5245     {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5246   \fi
5247 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5248 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5249 \ifnum\bbl@bidimode>\z@
5250 \IfBabelLayout{pars}
5251   {\def\@hangfrom#1{%
5252     \setbox\@tempboxa\hbox{#1}}%
5253     \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5254     \noindent\box\@tempboxa}
5255   \def\raggedright{%
5256     \let\\\@centercr
5257     \bbl@startskip\z@skip
5258     \@rightskip\@flushglue
5259     \bbl@endskip\@rightskip
5260     \parindent\z@
5261     \parfillskip\bbl@startskip}
5262   \def\raggedleft{%
5263     \let\\\@centercr
5264     \bbl@startskip\@flushglue
5265     \bbl@endskip\z@skip
5266     \parindent\z@
5267     \parfillskip\bbl@endskip}}
5268   {}
5269 \fi
5270 \IfBabelLayout{lists}
5271   {\bbl@sreplace\list
5272     {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5273     \def\bbl@listleftmargin{%
5274       \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5275     \ifcase\bbl@engine
5276       \def\labelenumii{}\theenumii{}% pdfTeX doesn't reverse ()
5277       \def\p@enumiii{\p@enumii}\theenumii{}%
5278     \fi
5279     \bbl@sreplace\@verbatim
5280     {\leftskip\@totalleftmargin}%
5281     {\bbl@startskip\textwidth
5282       \advance\bbl@startskip-\linewidth}%
5283     \bbl@sreplace\@verbatim
5284     {\rightskip\z@skip}%
5285     {\bbl@endskip\z@skip}}%

```

```

5286 {}
5287 \IfBabelLayout{contents}
5288 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5289 \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5290 {}
5291 \IfBabelLayout{columns}
5292 {\bbl@sreplace\@outpdblcol{\hb@xt@\textwidth}{\bbl@outpbox}%
5293 \def\bbl@outpbox#1{%
5294 \hb@xt@\textwidth{%
5295 \hskip\columnwidth
5296 \hfil
5297 {\normalcolor\vrule \@width\columnseprule}%
5298 \hfil
5299 \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5300 \hskip-\textwidth
5301 \hb@xt@\columnwidth{\box\@outputbox \hss}%
5302 \hskip\columnsep
5303 \hskip\columnwidth}}}%
5304 {}

```

Implicitly reverses sectioning labels in bidi=basic, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5305 \IfBabelLayout{counters*}%
5306 {\bbl@add\bbl@opt@layout{.counters.}%
5307 \AddToHook{shipout/before}{%
5308 \let\bbl@tempa\babelsublr
5309 \let\babelsublr\@firstofone
5310 \let\bbl@save@thepage\thepage
5311 \protected@edef\thepage{\thepage}%
5312 \let\babelsublr\bbl@tempa}%
5313 \AddToHook{shipout/after}{%
5314 \let\thepage\bbl@save@thepage}}{}
5315 \IfBabelLayout{counters}%
5316 {\let\bbl@latinarabic=\@arabic
5317 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5318 \let\bbl@asciroman=\@roman
5319 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5320 \let\bbl@asciiRoman=\@Roman
5321 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5322 \fi % end if layout
5323 </xetex | texxet>

```

10.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```

5324 <*texxet>
5325 \def\bbl@provide@extra#1{%
5326 % == auto-select encoding ==
5327 \ifx\bbl@encoding@select@off\@empty\else
5328 \bbl@ifunset{\bbl@encoding@#1}%
5329 {\def\elt##1{,##1,}%
5330 \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5331 \count@\z@
5332 \bbl@foreach\bbl@tempe{%
5333 \def\bbl@tempd{##1}% Save last declared
5334 \advance\count@\@ne}%
5335 \ifnum\count@>\@ne % (1)
5336 \getlocaleproperty*\bbl@tempe{#1}{identification/encodings}%
5337 \ifx\bbl@tempe\relax \let\bbl@tempe\@empty \fi
5338 \bbl@replace\bbl@tempe{ },}%
5339 \global\bbl@csarg\let{encoding@#1}\@empty
5340 \bbl@xin@{,\bbl@tempd,},{,\bbl@tempe,}%

```

```

5341      \ifin\else % if main encoding included in ini, do nothing
5342      \let\bbl@tempb\relax
5343      \bbl@foreach\bbl@tempa{%
5344      \ifx\bbl@tempb\relax
5345      \bbl@xin@{,##1,}{,\bbl@tempe,}%
5346      \ifin\def\bbl@tempb{##1}\fi
5347      \fi}%
5348      \ifx\bbl@tempb\relax\else
5349      \bbl@exp{%
5350      \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>%
5351      \gdef\<bbl@encoding@#1>{%
5352      \\babel@save\\f@encoding
5353      \\bbl@add\\originalTeX{\\selectfont}%
5354      \\fontencoding{\bbl@tempb}%
5355      \\selectfont}}%
5356      \fi
5357      \fi
5358      \fi}%
5359      }%
5360      \fi}
5361      </texxet>

```

10.5. LuaTeX

The loader for luatex is based solely on `language.dat`, which is read on the fly. The code shouldn't be executed when the format is build, so we check if `\AddBabelHook` is defined. Then comes a modified version of the loader in `hyphen.cfg` (without the `hyphenmins` stuff, which is under the direct control of `babel`).

The names `\l@<language>` are defined and take some value from the beginning because all `ldf` files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the `ldf` finishes). If a language has been loaded, `\bbl@hyphendata@<num>` exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in `language.dat` have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, `lua(e)tex` is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on `babel`, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format `language.dat` is used (under the principle of a single source), instead of `language.def`.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by `babel`) provide a command to allocate them (although there are packages like `ctablestack`). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, `etex.sty` changes the way languages are allocated.

This files is read at three places: (1) when `plain.def`, `babel.sty` starts, to read the list of available languages from `language.dat` (for the base option); (2) at `hyphen.cfg`, to modify some macros; (3) in the middle of `plain.def` and `babel.sty`, by `babel.def`, with the commands and other definitions for luatex (e.g., `\babelpatterns`).

```

5362 <*\luatex>
5363 \directlua{ Babel = Babel or {} } % DL2
5364 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5365 \bbl@trace{Read language.dat}
5366 \ifx\bbl@readstream\undefined
5367   \csname newread\endcsname\bbl@readstream
5368 \fi

```

```

5369 \begingroup
5370 \toks@{}
5371 \count@ \z@ % 0=start, 1=0th, 2=normal
5372 \def\bbl@process@line#1#2 #3 #4 {%
5373   \ifx=#1%
5374     \bbl@process@synonym{#2}%
5375   \else
5376     \bbl@process@language{#1#2}{#3}{#4}%
5377   \fi
5378   \ignorespaces}
5379 \def\bbl@manylang{%
5380   \ifnum\bbl@last>\@ne
5381     \bbl@info{Non-standard hyphenation setup}%
5382   \fi
5383   \let\bbl@manylang\relax}
5384 \def\bbl@process@language#1#2#3{%
5385   \ifcase\count@
5386     \@ifundefined{zth@#1}{\count@\tw@}{\count@\@ne}%
5387   \or
5388     \count@\tw@
5389   \fi
5390   \ifnum\count@=\tw@
5391     \expandafter\addlanguage\csname l@#1\endcsname
5392     \language\allocationnumber
5393     \chardef\bbl@last\allocationnumber
5394     \bbl@manylang
5395     \let\bbl@elt\relax
5396     \xdef\bbl@languages{%
5397       \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5398   \fi
5399   \the\toks@
5400   \toks@{}}
5401 \def\bbl@process@synonym@aux#1#2{%
5402   \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5403   \let\bbl@elt\relax
5404   \xdef\bbl@languages{%
5405     \bbl@languages\bbl@elt{#1}{#2}{}}}%
5406 \def\bbl@process@synonym#1{%
5407   \ifcase\count@
5408     \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5409   \or
5410     \@ifundefined{zth@#1}{\bbl@process@synonym@aux{#1}{0}}{}%
5411   \else
5412     \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5413   \fi}
5414 \ifx\bbl@languages\@undefined % Just a (sensible?) guess
5415   \chardef\l@english\z@
5416   \chardef\l@USenglish\z@
5417   \chardef\bbl@last\z@
5418   \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}
5419   \gdef\bbl@languages{%
5420     \bbl@elt{english}{0}{hyphen.tex}}%
5421     \bbl@elt{USenglish}{0}{}%
5422 \else
5423   \global\let\bbl@languages@format\bbl@languages
5424   \def\bbl@elt#1#2#3#4{% Remove all except language 0
5425     \ifnum#2>\z@ \else
5426       \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5427     \fi}%
5428   \xdef\bbl@languages{\bbl@languages}%
5429 \fi
5430 \def\bbl@elt#1#2#3#4{\@namedef{zth@#1}} % Define flags
5431 \bbl@languages

```

```

5432 \openin\bbl@readstream=language.dat
5433 \ifeof\bbl@readstream
5434   \bbl@warning{I couldn't find language.dat. No additional\\%
5435               patterns loaded. Reported}%
5436 \else
5437   \loop
5438     \endlinechar\m@ne
5439     \read\bbl@readstream to \bbl@line
5440     \endlinechar`\^^M
5441     \if T\ifeof\bbl@readstream F\fi T\relax
5442     \ifx\bbl@line\@empty\else
5443       \edef\bbl@line{\bbl@line\space\space\space}%
5444       \expandafter\bbl@process@line\bbl@line\relax
5445     \fi
5446   \repeat
5447 \fi
5448 \closein\bbl@readstream
5449 \endgroup
5450 \bbl@trace{Macros for reading patterns files}
5451 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5452 \ifx\babelcatcodetablenum\@undefined
5453   \ifx\newcatcodetable\@undefined
5454     \def\babelcatcodetablenum{5211}
5455     \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5456   \else
5457     \newcatcodetable\babelcatcodetablenum
5458     \newcatcodetable\bbl@pattcodes
5459   \fi
5460 \else
5461   \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5462 \fi
5463 \def\bbl@luapatterns#1#2{%
5464   \bbl@get@enc#1:.\@@@
5465   \setbox\z@\hbox\bgroup
5466   \begingroup
5467     \savecatcodetable\babelcatcodetablenum\relax
5468     \initcatcodetable\bbl@pattcodes\relax
5469     \catcodetable\bbl@pattcodes\relax
5470     \catcode`\#=6 \catcode`\$=3 \catcode`\&=4 \catcode`\^=7
5471     \catcode`\_ =8 \catcode`\{=1 \catcode`\}=2 \catcode`\~ =13
5472     \catcode`\@=11 \catcode`\^^I=10 \catcode`\^^J=12
5473     \catcode`\<=12 \catcode`\>=12 \catcode`\*=12 \catcode`\.=12
5474     \catcode`\-=12 \catcode`\/=12 \catcode`\[=12 \catcode`\]=12
5475     \catcode`\`=12 \catcode`\'=12 \catcode`\`=12
5476     \input #1\relax
5477     \catcodetable\babelcatcodetablenum\relax
5478   \endgroup
5479   \def\bbl@tempa{#2}%
5480   \ifx\bbl@tempa\@empty\else
5481     \input #2\relax
5482   \fi
5483 \egroup}%
5484 \def\bbl@patterns@lua#1{%
5485   \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5486     \csname l@#1\endcsname
5487     \edef\bbl@tempa{#1}%
5488   \else
5489     \csname l@#1:\f@encoding\endcsname
5490     \edef\bbl@tempa{#1:\f@encoding}%
5491   \fi\relax
5492   \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5493   \@ifundefined{bbl@hyphendata@the\language}%
5494     {\def\bbl@elt##1##2##3##4{%

```

```

5495 \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5496 \def\bbl@tempb{##3}%
5497 \ifx\bbl@tempb\empty\else % if not a synonymous
5498 \def\bbl@tempc{{##3}{##4}}%
5499 \fi
5500 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5501 \fi}%
5502 \bbl@languages
5503 \@ifundefined{bbl@hyphendata@the\language}%
5504 {\bbl@info{No hyphenation patterns were set for\%
5505 language '\bbl@tempa'. Reported}}%
5506 {\expandafter\expandafter\expandafter\bbl@luapatterns
5507 \csname bbl@hyphendata@the\language\endcsname}}}%
5508 \endinput\fi

```

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

```

5509 \ifx\DisableBabelHook\@undefined
5510 \AddBabelHook{luatex}{everylanguage}{%
5511 \def\process@language##1##2##3{%
5512 \def\process@line####1####2 ####3 ####4 {}}
5513 \AddBabelHook{luatex}{loadpatterns}{%
5514 \input #1\relax
5515 \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5516 {{#1}}}%
5517 \AddBabelHook{luatex}{loadexceptions}{%
5518 \input #1\relax
5519 \def\bbl@tempb##1##2{{##1}{##2}}%
5520 \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5521 {\expandafter\expandafter\expandafter\bbl@tempb
5522 \csname bbl@hyphendata@the\language\endcsname}}
5523 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5524 \begingroup
5525 \catcode`\%=12
5526 \catcode`\'=12
5527 \catcode`\%=12
5528 \catcode`\:=12
5529 \directlua{
5530 Babel.locale_props = Babel.locale_props or {}
5531 function Babel.lua_error(e, a)
5532 tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5533 e .. '}' .. (a or '') .. '}{'}])
5534 end
5535
5536 function Babel.bytes(line)
5537 return line:gsub(".",
5538 function (chr) return unicode.utf8.char(string.byte(chr)) end)
5539 end
5540
5541 function Babel.priority_in_callback(name,description)
5542 for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5543 if v == description then return i end
5544 end
5545 return false
5546 end
5547
5548 function Babel.begin_process_input()
5549 if luatexbase and luatexbase.add_to_callback then
5550 luatexbase.add_to_callback('process_input_buffer',
5551 Babel.bytes, 'Babel.bytes')
5552 else
5553 Babel.callback = callback.find('process_input_buffer')

```

```

5554     callback.register('process_input_buffer',Babel.bytes)
5555 end
5556 end
5557 function Babel.end_process_input ()
5558     if luatexbase and luatexbase.remove_from_callback then
5559         luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5560     else
5561         callback.register('process_input_buffer',Babel.callback)
5562     end
5563 end
5564
5565 function Babel.str_to_nodes(fn, matches, base)
5566     local n, head, last
5567     if fn == nil then return nil end
5568     for s in string.utfvalues(fn(matches)) do
5569         if base.id == 7 then
5570             base = base.replace
5571         end
5572         n = node.copy(base)
5573         n.char = s
5574         if not head then
5575             head = n
5576         else
5577             last.next = n
5578         end
5579         last = n
5580     end
5581     return head
5582 end
5583
5584 Babel.linebreaking = Babel.linebreaking or {}
5585 Babel.linebreaking.before = {}
5586 Babel.linebreaking.after = {}
5587 Babel.locale = {}
5588 function Babel.linebreaking.add_before(func, pos)
5589     tex.print([[noexpand\csname bbl@lua hyphenate\endcsname]])
5590     if pos == nil then
5591         table.insert(Babel.linebreaking.before, func)
5592     else
5593         table.insert(Babel.linebreaking.before, pos, func)
5594     end
5595 end
5596 function Babel.linebreaking.add_after(func)
5597     tex.print([[noexpand\csname bbl@lua hyphenate\endcsname]])
5598     table.insert(Babel.linebreaking.after, func)
5599 end
5600
5601 function Babel.addpatterns(pp, lg)
5602     local lg = lang.new(lg)
5603     local pats = lang.patterns(lg) or ''
5604     lang.clear_patterns(lg)
5605     for p in pp:gmatch('[^%s]+') do
5606         ss = ''
5607         for i in string.utfcharacters(p:gsub('%d', '')) do
5608             ss = ss .. '%d?' .. i
5609         end
5610         ss = ss:gsub('^%d%?%.','%.') .. '%d?'
5611         ss = ss:gsub('%.%d%?$', '%%.')
5612         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5613         if n == 0 then
5614             tex.sprint(
5615                 [[\string\csname\space bbl@info\endcsname{New pattern: }]]
5616                 .. p .. [[]])

```



```

5617     pats = pats .. ' ' .. p
5618   else
5619     tex.sprint(
5620       [[\string\csname\space bbl@info\endcsname{Renew pattern: }]
5621       .. p .. [[]]])
5622   end
5623 end
5624 lang.patterns(lg, pats)
5625 end
5626
5627 Babel.characters = Babel.characters or {}
5628 Babel.ranges = Babel.ranges or {}
5629 function Babel.hlist_has_bidi(head)
5630   local has_bidi = false
5631   local ranges = Babel.ranges
5632   for item in node.traverse(head) do
5633     if item.id == node.id'glyph' then
5634       local itemchar = item.char
5635       local chardata = Babel.characters[itemchar]
5636       local dir = chardata and chardata.d or nil
5637       if not dir then
5638         for nn, et in ipairs(ranges) do
5639           if itemchar < et[1] then
5640             break
5641           elseif itemchar <= et[2] then
5642             dir = et[3]
5643             break
5644           end
5645         end
5646       end
5647       if dir and (dir == 'al' or dir == 'r') then
5648         has_bidi = true
5649       end
5650     end
5651   end
5652   return has_bidi
5653 end
5654 function Babel.set_chranges_b (script, chrng)
5655   if chrng == '' then return end
5656   texio.write('Replacing ' .. script .. ' script ranges')
5657   Babel.script_blocks[script] = {}
5658   for s, e in string.gmatch(chrng..' ', '(.)%.%.(.)%s') do
5659     table.insert(
5660       Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5661   end
5662 end
5663
5664 function Babel.discard_sublr(str)
5665   if str:find( [[\string\indexentry]] ) and
5666     str:find( [[\string\babelsublr]] ) then
5667     str = str:gsub( [[\string\babelsublr%s*(%b{})]],
5668       function(m) return m:sub(2,-2) end )
5669   end
5670   return str
5671 end
5672 }
5673 \endgroup
5674 \ifx\newattribute\undefined\else % Test for plain
5675   \newattribute\bbl@attr@locale % DL4
5676   \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5677   \AddBabelHook{luatex}{beforeextras}{%
5678     \setattribute\bbl@attr@locale\localeid}
5679 \fi

```

```

5680 %
5681 \def\BabelStringsDefault{unicode}
5682 \let\luabbbl@stop\relax
5683 \AddBabelHook{luatex}{encodedcommands}{%
5684   \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5685   \ifx\bbl@tempa\bbl@tempb\else
5686     \directlua{Babel.begin_process_input()}%
5687     \def\luabbbl@stop{%
5688       \directlua{Babel.end_process_input()}}%
5689   \fi}%
5690 \AddBabelHook{luatex}{stopcommands}{%
5691   \luabbbl@stop
5692   \let\luabbbl@stop\relax}
5693 %
5694 \AddBabelHook{luatex}{patterns}{%
5695   \@ifundefined{bbl@hyphendata@the\language}%
5696   {\def\bbl@elt##1##2##3##4{%
5697     \ifnum##2=\csname l@##2\endcsname % #2=spanish, dutch:OT1...
5698     \def\bbl@tempb{##3}%
5699     \ifx\bbl@tempb\@empty\else % if not a synonymous
5700       \def\bbl@tempc{##3}{##4}%
5701       \fi
5702       \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5703     \fi}%
5704   \bbl@languages
5705   \@ifundefined{bbl@hyphendata@the\language}%
5706   {\bbl@info{No hyphenation patterns were set for\%
5707     language '#2'. Reported}}%
5708   {\expandafter\expandafter\expandafter\bbl@luapatterns
5709     \csname bbl@hyphendata@the\language\endcsname}}}%
5710 \@ifundefined{bbl@patterns@}{}%
5711 \begingroup
5712   \bbl@xin@{,\number\language,}{,\bbl@pttnlist}%
5713   \ifin@ \else
5714     \ifx\bbl@patterns@\@empty \else
5715       \directlua{ Babel.addpatterns(
5716         [[\bbl@patterns@]], \number\language) }%
5717       \fi
5718       \@ifundefined{bbl@patterns@#1}%
5719       \@empty
5720       {\directlua{ Babel.addpatterns(
5721         [[\space\csname bbl@patterns@#1\endcsname]],
5722         \number\language) }}%
5723       \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5724     \fi
5725   \endgroup}%
5726 \bbl@exp{%
5727   \bbl@ifunset{bbl@prehc@languagename}{}%
5728   {\bbl@ifblank{\bbl@cs{prehc@languagename}}{}}%
5729   {\prehyphenchar=\bbl@cl{prehc}\relax}}}%

```

\babelpatterns This macro adds patterns. Two macros are used to store them: \bbl@patterns@ for the global ones and \bbl@patterns@<language> for language ones. We make sure there is a space between words when multiple commands are used.

```

5730 \@onlypreamble\babelpatterns
5731 \AtEndOfPackage{%
5732   \newcommand\babelpatterns[2][\@empty]{%
5733     \ifx\bbl@patterns@\relax
5734       \let\bbl@patterns@\@empty
5735     \fi
5736     \ifx\bbl@pttnlist@\@empty \else
5737       \bbl@warning{%
5738         You must not intermingle \string\selectlanguage\space and\%

```

```

5739     \string\babelpatterns\space or some patterns will not\\%
5740     be taken into account. Reported}%
5741 \fi
5742 \ifx\@empty#1%
5743     \protected@edef\bbbl@patterns@\bbbl@patterns@\space#2}%
5744 \else
5745     \edef\bbbl@tempb{\zap@space#1 \@empty}%
5746     \bbbl@for\bbbl@tempa\bbbl@tempb{%
5747         \bbbl@fixname\bbbl@tempa
5748         \bbbl@iflanguage\bbbl@tempa{%
5749             \bbbl@csarg\protected@edef{patterns@\bbbl@tempa}{%
5750                 \@ifundefined{bbbl@patterns@\bbbl@tempa}%
5751                 \@empty
5752                 {\csname bbl@patterns@\bbbl@tempa\endcsname\space}%
5753                 #2}}}%
5754 \fi}}

```

10.6. Southeast Asian scripts

First, some general code for line breaking, used by \babelposthyphenation.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5755 \def\bbbl@intraspace#1 #2 #3\@@{%
5756 \directlua{
5757     Babel.intraspaces = Babel.intraspaces or {}
5758     Babel.intraspaces['\csname bbl@sbcpr@languagename\endcsname'] = %
5759     {b = #1, p = #2, m = #3}
5760     Babel.locale_props[\the\localeid].intraspace = %
5761     {b = #1, p = #2, m = #3}
5762 }}
5763 \def\bbbl@intrapenalty#1\@@{%
5764 \directlua{
5765     Babel.intrapenalties = Babel.intrapenalties or {}
5766     Babel.intrapenalties['\csname bbl@sbcpr@languagename\endcsname'] = #1
5767     Babel.locale_props[\the\localeid].intrapenalty = #1
5768 }}
5769 \beginingroup
5770 \catcode`\%=12
5771 \catcode`\&=14
5772 \catcode`\'=12
5773 \catcode`\-=12
5774 \gdef\bbbl@seaintraspace{&
5775 \let\bbbl@seaintraspace\relax
5776 \directlua{
5777     Babel.sea_enabled = true
5778     Babel.sea_ranges = Babel.sea_ranges or {}
5779     function Babel.set_chranges (script, chrng)
5780         local c = 0
5781         for s, e in string.gmatch(chrng..' ', '(.-%.%.(.-%)s') do
5782             Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5783             c = c + 1
5784         end
5785     end
5786     function Babel.sea_disc_to_space (head)
5787         local sea_ranges = Babel.sea_ranges
5788         local last_char = nil
5789         local quad = 655360 & 10 pt = 655360 = 10 * 65536
5790         for item in node.traverse(head) do
5791             local i = item.id
5792             if i == node.id'glyph' then
5793                 last_char = item
5794             elseif i == 7 and item.subtype == 3 and last_char

```

```

5795         and last_char.char > 0x0C99 then
5796     quad = font.getfont(last_char.font).size
5797     for lg, rg in pairs(sea_ranges) do
5798         if last_char.char > rg[1] and last_char.char < rg[2] then
5799             lg = lg:sub(1, 4)  &% Remove trailing number of, e.g., Cyril
5800             local intraspace = Babel.intraspaces[lg]
5801             local intrapenalty = Babel.intrapenalties[lg]
5802             local n
5803             if intrapenalty ~= 0 then
5804                 n = node.new(14, 0)    &% penalty
5805                 n.penalty = intrapenalty
5806                 node.insert_before(head, item, n)
5807             end
5808             n = node.new(12, 13)      &% (glue, spaceskip)
5809             node.setglue(n, intraspace.b * quad,
5810                           intraspace.p * quad,
5811                           intraspace.m * quad)
5812             node.insert_before(head, item, n)
5813             node.remove(head, item)
5814         end
5815     end
5816 end
5817 end
5818 end
5819 }&
5820 \bbl@luahyphenate}

```

10.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5821 \catcode`\%=14
5822 \gdef\bbl@cjkintraspaces{%
5823     \let\bbl@cjkintraspaces\relax
5824     \directlua{
5825         require('babel-data-cjk.lua')
5826         Babel.cjk_enabled = true
5827         function Babel.cjk_linebreak(head)
5828             local GLYPH = node.id'glyph'
5829             local last_char = nil
5830             local quad = 655360      % 10 pt = 655360 = 10 * 65536
5831             local last_class = nil
5832             local last_lang = nil
5833             for item in node.traverse(head) do
5834                 if item.id == GLYPH then
5835                     local lang = item.lang
5836                     local LOCALE = node.get_attribute(item,
5837                                                         Babel.attr_locale)
5838                     local props = Babel.locale_props[LOCALE] or {}
5839                     local class = Babel.cjk_class[item.char].c
5840                     if props.cjk_quotes and props.cjk_quotes[item.char] then
5841                         class = props.cjk_quotes[item.char]
5842                     end
5843                     if class == 'cp' then class = 'cl' % }) as CL
5844                     elseif class == 'id' then class = 'I'
5845                     elseif class == 'cj' then class = 'I' % loose
5846                     end
5847                     local br = 0
5848                     if class and last_class and Babel.cjk_breaks[last_class][class] then

```

```

5849         br = Babel.cjk_breaks[last_class][class]
5850     end
5851     if br == 1 and props.linebreak == 'c' and
5852         lang ~= \the\l@nohyphenation\space and
5853         last_lang ~= \the\l@nohyphenation then
5854         local intrapenalty = props.intrapenalty
5855         if intrapenalty ~= 0 then
5856             local n = node.new(14, 0)      % penalty
5857             n.penalty = intrapenalty
5858             node.insert_before(head, item, n)
5859         end
5860         local intraspace = props.intraspace
5861         local n = node.new(12, 13)      % (glue, spaceskip)
5862         node.setglue(n, intraspace.b * quad,
5863             intraspace.p * quad,
5864             intraspace.m * quad)
5865         node.insert_before(head, item, n)
5866     end
5867     if font.getfont(item.font) then
5868         quad = font.getfont(item.font).size
5869     end
5870     last_class = class
5871     last_lang = lang
5872     else % if penalty, glue or anything else
5873         last_class = nil
5874     end
5875 end
5876 lang.hyphenate(head)
5877 end
5878 }%
5879 \bbl@luahyphenate}
5880 \gdef\bbl@luahyphenate{%
5881 \let\bbl@luahyphenate\relax
5882 \directlua{
5883     luatexbase.add_to_callback('hyphenate',
5884     function (head, tail)
5885         if Babel.linebreaking.before then
5886             for k, func in ipairs(Babel.linebreaking.before) do
5887                 func(head)
5888             end
5889         end
5890         lang.hyphenate(head)
5891         if Babel.cjk_enabled then
5892             Babel.cjk_linebreak(head)
5893         end
5894         if Babel.linebreaking.after then
5895             for k, func in ipairs(Babel.linebreaking.after) do
5896                 func(head)
5897             end
5898         end
5899         if Babel.set_hboxed then
5900             Babel.set_hboxed(head)
5901         end
5902         if Babel.sea_enabled then
5903             Babel.sea_disc_to_space(head)
5904         end
5905     end,
5906     'Babel.hyphenate')
5907 }}
5908 \endgroup
5909 %
5910 \def\bbl@provide@intraspace{%
5911     \bbl@ifunset\bbl@intsp@{language}{}}%

```

```

5912 {\expandafter\ifx\csname bbl@intsp@\language\endcsname\empty\else
5913 \bbl@xin@{/c}{/\bbl@cl{\lnbrk}}}%
5914 \ifin@ % cjk
5915 \bbl@cjk intraspace
5916 \directlua{
5917     Babel.locale_props = Babel.locale_props or {}
5918     Babel.locale_props[\the\localeid].linebreak = 'c'
5919 }%
5920 \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@%
5921 \ifx\bbl@KVP@intrapenalty\@nnil
5922 \bbl@intrapenalty0\@
5923 \fi
5924 \else % sea
5925 \bbl@seaintraspace
5926 \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@%
5927 \directlua{
5928     Babel.sea_ranges = Babel.sea_ranges or {}
5929     Babel.set_chranges('\bbl@cl{sbcpr}',
5930                       '\bbl@cl{chrng}')
5931 }%
5932 \ifx\bbl@KVP@intrapenalty\@nnil
5933 \bbl@intrapenalty0\@
5934 \fi
5935 \fi
5936 \fi
5937 \ifx\bbl@KVP@intrapenalty\@nnil\else
5938 \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@
5939 \fi}}

```

10.8. Arabic justification

WIP. `\bbl@arabicjust` is executed with both elongated and kashida. This must be fine tuned. The attribute `kashida` is set by transforms with `kashida`.

```

5940 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
5941 \def\bblar@chars{%
5942     0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
5943     0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
5944     0640,0641,0642,0643,0644,0645,0646,0647,0649}
5945 \def\bblar@elongated{%
5946     0626,0628,062A,062B,0633,0634,0635,0636,063B,%
5947     063C,063D,063E,063F,0641,0642,0643,0644,0646,%
5948     0649,064A}
5949 \begingroup
5950 \catcode`\_ =11 \catcode`\:=11
5951 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
5952 \endgroup
5953 \gdef\bbl@arabicjust{%
5954     \let\bbl@arabicjust\relax
5955     \newattribute\bblar@kashida
5956     \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
5957     \bblar@kashida=\z@
5958     \bbl@patchfont{\bbl@parsejalt}}%
5959 \directlua{
5960     Babel.arabic.elong_map = Babel.arabic.elong_map or {}
5961     Babel.arabic.elong_map[\the\localeid] = {}
5962     luatexbase.add_to_callback('post_linebreak_filter',
5963                               Babel.arabic.justify, 'Babel.arabic.justify')
5964     luatexbase.add_to_callback('hpack_filter',
5965                               Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
5966 }%

```

Save both node lists to make replacement.

```

5967 \def\bblar@fetchjalt#1#2#3#4{%

```

```

5968 \bbl@exp{\bbl@foreach{#1}}{%
5969 \bbl@ifunset{bblar@JE@##1}%
5970 {\setbox\z@\hbox{\textdir TRT ^^^200d\char"##1#2}}%
5971 {\setbox\z@\hbox{\textdir TRT ^^^200d\char"\@nameuse{bblar@JE@##1}#2}}%
5972 \directlua{%
5973 local last = nil
5974 for item in node.traverse(tex.box[0].head) do
5975 if item.id == node.id'glyph' and item.char > 0x600 and
5976 not (item.char == 0x200D) then
5977 last = item
5978 end
5979 end
5980 Babel.arabic.#3['##1#4'] = last.char
5981 }}}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswb?). What about kaf? And diacritic positioning?

```

5982 \gdef\bbl@parsejalt{%
5983 \ifx\addfontfeature\undefined\else
5984 \bbl@xin{/e}{/\bbl@cl{lnbrk}}%
5985 \ifin@
5986 \directlua{%
5987 if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
5988 Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
5989 tex.print([[string\cswb\space bbl@parsejalti\endcswb]])
5990 end
5991 }%
5992 \fi
5993 \fi}
5994 \gdef\bbl@parsejalti{%
5995 \begingroup
5996 \let\bbl@parsejalt\relax % To avoid infinite loop
5997 \edef\bbl@tempb{\fontid\font}%
5998 \bblar@nofswarn
5999 \@nameuse{tracinglostchars}\z@
6000 \bblar@fetchjalt\bblar@elongated{}{from}{}%
6001 \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6002 \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6003 \addfontfeature{RawFeature+=jalt}%
6004 % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf
6005 \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6006 \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6007 \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6008 \directlua{%
6009 for k, v in pairs(Babel.arabic.from) do
6010 if Babel.arabic.dest[k] and
6011 not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6012 Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6013 [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6014 end
6015 end
6016 }%
6017 \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6018 \begingroup
6019 \catcode`#=11
6020 \catcode`~=11
6021 \directlua{
6022
6023 Babel.arabic = Babel.arabic or {}
6024 Babel.arabic.from = {}
6025 Babel.arabic.dest = {}
6026 Babel.arabic.justify_factor = 0.95

```

```

6027 Babel.arabic.justify_enabled = true
6028 Babel.arabic.kashida_limit = -1
6029
6030 function Babel.arabic.justify(head)
6031   if not Babel.arabic.justify_enabled then return head end
6032   for line in node.traverse_id(node.id'hlist', head) do
6033     Babel.arabic.justify_hlist(head, line)
6034   end
6035   % In case the very first item is a line (eg, in \vbox):
6036   while head.prev do head = head.prev end
6037   return head
6038 end
6039
6040 function Babel.arabic.justify_hbox(head, gc, size, pack)
6041   local has_inf = false
6042   if Babel.arabic.justify_enabled and pack == 'exactly' then
6043     for n in node.traverse_id(12, head) do
6044       if n.stretch_order > 0 then has_inf = true end
6045     end
6046     if not has_inf then
6047       Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6048     end
6049   end
6050   return head
6051 end
6052
6053 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6054   local d, new
6055   local k_list, k_item, pos_inline
6056   local width, width_new, full, k_curr, wt_pos, goal, shift
6057   local subst_done = false
6058   local elong_map = Babel.arabic.elong_map
6059   local cnt
6060   local last_line
6061   local GLYPH = node.id'glyph'
6062   local KASHIDA = Babel.attr_kashida
6063   local LOCALE = Babel.attr_locale
6064
6065   if line == nil then
6066     line = {}
6067     line.glue_sign = 1
6068     line.glue_order = 0
6069     line.head = head
6070     line.shift = 0
6071     line.width = size
6072   end
6073
6074   % Exclude last line.
6075   if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6076     elongs = {} % Stores elongated candidates of each line
6077     k_list = {} % And all letters with kashida
6078     pos_inline = 0 % Not yet used
6079
6080     for n in node.traverse_id(GLYPH, line.head) do
6081       pos_inline = pos_inline + 1 % To find where it is. Not used.
6082
6083       % Elongated glyphs
6084       if elong_map then
6085         local locale = node.get_attribute(n, LOCALE)
6086         if elong_map[locale] and elong_map[locale][n.font] and
6087           elong_map[locale][n.font][n.char] then
6088           table.insert(elongs, {node = n, locale = locale} )
6089           node.set_attribute(n.prev, KASHIDA, 0)

```



```

6090         end
6091     end
6092
6093     % Tatwil. First create a list of nodes marked with kashida. The
6094     % rest of nodes can be ignored. The list of used weights is build
6095     % when transforms with the key kashida= are declared.
6096     if Babel.kashida_wts then
6097         local k_wt = node.get_attribute(n, KASHIDA)
6098         if k_wt > 0 then % todo. parameter for multi inserts
6099             table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6100         end
6101     end
6102
6103 end % of node.traverse_id
6104
6105 if #elongs == 0 and #k_list == 0 then goto next_line end
6106 full = line.width
6107 shift = line.shift
6108 goal = full * Babel.arabic.justify_factor % A bit crude
6109 width = node.dimensions(line.head) % The 'natural' width
6110
6111 % == Elongated ==
6112 % Original idea taken from 'chickenize'
6113 while (#elongs > 0 and width < goal) do
6114     subst_done = true
6115     local x = #elongs
6116     local curr = elongs[x].node
6117     local oldchar = curr.char
6118     curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6119     width = node.dimensions(line.head) % Check if the line is too wide
6120     % Substitute back if the line would be too wide and break:
6121     if width > goal then
6122         curr.char = oldchar
6123         break
6124     end
6125     % If continue, pop the just substituted node from the list:
6126     table.remove(elongs, x)
6127 end
6128
6129 % == Tatwil ==
6130 % Traverse the kashida node list so many times as required, until
6131 % the line is filled. The first pass adds a tatweel after each
6132 % node with kashida in the line, the second pass adds another one,
6133 % and so on. In each pass, add first the kashida with the highest
6134 % weight, then with lower weight and so on.
6135 if #k_list == 0 then goto next_line end
6136
6137 width = node.dimensions(line.head) % The 'natural' width
6138 k_curr = #k_list % Traverse backwards, from the end
6139 wt_pos = 1
6140
6141 while width < goal do
6142     subst_done = true
6143     k_item = k_list[k_curr].node
6144     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6145         d = node.copy(k_item)
6146         d.char = 0x0640
6147         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6148         d.xoffset = 0
6149         line.head, new = node.insert_after(line.head, k_item, d)
6150         width_new = node.dimensions(line.head)
6151         if width > goal or width == width_new then
6152             node.remove(line.head, new) % Better compute before

```

```

6153         break
6154     end
6155     if Babel.fix_diacr then
6156         Babel.fix_diacr(k_item.next)
6157     end
6158     width = width_new
6159 end
6160 if k_curr == 1 then
6161     k_curr = #k_list
6162     wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6163 else
6164     k_curr = k_curr - 1
6165 end
6166 end
6167
6168 % Limit the number of tatweel by removing them. Not very efficient,
6169 % but it does the job in a quite predictable way.
6170 if Babel.arabic.kashida_limit > -1 then
6171     cnt = 0
6172     for n in node.traverse_id(GLYPH, line.head) do
6173         if n.char == 0x0640 then
6174             cnt = cnt + 1
6175             if cnt > Babel.arabic.kashida_limit then
6176                 node.remove(line.head, n)
6177             end
6178         else
6179             cnt = 0
6180         end
6181     end
6182 end
6183
6184 ::next_line::
6185
6186 % Must take into account marks and ins, see luatex manual.
6187 % Have to be executed only if there are changes. Investigate
6188 % what's going on exactly.
6189 if subst_done and not gc then
6190     d = node.hpack(line.head, full, 'exactly')
6191     d.shift = shift
6192     node.insert_before(head, line, d)
6193     node.remove(head, line)
6194 end
6195 end % if process line
6196 end
6197 }
6198 \endgroup
6199 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6200 \def\bbl@scr@node@list{%
6201   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6202   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6203 \ifnum\bbl@bidimode=102 % bidi-r
6204   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6205 \fi
6206 \def\bbl@set@renderer{%
6207   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%

```

```

6208 \ifin@
6209 \let\bbl@unset@renderer\relax
6210 \else
6211 \bbl@exp{%
6212 \def\\bbl@unset@renderer{%
6213 \def\<g__fontspec_default_fontopts_clist>{%
6214 \[g__fontspec_default_fontopts_clist]}}%
6215 \def\<g__fontspec_default_fontopts_clist>{%
6216 Renderer=Harfbuzz,\[g__fontspec_default_fontopts_clist]}}%
6217 \fi}
6218 <@Font selection@>

```

10.10 Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6219 \directlua{% DL6
6220 Babel.script_blocks = {
6221 ['dflt'] = {},
6222 ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6223             {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6224 ['Armn'] = {{0x0530, 0x058F}},
6225 ['Beng'] = {{0x0980, 0x09FF}},
6226 ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6227 ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6228 ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6229            {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6230 ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6231 ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6232            {0xAB00, 0xAB2F}},
6233 ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6234 % Don't follow strictly Unicode, which places some Coptic letters in
6235 % the 'Greek and Coptic' block
6236 ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6237 ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6238            {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6239            {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6240            {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6241            {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6242            {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6243 ['Hebr'] = {{0x0590, 0x05FF},
6244            {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6245 ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6246            {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6247 ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6248 ['Knda'] = {{0x0C80, 0x0CFF}},
6249 ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6250            {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6251            {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6252 ['Lao'] = {{0x0E80, 0x0EFF}},
6253 ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6254            {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6255            {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6256 ['Mahj'] = {{0x11150, 0x1117F}},

```

```

6257 ['Mlym'] = {{0x0D00, 0x0D7F}},
6258 ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6259 ['Orya'] = {{0x0B00, 0x0B7F}},
6260 ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6261 ['Sycr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6262 ['Taml'] = {{0x0B80, 0x0BFF}},
6263 ['Telu'] = {{0x0C00, 0x0C7F}},
6264 ['Tfng'] = {{0x2D30, 0x2D7F}},
6265 ['Thai'] = {{0x0E00, 0x0E7F}},
6266 ['Tibt'] = {{0x0F00, 0x0FFF}},
6267 ['Vaii'] = {{0xA500, 0xA63F}},
6268 ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6269 }
6270
6271 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6272 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6273 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6274
6275 function Babel.locale_map(head)
6276   if not Babel.locale_mapped then return head end
6277
6278   local LOCALE = Babel.attr_locale
6279   local GLYPH = node.id('glyph')
6280   local inmath = false
6281   local toloc_save
6282   for item in node.traverse(head) do
6283     local toloc
6284     if not inmath and item.id == GLYPH then
6285       % Optimization: build a table with the chars found
6286       if Babel.chr_to_loc[item.char] then
6287         toloc = Babel.chr_to_loc[item.char]
6288       else
6289         for lc, maps in pairs(Babel.loc_to_scr) do
6290           for _, rg in pairs(maps) do
6291             if item.char >= rg[1] and item.char <= rg[2] then
6292               Babel.chr_to_loc[item.char] = lc
6293               toloc = lc
6294               break
6295             end
6296           end
6297         end
6298         % Treat composite chars in a different fashion, because they
6299         % 'inherit' the previous locale.
6300         if (item.char >= 0x0300 and item.char <= 0x036F) or
6301            (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6302            (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6303           Babel.chr_to_loc[item.char] = -2000
6304           toloc = -2000
6305         end
6306         if not toloc then
6307           Babel.chr_to_loc[item.char] = -1000
6308         end
6309       end
6310       if toloc == -2000 then
6311         toloc = toloc_save
6312       elseif toloc == -1000 then
6313         toloc = nil
6314       end
6315       if toloc and Babel.locale_props[toloc] and
6316          Babel.locale_props[toloc].letters and
6317          tex.getcatcode(item.char) \string~= 11 then
6318         toloc = nil
6319       end

```

```

6320     if toloc and Babel.locale_props[toloc].script
6321         and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6322         and Babel.locale_props[toloc].script ==
6323             Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6324         toloc = nil
6325     end
6326     if toloc then
6327         if Babel.locale_props[toloc].lg then
6328             item.lang = Babel.locale_props[toloc].lg
6329             node.set_attribute(item, LOCALE, toloc)
6330         end
6331         if Babel.locale_props[toloc]['/'..item.font] then
6332             item.font = Babel.locale_props[toloc]['/'..item.font]
6333         end
6334     end
6335     toloc_save = toloc
6336 elseif not inmath and item.id == 7 then % Apply recursively
6337     item.replace = item.replace and Babel.locale_map(item.replace)
6338     item.pre      = item.pre and Babel.locale_map(item.pre)
6339     item.post     = item.post and Babel.locale_map(item.post)
6340 elseif item.id == node.id'math' then
6341     inmath = (item.subtype == 0)
6342 end
6343 end
6344 return head
6345 end
6346 }

```

The code for \babelcharproperty is straightforward. Just note the modified lua table can be different.

```

6347 \newcommand\babelcharproperty[1]{%
6348   \count@=#1\relax
6349   \ifvmode
6350     \expandafter\bbl@chprop
6351   \else
6352     \bbl@error{charproperty-only-vertical}{}{}{}%
6353   \fi}
6354 \newcommand\bbl@chprop[3][\the\count@]{%
6355   \@tempcnta=#1\relax
6356   \bbl@ifunset{bbl@chprop@#2}% {unknown-char-property}
6357   {\bbl@error{unknown-char-property}{}{#2}{}%
6358   }%
6359   \loop
6360     \bbl@cs{chprop@#2}{#3}%
6361   \ifnum\count@<\@tempcnta
6362     \advance\count@\@ne
6363   \repeat}
6364 %
6365 \def\bbl@chprop@direction#1{%
6366   \directlua{
6367     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6368     Babel.characters[\the\count@]['d'] = '#1'
6369   }}
6370 \let\bbl@chprop@bc\bbl@chprop@direction
6371 %
6372 \def\bbl@chprop@mirror#1{%
6373   \directlua{
6374     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6375     Babel.characters[\the\count@]['m'] = '\number#1'
6376   }}
6377 \let\bbl@chprop@bmg\bbl@chprop@mirror
6378 %
6379 \def\bbl@chprop@linebreak#1{%

```

```

6380 \directlua{
6381   Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6382   Babel.cjk_characters[\the\count@]['c'] = '#1'
6383 }
6384 \let\bbl@chprop@lb\bbl@chprop@linebreak
6385 %
6386 \def\bbl@chprop@locale#1{%
6387   \directlua{
6388     Babel.chr_to_loc = Babel.chr_to_loc or {}
6389     Babel.chr_to_loc[\the\count@] =
6390       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6391   }

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6392 \directlua{% DL7
6393   Babel.nohyphenation = \the\l@nohyphenation
6394 }

```

Now the $\TeX$ high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ becomes $\text{function}(m) \text{ return } m[1]..m[1]..'-'$ end, where m are the matches returned after applying the pattern. With a mapped capture the functions are similar to $\text{function}(m) \text{ return } \text{Babel.capt_map}(m[1],1)$ end, where the last argument identifies the mapping to be applied to $m[1]$. The way it is carried out is somewhat tricky, but the effect is not dissimilar to lua load – save the code as string in a $\TeX$ macro, and expand this macro at the appropriate place. As $\text{\directlua}$ does not take into account the current catcode of $@$, we just avoid this character in macro names (which explains the internal group, too).

```

6395 \begingroup
6396 \catcode`\~ = 12
6397 \catcode`\% = 12
6398 \catcode`\& = 14
6399 \catcode`\| = 12
6400 \gdef\babelprehyphenation{%&%
6401   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}[]}]
6402 \gdef\babelposthyphenation{%&%
6403   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}[]}]
6404 %
6405 \gdef\bbl@settransform#1[#2]#3#4#5{%&%
6406   \ifcase#1
6407     \bbl@activateprehyphen
6408   \or
6409     \bbl@activateposthyphen
6410   \fi
6411 \begingroup
6412   \def\babeltempa{\bbl@add@list\babeltempb}%&%
6413   \let\babeltempb\empty
6414   \def\bbl@tempa{#5}%&%
6415   \bbl@replace\bbl@tempa{,}{ ,}%&% TODO. Ugly trick to preserve {}
6416   \expandafter\bbl@foreach\expandafter{\bbl@tempa}{%&%
6417     \bbl@ifsamestring{##1}{remove}%&%
6418     {\bbl@add@list\babeltempb{nil}}}%&%
6419     {\directlua{
6420       local rep = [= [#1] =]
6421       local three_args = '%s*=%s*([%- %d%.%a{ }| |)+)%s+([%- %d%.%a{ }| |)+)%s+([%- %d%.%a{ }| |)+)'
6422       &% Numeric passes directly: kern, penalty...
6423       rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6424       rep = rep:gsub('^%s*(insert)%s*', 'insert = true, ')
6425       rep = rep:gsub('^%s*(after)%s*', 'after = true, ')
6426       rep = rep:gsub('(string)%s*=%s*([%-s,]*)', Babel.capture_func)
6427       rep = rep:gsub('node%s*=%s*([%-+)%s*([%-+])', Babel.capture_node)
6428       rep = rep:gsub('(norule)' .. three_args,
6429         'norule = {' .. '%2, %3, %4' .. '}')
6430       if #1 == 0 or #1 == 2 then

```

```

6431     rep = rep:gsub( '(space)' .. three_args,
6432     'space = {' .. '%2, %3, %4' .. '}' )
6433     rep = rep:gsub( '(spacefactor)' .. three_args,
6434     'spacefactor = {' .. '%2, %3, %4' .. '}' )
6435     rep = rep:gsub( '(kashida)%s*=%s*([^\s,]*)', Babel.capture_kashida)
6436     &% Transform values
6437     rep, n = rep:gsub( '{([%a%-%.]+)|([%a%_%.]+)}',
6438     function(v,d)
6439         return string.format (
6440             '{\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6441             v,
6442             load( 'return Babel.locale_props'..
6443             '\the\csname bbl@id@@#3\endcsname].' .. d)() )
6444         end )
6445     rep, n = rep:gsub( '{([%a%-%.]+)|([%-d%.]+)}',
6446     '{\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6447 end
6448 if #1 == 1 then
6449     rep = rep:gsub( '(no)%s*=%s*([^\s,]*)', Babel.capture_func)
6450     rep = rep:gsub( '(pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6451     rep = rep:gsub( '(post)%s*=%s*([^\s,]*)', Babel.capture_func)
6452 end
6453 tex.print([[string\babeltempa{[] .. rep .. [[]]])
6454 }]}&%
6455 \bbl@foreach\babeltempb{&%
6456     \bbl@forkv{##1}}{&%
6457         \in@{,###1},{}{nil,step,data,remove,insert,string,no,pre,no,&%
6458         post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6459         \ifin\else
6460             \bbl@error{bad-transform-option}{###1}{}}{&%
6461             \fi}}&%
6462 \let\bbl@kv@attribute\relax
6463 \let\bbl@kv@label\relax
6464 \let\bbl@kv@fonts\empty
6465 \let\bbl@kv@prepend\relax
6466 \bbl@forkv{#2}{\bbl@csarg\edef{kv{##1}{##2}}&%
6467 \ifx\bbl@kv@fonts\empty\else\bbl@settransfont\fi
6468 \ifx\bbl@kv@attribute\relax
6469     \ifx\bbl@kv@label\relax\else
6470         \bbl@exp{\\bbl@trim@def\\bbl@kv@fonts{\bbl@kv@fonts}}&%
6471         \bbl@replace\bbl@kv@fonts{ }{,}&%
6472         \edef\bbl@kv@attribute{\bbl@ATR@{\bbl@kv@label @#3@{\bbl@kv@fonts}}&%
6473         \count@ \z@
6474         \def\bbl@elt##1##2##3{&%
6475             \bbl@ifsamestring{#3,\bbl@kv@label}{##1,##2}&%
6476             {\bbl@ifsamestring{\bbl@kv@fonts}{##3}&%
6477             {\count@ \@ne}&%
6478             {\bbl@error{font-conflict-transforms}{}}{}}}&%
6479             {}}}&%
6480 \bbl@transfont@list
6481 \ifnum\count@=\z@
6482     \bbl@exp{\global\\bbl@add\\bbl@transfont@list
6483     {\bbl@elt{#3}{\bbl@kv@label}{\bbl@kv@fonts}}}&%
6484 \fi
6485 \bbl@ifunset{\bbl@kv@attribute}&%
6486     {\global\bbl@carg\newattribute{\bbl@kv@attribute}}&%
6487     {}&%
6488     \global\bbl@carg\setattribute{\bbl@kv@attribute}\@ne
6489 \fi
6490 \else
6491     \edef\bbl@kv@attribute{\expandafter\bbl@striplash\bbl@kv@attribute}&%
6492 \fi
6493 \directlua{

```

```

6494     local lbkr = Babel.linebreaking.replacements[#1]
6495     local u = unicode.utf8
6496     local id, attr, label
6497     if #1 == 0 then
6498         id = \the\csname bbl@id@@#3\endcsname\space
6499     else
6500         id = \the\csname l@#3\endcsname\space
6501     end
6502     \ifx\bbl@kv@attribute\relax
6503         attr = -1
6504     \else
6505         attr = luatexbase.registernumber'\bbl@kv@attribute'
6506     \fi
6507     \ifx\bbl@kv@label\relax\else &% Same refs:
6508         label = [==[\bbl@kv@label]==]
6509     \fi
6510     &% Convert pattern:
6511     local patt = string.gsub([==[#4]==], '%s', '')
6512     if #1 == 0 then
6513         patt = string.gsub(patt, '|', ' ')
6514     end
6515     if not u.find(patt, '()', nil, true) then
6516         patt = '()' .. patt .. '()'
6517     end
6518     patt = string.gsub(patt, '%(%)%^', '^()')
6519     patt = string.gsub(patt, '%$$(%)', '()$')
6520     patt = u.gsub(patt, '{(.)}',
6521         function (n)
6522             return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6523         end)
6524     patt = u.gsub(patt, '{(%x%x%x%x+)}',
6525         function (n)
6526             return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6527         end)
6528     lbkr[id] = lbkr[id] or {}
6529     table.insert(lbkr[id], \ifx\bbl@kv@prepend\relax\else 1,\fi
6530         { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6531     }&%
6532 \endgroup}
6533 \endgroup
6534 %
6535 \let\bbl@transfont@list\@empty
6536 \def\bbl@settransfont{%
6537     \global\let\bbl@settransfont\relax % Execute only once
6538     \gdef\bbl@transfont{%
6539         \def\bbl@elt####1####2####3{%
6540             \bbl@ifblank{####3}%
6541                 {\count@tw@}% Do nothing if no fonts
6542                 {\count@z@
6543                     \bbl@vforeach{####3}{%
6544                         \def\bbl@tempd{#####1}%
6545                         \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6546                         \ifx\bbl@tempd\bbl@tempe
6547                             \count@\@ne
6548                         \else\ifx\bbl@tempd\bbl@transfam
6549                             \count@\@ne
6550                         \fi\fi}%
6551                     \ifcase\count@
6552                         \bbl@csarg\unsetattribute{ATR@####2@####1@####3}%
6553                     \or
6554                         \bbl@csarg\setattribute{ATR@####2@####1@####3}\@ne
6555                     \fi}%
6556                 \bbl@transfont@list}%

```



```

6557 \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6558 \gdef\bbl@transfam{-unknown-}%
6559 \bbl@foreach\bbl@font@fams{%
6560   \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6561   \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6562   {\xdef\bbl@transfam{##1}}%
6563   {}}
6564 %
6565 \DeclareRobustCommand\enablelocaletransform[1]{%
6566   \bbl@ifunset{bbl@ATR@#1@\language @}%
6567   {\bbl@error{transform-not-available}{#1}{}}%
6568   {\bbl@csarg\setattribute{ATR@#1@\language @}{\@ne}}
6569 \DeclareRobustCommand\disablelocaletransform[1]{%
6570   \bbl@ifunset{bbl@ATR@#1@\language @}%
6571   {\bbl@error{transform-not-available-b}{#1}{}}%
6572   {\bbl@csarg\unsetattribute{ATR@#1@\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6573 \def\bbl@activateposthyphen{%
6574   \let\bbl@activateposthyphen\relax
6575   \ifx\bbl@attr@hboxed\undefined
6576     \newattribute\bbl@attr@hboxed
6577   \fi
6578   \directlua{
6579     require('babel-transforms.lua')
6580     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6581   }}
6582 \def\bbl@activateprehyphen{%
6583   \let\bbl@activateprehyphen\relax
6584   \ifx\bbl@attr@hboxed\undefined
6585     \newattribute\bbl@attr@hboxed
6586   \fi
6587   \directlua{
6588     require('babel-transforms.lua')
6589     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6590   }}
6591 \newcommand\SetTransformValue[3]{%
6592   \directlua{
6593     Babel.locale_props[\the\csname bbl@id@@#1\endcsname].vars["#2"] = #3
6594   }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6595 \newcommand\ShowBabelTransforms[1]{%
6596   \bbl@activateprehyphen
6597   \bbl@activateposthyphen
6598   \begingroup
6599   \directlua{ Babel.show_transforms = true }%
6600   \setbox\z@\vbox{#1}%
6601   \directlua{ Babel.show_transforms = false }%
6602   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `] == ]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6603 \newcommand\localeprehyphenation[1]{%
6604   \directlua{ Babel.string_prehyphenation([==#1==], \the\localeid) }}

```

10.11.Bidi

As a first step, add a handler for bidi and digits (and potentially other processes) just before luaotfload is applied, which is loaded by default by $\TeX$. Just in case, consider the possibility it has not been loaded.

```
6605 \def\bbl@activate@preotf{%
6606   \let\bbl@activate@preotf\relax % only once
6607   \directlua{
6608     function Babel.pre_otfload_v(head)
6609       if Babel.numbers and Babel.digits_mapped then
6610         head = Babel.numbers(head)
6611       end
6612       if Babel.bidi_enabled then
6613         head = Babel.bidi(head, false, dir)
6614       end
6615       return head
6616     end
6617     %
6618     function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6619       if Babel.numbers and Babel.digits_mapped then
6620         head = Babel.numbers(head)
6621       end
6622       if Babel.bidi_enabled then
6623         head = Babel.bidi(head, false, dir)
6624       end
6625       return head
6626     end
6627     %
6628     luatexbase.add_to_callback('pre_linebreak_filter',
6629       Babel.pre_otfload_v,
6630       'Babel.pre_otfload_v',
6631       Babel.priority_in_callback('pre_linebreak_filter',
6632         'luaotfload.node_processor') or nil)
6633     %
6634     luatexbase.add_to_callback('hpack_filter',
6635       Babel.pre_otfload_h,
6636       'Babel.pre_otfload_h',
6637       Babel.priority_in_callback('hpack_filter',
6638         'luaotfload.node_processor') or nil)
6639   }}
```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@mathboxdir` hack is activated every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in basic-r.

```
6640 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6641   \let\bbl@beforeforeign\leavevmode
6642   \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6643   \RequirePackage{luatexbase}
6644   \bbl@activate@preotf
6645   \directlua{
6646     require('babel-data-bidi.lua')
6647     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6648       require('babel-bidi-basic.lua')
6649     \or
6650       require('babel-bidi-basic-r.lua')
6651     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6652     table.insert(Babel.ranges, {0xF0000, 0xFFFFFD, 'on'})
6653     table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6654   \fi}
6655   \newattribute\bbl@attr@dir
6656   \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6657   \bbl@exp{\output{\bodydir\pagedir\the\output}}
```

```

6658 \fi
6659 %
6660 \chardef\bbl@thetextdir\z@
6661 \chardef\bbl@thepardir\z@
6662 \def\bbl@setluadir#1#2{% 1=\text/pardirection 2= 0:l 1:r 2:a1
6663   \ifcase#2\relax
6664     \ifcase#1\else#1=\z@\fi
6665   \else
6666     \ifcase#1#1=\@ne\fi
6667   \fi}

\bbl@attr@dir stores the directions with a mask: ..00PPTT, with masks 0xC (PP is the par dir) and
0x3 (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

6668 \def\bbl@thedir{0}
6669 \def\bbl@textdir#1{%
6670   \bbl@setluadir\textdirection{#1}%
6671   \chardef\bbl@thetextdir#1\relax
6672   \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6673   \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6674 \def\bbl@pardir#1{% Used twice
6675   \bbl@setluadir\pardirection{#1}%
6676   \chardef\bbl@thepardir#1\relax}
6677 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6678 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to
'tabular', which is based on a fake math.

6679 \ifnum\bbl@bidimode>\z@ % Any bidi=
6680   \def\bbl@insidemath{0}%
6681   \def\bbl@everymath{\def\bbl@insidemath{1}}
6682   \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6683   \frozen@everymath\expandafter{%
6684     \expandafter\bbl@everymath\the\frozen@everymath}
6685   \frozen@everydisplay\expandafter{%
6686     \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6687   \AtBeginDocument{
6688     \directlua{
6689       function Babel.math_box_dir(head)
6690         if not (token.get_macro('bbl@insidemath') == '0') then
6691           if Babel.hlist_has_bidi(head) then
6692             local d = node.new(node.id'dir')
6693             d.dir = '+TRT'
6694             for item in node.traverse(head) do
6695               if item.id == 11 or item.id == node.id'glyph' then
6696                 head = node.insert_before(head, item, d)
6697                 break
6698             end
6699           end
6700           local inmath = false
6701           for item in node.traverse(head) do
6702             if item.id == 11 then
6703               inmath = (item.subtype == 0)
6704             elseif not inmath then
6705               node.set_attribute(item,
6706                 Babel.attr_dir, token.get_macro('bbl@thedir'))
6707             end
6708           end
6709         end
6710       end
6711       return head
6712     end
6713     luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6714       "Babel.math_box_dir", 0)
6715     if Babel.unset_atdir then

```

```

6716     luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6717     "Babel.unset_atdir")
6718     luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6719     "Babel.unset_atdir")
6720     end
6721     luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6722     local dir = tex.mathdirection
6723     local n = node.new("dir")
6724     n.direction = dir
6725     head = node.insert_before(head, head, n)
6726     n = node.new("dir", 1)
6727     n.direction = dir
6728     head = node.insert_after(head, node.tail(head), n)
6729     return head
6730     end, "Babel.ensure_math_dir")
6731   }}%
6732 \fi

Experimental. Tentative name.

6733 \DeclareRobustCommand\localebox[1]{%
6734   {\def\bbl@insidemath{0}%
6735     \mbox{\foreignlanguage{\language}\{#1\}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidibasic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I’ve made some progress in graphics, but they’re essentially hacks; I’ve also made some progress in ‘tabular’, but when I decided to tackle math (both standard math and ‘amsmath’) the nightmare began. I’m still not sure how ‘amsmath’ should be modified, but the main problem is that, boxes are “generic” containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with ‘math’ (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolumn` still fails.

```

6736 \bbl@trace{Redefinitions for bidi layout}
6737 %
6738 <<(*More package options)>> ≡
6739 \chardef\bbl@eqnpos\z@
6740 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6741 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6742 <</More package options>>
6743 %
6744 \ifnum\bbl@bidimode>\z@ % Any bidi=
6745   \matheqdirmode\@ne % Several luatex primitives.
6746   \mathemptydisplaymode\@ne % For fixes.
6747   \breakafterdirmode\@ne %
6748   \fixupboxesmode\@ne %
6749   \let\bbl@eqnudir\relax
6750   \def\bbl@eqdel{()}
6751   \def\bbl@eqnum{%
6752     {\normalfont\normalcolor
6753       \expandafter\@firstoftwo\bbl@eqdel
6754       \theequation

```

```

6755 \expandafter\@secondoftwo\bbl@eqdel}}
6756 \def\bbl@puteqno#1{\eqno\hbox{#1}}
6757 \def\bbl@putleqno#1{\leqno\hbox{#1}}
6758 \def\bbl@eqno@flip#1{% So
6759 \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6760 \eqno
6761 \hb@xt@.01pt{%
6762 \hb@xt@displaywidth{\hss{#1\glet\bbl@upset\@currentlabel}}\hss}%
6763 \else
6764 \leqno\hbox{#1\glet\bbl@upset\@currentlabel}%
6765 \fi
6766 \bbl@exp{\def\\@currentlabel{\bbl@upset}}}}
6767 \def\bbl@leqno@flip#1{%
6768 \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6769 \leqno
6770 \hb@xt@.01pt{%
6771 \hss\hb@xt@displaywidth{#1\glet\bbl@upset\@currentlabel}\hss}%
6772 \else
6773 \eqno\hbox{#1\glet\bbl@upset\@currentlabel}%
6774 \fi
6775 \bbl@exp{\def\\@currentlabel{\bbl@upset}}}}
6776 %
6777 \AtBeginDocument{%
6778 \ifx\bbl@noamsmath\relax\else
6779 \ifx\maketag@@@ \@undefined % Normal equation, eqnarray
6780 \ifnum\bbl@eqnpos=\tw@
6781 \bbl@replace\equation{\hb@xt@linewidth}%
6782 {\hbox bdir\mathdirection to\linewidth}%
6783 \bbl@carg\bbl@sreplace{[ ]}{\hb@xt@linewidth}%
6784 {\hbox bdir\mathdirection to\linewidth}%
6785 \fi
6786 \AddToHook{env/equation/begin}{%
6787 \ifnum\bbl@thetextdir>\z@
6788 \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6789 \let\@eqnnum\bbl@eqnum
6790 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6791 \chardef\bbl@thetextdir\z@
6792 \bbl@add\normalfont{\bbl@eqnodir}%
6793 \ifcase\bbl@eqnpos
6794 \let\bbl@puteqno\bbl@eqno@flip
6795 \or
6796 \let\bbl@puteqno\bbl@leqno@flip
6797 \fi
6798 \fi}%
6799 \ifnum\bbl@eqnpos=\tw@\else
6800 \def\endequation{\bbl@puteqno{\@eqnnum}$$\@ignoretrue}%
6801 \fi
6802 \AddToHook{env/eqnarray/begin}{%
6803 \ifnum\bbl@thetextdir>\z@
6804 \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6805 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6806 \chardef\bbl@thetextdir\z@
6807 \bbl@add\normalfont{\bbl@eqnodir}%
6808 \ifnum\bbl@eqnpos=\@ne
6809 \def\@eqnnum{%
6810 \setbox\z@\hbox{\bbl@eqnum}%
6811 \hbox to0.01pt{\hss\hbox todisplaywidth{box\z@\hss}}}%
6812 \else
6813 \let\@eqnnum\bbl@eqnum
6814 \fi
6815 \fi}
6816 \else % amstex
6817 \ifnum\bbl@eqnpos=\tw@

```

```

6818 \bbl@exp{% Hack to hide maybe undefined conditionals:
6819 \\bbl@sreplace\\multline\cr{<f@flegn>\tabskip\z@skip<fi>\cr{}}%
6820 \fi
6821 \bbl@exp{% Hack to hide maybe undefined conditionals:
6822 \chardef\bbl@eqnpos=0%
6823 <iftagsleft@>1<else><f@flegn>2<fi><fi>\relax}%
6824 \ifnum\bbl@eqnpos=\@ne
6825 \let\bbl@ams@lap\hbox
6826 \else
6827 \let\bbl@ams@lap\llap
6828 \fi
6829 \ExplSyntaxOn % Required by \bbl@sreplace with \intertext@
6830 \bbl@sreplace\intertext@{\normalbaselines}%
6831 {\normalbaselines
6832 \ifx\bbl@eqnodir\relax\else\bbl@pdir\@ne\bbl@eqnodir\fi}%
6833 \ExplSyntaxOff
6834 \def\bbl@ams@tagbox#1#2{#1{\bbl@eqnodir#2}}% #1=hbox|@lap|flip
6835 \ifx\bbl@ams@lap\hbox % leqno
6836 \def\bbl@ams@flip#1{%
6837 \hbox to 0.01pt{\hss\hbox to\displaywidth{#1}\hss}}%
6838 \else % eqno
6839 \def\bbl@ams@flip#1{%
6840 \hbox to 0.01pt{\hbox to\displaywidth{\hss{#1}}\hss}}%
6841 \fi
6842 \def\bbl@ams@preset#1{%
6843 \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6844 \ifnum\bbl@thetextdir>\z@
6845 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6846 \bbl@sreplace\textdef@{\hbox}{\bbl@ams@tagbox\hbox}%
6847 \bbl@sreplace\maketag@@@{\hbox}{\bbl@ams@tagbox#1}%
6848 \fi}%
6849 \def\bbl@ams@equation{%
6850 \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6851 \ifnum\bbl@thetextdir>\z@
6852 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6853 \chardef\bbl@thetextdir\z@
6854 \bbl@add\normalfont{\bbl@eqnodir}%
6855 \ifcase\bbl@eqnpos
6856 \def\veqno##1##2{\bbl@eqno@flip{##1##2}}%
6857 \or
6858 \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6859 \fi
6860 \fi}%
6861 \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6862 \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6863 \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6864 \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6865 \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6866 \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6867 \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6868 \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6869 \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6870 \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6871 \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\hbox}%
6872 % Hackish, for proper alignment. Don't ask me why it works!:
6873 \bbl@exp{% Avoid a 'visible' conditional
6874 \\AddToHook{env/align*/end}{<iftag>\<else>\\tag*{\<fi>}}%
6875 \\AddToHook{env/alignat*/end}{<iftag>\<else>\\tag*{\<fi>}}%
6876 \AddToHook{env/flalign/begin}{\bbl@ams@preset\hbox}%
6877 \AddToHook{env/split/before}{%
6878 \def\bbl@mathboxdir{\def\bbl@insidemath{1}}%
6879 \ifnum\bbl@thetextdir>\z@
6880 \bbl@ifsamestring\@currentenv{equation}%

```

```

6881         {\ifx\babel@ams@lap\hbox % legno
6882         \def\babel@ams@flip#1{%
6883             \hbox to 0.01pt{\hbox to\displaywidth{#{1}\hss}\hss}}%
6884         \else
6885         \def\babel@ams@flip#1{%
6886             \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}%
6887         \fi}%
6888     {}%
6889 \fi}%
6890 \fi\fi}
6891 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6892 \def\babel@provide@extra#1{%
6893     % == onchar ==
6894     \ifx\babel@KVP@onchar\@nnil\else
6895         \babel@luahyphenate
6896         \babel@exp{%
6897             \\\AddToHook{env/document/before}{%
6898                 {\let\\babel@ifrestoring\\@firstoftwo
6899                     \\\select@language{#1}{}}}%
6900         \directlua{
6901             if Babel.locale_mapped == nil then
6902                 Babel.locale_mapped = true
6903                 Babel.linebreaking.add_before(Babel.locale_map, 1)
6904                 Babel.loc_to_scr = {}
6905                 Babel.chr_to_loc = Babel.chr_to_loc or {}
6906             end
6907             Babel.locale_props[\the\localeid].letters = false
6908         }%
6909         \babel@xin@{ letters }{ \babel@KVP@onchar\space}%
6910         \ifin@
6911             \directlua{
6912                 Babel.locale_props[\the\localeid].letters = true
6913             }%
6914         \fi
6915         \babel@xin@{ ids }{ \babel@KVP@onchar\space}%
6916         \ifin@
6917             \ifx\babel@starthyphens\@undefined % Needed if no explicit selection
6918                 \AddBabelHook{babel-onchar}{beforestart}{\babel@starthyphens}%
6919             \fi
6920             \babel@exp{\\babel@add\\babel@starthyphens
6921                 {\\babel@patterns@lua{\language\language}}}%
6922             \directlua{
6923                 if Babel.script_blocks['\babel@cl{sbc}'] then
6924                     Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\babel@cl{sbc}']
6925                     Babel.locale_props[\the\localeid].lg = \the\@nameuse{l@language}\space
6926                 end
6927             }%
6928         \fi
6929         \babel@xin@{ fonts }{ \babel@KVP@onchar\space}%
6930         \ifin@
6931             \babel@ifunset{babel@lsys@language}{\babel@provide@lsys{language}}{%
6932                 \babel@ifunset{babel@wdir@language}{\babel@provide@dirs{language}}{%
6933                     \directlua{
6934                         if Babel.script_blocks['\babel@cl{sbc}'] then
6935                             Babel.loc_to_scr[\the\localeid] =
6936                                 Babel.script_blocks['\babel@cl{sbc}']
6937                         end}%
6938                     \ifx\babel@mapselect\@undefined
6939                         \AtBeginDocument{%
6940                             \babel@patchfont{\babel@mapselect}}%
6941                         {\selectfont}}%

```

```

6942 \def\bb@mapselect{%
6943 \let\bb@mapselect\relax
6944 \edef\bb@prefontid{\fontid\font}}%
6945 \def\bb@mapdir##1{%
6946 \begingroup
6947 \setbox\z@\hbox{% Force text mode
6948 \def\language{##1}%
6949 \let\bb@ifrestoring\@firstoftwo % To avoid font warning
6950 \bb@switchfont
6951 \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
6952 \directlua{
6953 Babel.locale_props[\the\csname \bb@id@##1\endcsname]%
6954 [\bb@prefontid] = \fontid\font\space}%
6955 \fi}%
6956 \endgroup}%
6957 \fi
6958 \bb@exp{\bb@add\bb@mapselect{\bb@mapdir{\language}}}%
6959 \fi
6960 \fi
6961 % == mapfont ==
6962 % For bidi texts, to switch the font based on direction. Deprecated
6963 \ifx\bb@KVP@mapfont\@nnil\else
6964 \bb@ifsamestring{\bb@KVP@mapfont}{direction}}{%
6965 {\bb@error{unknown-mapfont}}}%
6966 \bb@ifunset{\bb@lsys\language}{\bb@provide\lsys\language}}{%
6967 \bb@ifunset{\bb@wdir\language}{\bb@provide\dirs\language}}{%
6968 \ifx\bb@mapselect\@undefined
6969 \AtBeginDocument{%
6970 \bb@patchfont{\bb@mapselect}}%
6971 {\selectfont}}%
6972 \def\bb@mapselect{%
6973 \let\bb@mapselect\relax
6974 \edef\bb@prefontid{\fontid\font}}%
6975 \def\bb@mapdir##1{%
6976 {\def\language{##1}%
6977 \let\bb@ifrestoring\@firstoftwo % avoid font warning
6978 \bb@switchfont
6979 \directlua{Babel.fontmap
6980 [\the\csname \bb@wdir##1\endcsname]%
6981 [\bb@prefontid]=\fontid\font}}}%
6982 \fi
6983 \bb@exp{\bb@add\bb@mapselect{\bb@mapdir{\language}}}%
6984 \fi
6985 % == Line breaking: CJK quotes ==
6986 \ifcase\bb@engine\or
6987 \bb@xin{/c}{\bb@cl{\lnbrk}}%
6988 \ifin@
6989 \bb@ifunset{\bb@quote\language}}{%
6990 {\directlua{
6991 Babel.locale_props[\the\localeid].cjk_quotes = {}
6992 local cs = 'op'
6993 for c in string.utfvalues(
6994 [[\csname \bb@quote\language\endcsname]]) do
6995 if Babel.cjk_characters[c].c == 'qu' then
6996 Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
6997 end
6998 cs = ( cs == 'op') and 'cl' or 'op'
6999 end
7000 }}%
7001 \fi
7002 \fi
7003 % == Counters: mapdigits ==
7004 % Native digits

```



```

7005 \ifx\bbl@KVP@mapdigits\@nnil\else
7006 \bbl@ifunset\bbl@dgnat@\languagename\}%
7007 {\bbl@activate@preotf
7008 \directlua{
7009   Babel.digits_mapped = true
7010   Babel.digits = Babel.digits or {}
7011   Babel.digits[\the\localeid] =
7012     table.pack(string.utfvalue('\bbl@cl{dgnat}'))
7013   if not Babel.numbers then
7014     function Babel.numbers(head)
7015       local LOCALE = Babel.attr_locale
7016       local GLYPH = node.id'glyph'
7017       local inmath = false
7018       for item in node.traverse(head) do
7019         if not inmath and item.id == GLYPH then
7020           local temp = node.get_attribute(item, LOCALE)
7021           if Babel.digits[temp] then
7022             local chr = item.char
7023             if chr > 47 and chr < 58 then
7024               item.char = Babel.digits[temp][chr-47]
7025             end
7026           end
7027         elseif item.id == node.id'math' then
7028           inmath = (item.subtype == 0)
7029         end
7030       end
7031       return head
7032     end
7033   end
7034 }}%
7035 \fi
7036 % == transforms ==
7037 \ifx\bbl@KVP@transforms\@nnil\else
7038 \def\bbl@elt##1##2##3{%
7039   \in@{$transforms.}{$##1}%
7040   \ifin@
7041     \def\bbl@tempa{##1}%
7042     \bbl@replace\bbl@tempa{transforms.}{}%
7043     \bbl@carg\bbl@transforms{babel\bbl@tempa}{##2}{##3}%
7044   \fi}%
7045 \bbl@exp{%
7046   \\bbl@ifblank{\bbl@cl{dgnat}}%
7047   {\let\\bbl@tempa\relax}%
7048   {\def\\bbl@tempa{%
7049     \\bbl@elt{transforms.prehyphenation}%
7050     {digits.native.1.0}{([0-9])}%
7051     \\bbl@elt{transforms.prehyphenation}%
7052     {digits.native.1.1}{string={1\string|0123456789\string|\bbl@cl{dgnat}}}}}%
7053 \ifx\bbl@tempa\relax\else
7054 \toks@{\expandafter\expandafter\expandafter{%
7055   \csname bbl@inidata@\languagename\endcsname}%
7056   \bbl@csarg\edef{inidata@\languagename}{%
7057     \unexpanded\expandafter{\bbl@tempa}%
7058     \the\toks@}%
7059 \fi
7060 \csname bbl@inidata@\languagename\endcsname
7061 \bbl@release@transforms\relax % \relax closes the last item.
7062 \fi}

```

Start tabular here:

```

7063 \def\localerestoredirs{%
7064   \ifcase\bbl@thetextdir
7065     \ifnum\textdirection=\z@\else\textdirection=\z@\fi

```

```

7066 \else
7067   \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi
7068 \fi
7069 \ifcase\bbl@thepardir
7070   \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7071 \else
7072   \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7073 \fi}
7074 %
7075 \IfBabelLayout{tabular}%
7076   {\chardef\bbl@tabular@mode\tw}% All RTL
7077   {\IfBabelLayout{notabular}%
7078     {\chardef\bbl@tabular@mode\z}%
7079     {\chardef\bbl@tabular@mode\@ne}}% Mixed, with LTR cols
7080 %
7081 \ifnum\bbl@bidimode>\@ne % Any lua bidi= except default=1
7082   % Redefine: vrules mess up dirs (why?).
7083   \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7084   \ifcase\bbl@tabular@mode\or % 1 = Mixed - default
7085     \let\bbl@parabefore\relax
7086     \AddToHook{para/before}{\bbl@parabefore}
7087     \AtBeginDocument{%
7088       \bbl@replace\@tabular{$}{$%
7089         \def\bbl@insidemath{0}%
7090         \def\bbl@parabefore{\localerestoredirs}}%
7091       \ifnum\bbl@tabular@mode=\@ne
7092         \bbl@ifunset{\@tabclassz}{\{%
7093           \bbl@exp{% Hide conditionals
7094             \\bbl@sreplace\\ \@tabclassz
7095             {\<ifcase>\\ \@chnum}%
7096             {\localerestoredirs\<ifcase>\\ \@chnum}}}%
7097           \@ifpackageloaded{colortbl}%
7098             {\bbl@sreplace\@classz
7099               {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7100             {\@ifpackageloaded{array}%
7101               {\bbl@exp{% Hide conditionals
7102                 \\bbl@sreplace\\ \@classz
7103                 {\<ifcase>\\ \@chnum}%
7104                 {\bgroup\\ \localerestoredirs\<ifcase>\\ \@chnum}%
7105                 \\bbl@sreplace\\ \@classz
7106                 {\do@row@strut\<fi>}{\do@row@strut\<fi>\egroup}}}%
7107               {}}}%
7108       \fi}%
7109   \or % 2 = All RTL - tabular
7110     \let\bbl@parabefore\relax
7111     \AddToHook{para/before}{\bbl@parabefore}%
7112     \AtBeginDocument{%
7113       \@ifpackageloaded{colortbl}%
7114         {\bbl@replace\@tabular{$}{$%
7115           \def\bbl@insidemath{0}%
7116           \def\bbl@parabefore{\localerestoredirs}}%
7117         \bbl@sreplace\@classz
7118         {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7119       {}}}%
7120 \fi

```

Very likely the \output routine must be patched in a quite general way to make sure the \bodydir is set to \pagedir. Note outside \output they can be different (and often are). For the moment, two *ad hoc* changes.

```

7121 \AtBeginDocument{%
7122   \@ifpackageloaded{multicol}%
7123   {\toks@\expandafter{\multi@column@out}%
7124     \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%

```

```

7125     {}%
7126     \@ifpackageloaded{paracol}%
7127     {\edef\pcol@output{%
7128       \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7129     {}}%
7130 \fi

```

Finish here if there in no layout.

```

7131 \ifx\bbbl@opt@layout\@nnil\endinput\fi

```

OMEGA provided a companion to `\mathdir` (`\nextfakemath`) for those cases where we did not want it to be applied, so that the writing direction of the main text was left unchanged. `\bbbl@nextfake` is an attempt to emulate it, because `luatex` has removed it without an alternative. Used in `tabular`, `\underline` and `\LaTeX`. Also, `\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7132 \chardef\bbbl@trace@vboxdir\z@
7133 \ifnum\bbbl@bidimode>\z@ % Any bidi=
7134   \def\bbbl@nextfake#1{% non-local changes, use always inside a group!
7135     \bbbl@exp{%
7136       \mathdir\the\bodydir
7137       #1%           Once entered in math, set boxes to restore values
7138       \def\bbbl@insidemath{0}%
7139       \<ifmmode>%
7140         \everyvbox{%
7141           \the\everyvbox
7142           \bodydir\the\bodydir
7143           \mathdir\the\mathdir
7144           \everyhbox{\the\everyhbox}%
7145           \everyvbox{\the\everyvbox}}%
7146         \everyhbox{%
7147           \the\everyhbox
7148           \bodydir\the\bodydir
7149           \mathdir\the\mathdir
7150           \everyhbox{\the\everyhbox}%
7151           \everyvbox{\the\everyvbox}}%
7152       \<fi>}}%
7153 \IfBabelLayout{nopars}{%
7154   {\edef\bbbl@opt@layout{\bbbl@opt@layout.pars.}}%
7155 \IfBabelLayout{pars}
7156   {\chardef\bbbl@trace@vboxdir\@ne
7157     \def\@hangfrom#1{%
7158       \setbox\@tempboxa\hbox{#1}%
7159       \hangindent\wd\@tempboxa
7160       \ifnum\bbbl@vbox@bodydir=\pardirection\else
7161         \shapemode\@ne
7162       \fi
7163       \noindent\box\@tempboxa}}
7164   {}
7165 \fi
7166 %
7167 \IfBabelLayout{tabular}
7168   {\let\bbbl@OL@tabular\@tabular
7169     \bbbl@exp{\in{\UseMathForPositioningText}{\@tabular}}}%
7170   \ifin@
7171     \bbbl@replace\@tabular{\UseMathForPositioningText$}%
7172     {\bbbl@nextfake{\UseMathForPositioningText$}}%
7173   \else
7174     \bbbl@replace\@tabular{$}{\bbbl@nextfake$}%
7175   \fi
7176   \let\bbbl@NL@tabular\@tabular
7177   \AtBeginDocument{%
7178     \ifx\bbbl@NL@tabular\@tabular\else
7179       \bbbl@exp{\in{\UseMathForPositioningText}{\@tabular}}}%
7180     \ifin@\else

```

```

7181         \ifin@
7182         \bbl@replace\@tabular{\UseMathForPositioningText$}%
7183         {\bbl@nextfake{\UseMathForPositioningText$}}%
7184         \else
7185         \bbl@replace\@tabular{$}{\bbl@nextfake$}%
7186         \fi
7187     \fi
7188     \let\bbl@NL@@tabular\@tabular
7189 \fi}}
7190 {}

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7191 \IfBabelLayout{lists}
7192 {\chardef\bbl@trace@vboxdir\@ne
7193  \let\bbl@OL@list\list
7194  \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7195  \let\bbl@NL@list\list
7196  \def\bbl@listparshape#1#2#3{%
7197    \parshape #1 #2 #3 %
7198    \ifnum\bbl@vbox@bodydir=\pardirection\else
7199    \shapemode\tw@
7200    \fi}}
7201 {}
7202 %
7203 \ifcase\bbl@trace@vboxdir\else
7204 \AtBeginDocument{\chardef\bbl@vbox@bodydir\pagedirection}%
7205 \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7206 \let\bbl@bidi@vbox\everyvbox
7207 \@nameuse{newtoks}\everyvbox % \outer in Plain
7208 \everyvbox\expandafter{\the\bbl@bidi@vbox}%
7209 \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7210 \fi
7211 %
7212 \IfBabelLayout{graphics}
7213 {\let\bbl@pictresetdir\relax
7214  \def\bbl@pictsetdir#1{%
7215    \ifcase\bbl@thetextdir
7216    \let\bbl@pictresetdir\relax
7217    \else
7218    \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7219    \or\textdir TLT
7220    \else\bodydir TLT \textdir TLT
7221    \fi
7222    % \(\text|par)dir required in pgf:
7223    \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7224    \fi}%
7225 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7226 \directlua{
7227   Babel.get_picture_dir = true
7228   Babel.picture_has_bidi = 0
7229   %
7230   function Babel.picture_dir (head)
7231     if not Babel.get_picture_dir then return head end
7232     if Babel.hlist_has_bidi(head) then
7233       Babel.picture_has_bidi = 1
7234     end
7235     return head
7236   end
7237   luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7238     "Babel.picture_dir")
7239 }%
7240 \AddToHook{package/graphics/after}{%

```

```

7241 \bbl@exp{%
7242   \\bbl@sreplace\\rotatebox{\hbox{\string#2}}
7243   {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}%
7244 \AddToHook{package/graphicx/after}{%
7245   \bbl@exp{%
7246     \\bbl@sreplace\\Grot@box@std{\hbox{\string#2}}
7247     {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}%
7248     \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7249     {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#3}}}%
7250     \\bbl@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}
7251 \AtBeginDocument{%
7252   \long\def\put(#1,#2)#3{%
7253     \@killglue
7254     % Try:
7255     \ifx\bbl@pictresetdir\relax
7256       \def\bbl@tempc{0}%
7257     \else
7258       \directlua{
7259         Babel.get_picture_dir = true
7260         Babel.picture_has_bidi = 0
7261       }%
7262       \setbox\z@\hb@xt\z@{%
7263         \@defaultunitsset\@tempdimc{#1}\unitlength
7264         \kern\@tempdimc
7265         #3\hss}%
7266       \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7267     \fi
7268     % Do:
7269     \@defaultunitsset\@tempdimc{#2}\unitlength
7270     \raise\@tempdimc\hb@xt\z@{%
7271       \@defaultunitsset\@tempdimc{#1}\unitlength
7272       \kern\@tempdimc
7273       {\ifnum\bbl@tempc>\z@\bbl@pictresetdir\fi#3}\hss}%
7274     \ignorespaces}%
7275   \MakeRobust\put}%
7276 \AtBeginDocument
7277 {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7278 \ifx\pgfpicture\undefined\else
7279   \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7280   \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7281   \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7282 \fi
7283 \ifx\tikzpicture\undefined\else
7284   \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw}%
7285   \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7286   \bbl@sreplace\tikz{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7287   \bbl@sreplace\tikzpicture{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7288 \fi
7289 }}
7290 {}

```

Implicitly reverses sectioning labels in bidi=basic-r, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes bidi=basic, but there are some additional readjustments for bidi=default.

```

7291 \IfBabelLayout{counters*}%
7292 {\bbl@add\bbl@opt@layout{.counters.}%
7293   \directlua{
7294     luatexbase.add_to_callback("process_output_buffer",
7295       Babel.discard_sublr , "Babel.discard_sublr") }%
7296   }{}
7297 \IfBabelLayout{counters}%
7298 {\let\bbl@0L@textsuperscript\textsuperscript
7299   \bbl@sreplace\textsuperscript{\m@th}{\m@th\mathdir\pagedir}%

```

```

7300 \let\bbl@latinarabic=\@arabic
7301 \let\bbl@0L@@arabic\@arabic
7302 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7303 \ifpackagewith{babel}{bidi=default}%
7304 {\let\bbl@asciroman=\@roman
7305 \let\bbl@0L@@roman\@roman
7306 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7307 \let\bbl@asciiRoman=\@Roman
7308 \let\bbl@0L@@roman\@Roman
7309 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7310 \let\bbl@0L@labelenumii\labelenumii
7311 \def\labelenumii{}\theenumii}%
7312 \let\bbl@0L@p@enumiii\p@enumiii
7313 \def\p@enumiii{\p@enumii}\theenumii{}\}\}\}

```

Some $\LaTeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7314 \IfBabelLayout{extras}%
7315 {\bbl@ncarg\let\bbl@0L@underline{underline }%
7316 \bbl@carg\bbl@sreplace{underline }\hbox}%
7317 {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7318 \let\bbl@0L@LaTeXe\LaTeXe
7319 \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7320 \if b\expandafter\@car\@fseries\@nil\boldmath\fi
7321 \babelsublr{%
7322 \LaTeX\kern.15em2\bbl@nextfake$_{\textstyle\varepsilon}$}}}
7323 {}
7324 </luatex>

```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7325 <{*transforms}>
7326 Babel.linebreaking.replacements = {}
7327 Babel.linebreaking.replacements[0] = {} -- pre
7328 Babel.linebreaking.replacements[1] = {} -- post
7329
7330 function Babel.tovalue(v)
7331   if type(v) == 'table' then
7332     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7333   else
7334     return v
7335   end
7336 end
7337
7338 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7339
7340 function Babel.set_hboxed(head, gc)
7341   for item in node.traverse(head) do
7342     node.set_attribute(item, Babel.attr_hboxed, 1)
7343   end
7344   return head
7345 end

```

```

7346
7347 Babel.fetch_subtext = {}
7348
7349 Babel.ignore_pre_char = function(node)
7350   return (node.lang == Babel.nohyphenation)
7351 end
7352
7353 Babel.show_transforms = false
7354
7355 -- Merging both functions doesn't seem feasible, because there are too
7356 -- many differences.
7357 Babel.fetch_subtext[0] = function(head)
7358   local word_string = ''
7359   local word_nodes = {}
7360   local lang
7361   local item = head
7362   local inmath = false
7363
7364   while item do
7365
7366     if item.id == 11 then
7367       inmath = (item.subtype == 0)
7368     end
7369
7370     if inmath then
7371       -- pass
7372
7373     elseif item.id == 29 then
7374       local locale = node.get_attribute(item, Babel.attr_locale)
7375
7376       if lang == locale or lang == nil then
7377         lang = lang or locale
7378         if Babel.ignore_pre_char(item) then
7379           word_string = word_string .. Babel.us_char
7380         else
7381           if node.has_attribute(item, Babel.attr_hboxed) then
7382             word_string = word_string .. Babel.us_char
7383           else
7384             word_string = word_string .. unicode.utf8.char(item.char)
7385           end
7386         end
7387         word_nodes[#word_nodes+1] = item
7388       else
7389         break
7390       end
7391
7392     elseif item.id == 12 and item.subtype == 13 then
7393       if node.has_attribute(item, Babel.attr_hboxed) then
7394         word_string = word_string .. Babel.us_char
7395       else
7396         word_string = word_string .. ' '
7397       end
7398       word_nodes[#word_nodes+1] = item
7399
7400       -- Ignore leading unrecognized nodes, too.
7401     elseif word_string ~= '' then
7402       word_string = word_string .. Babel.us_char
7403       word_nodes[#word_nodes+1] = item -- Will be ignored
7404     end
7405
7406     item = item.next
7407   end
7408

```

```

7409 -- Here and above we remove some trailing chars but not the
7410 -- corresponding nodes. But they aren't accessed.
7411 if word_string:sub(-1) == ' ' then
7412     word_string = word_string:sub(1,-2)
7413 end
7414 if Babel.show_transforms then texio.write_nl(word_string) end
7415 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7416 return word_string, word_nodes, item, lang
7417 end
7418
7419 Babel.fetch_subtext[1] = function(head)
7420     local word_string = ''
7421     local word_nodes = {}
7422     local lang
7423     local item = head
7424     local inmath = false
7425
7426     while item do
7427
7428         if item.id == 11 then
7429             inmath = (item.subtype == 0)
7430         end
7431
7432         if inmath then
7433             -- pass
7434
7435         elseif item.id == 29 then
7436             if item.lang == lang or lang == nil then
7437                 lang = lang or item.lang
7438                 if node.has_attribute(item, Babel.attr_hboxed) then
7439                     word_string = word_string .. Babel.us_char
7440                 elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7441                     word_string = word_string .. Babel.us_char
7442                 else
7443                     word_string = word_string .. unicode.utf8.char(item.char)
7444                 end
7445                 word_nodes[#word_nodes+1] = item
7446             else
7447                 break
7448             end
7449
7450         elseif item.id == 7 and item.subtype == 2 then
7451             if node.has_attribute(item, Babel.attr_hboxed) then
7452                 word_string = word_string .. Babel.us_char
7453             else
7454                 word_string = word_string .. '='
7455             end
7456             word_nodes[#word_nodes+1] = item
7457
7458         elseif item.id == 7 and item.subtype == 3 then
7459             if node.has_attribute(item, Babel.attr_hboxed) then
7460                 word_string = word_string .. Babel.us_char
7461             else
7462                 word_string = word_string .. '|'
7463             end
7464             word_nodes[#word_nodes+1] = item
7465
7466         -- (1) Go to next word if nothing was found, and (2) implicitly
7467         -- remove leading USs.
7468         elseif word_string == '' then
7469             -- pass
7470
7471         -- This is the responsible for splitting by words.

```



```

7472     elseif (item.id == 12 and item.subtype == 13) then
7473         break
7474
7475     else
7476         word_string = word_string .. Babel.us_char
7477         word_nodes[#word_nodes+1] = item -- Will be ignored
7478     end
7479
7480     item = item.next
7481 end
7482 if Babel.show_transforms then texio.write_nl(word_string) end
7483 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7484 return word_string, word_nodes, item, lang
7485 end
7486
7487 function Babel.pre_hyphenate_replace(head)
7488     Babel.hyphenate_replace(head, 0)
7489 end
7490
7491 function Babel.post_hyphenate_replace(head)
7492     Babel.hyphenate_replace(head, 1)
7493 end
7494
7495 Babel.us_char = string.char(31)
7496
7497 function Babel.hyphenate_replace(head, mode)
7498     local u = unicode.utf8
7499     local lbkr = Babel.linebreaking.replacements[mode]
7500     local tovalue = Babel.tovalue
7501
7502     local word_head = head
7503
7504     if Babel.show_transforms then
7505         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7506     end
7507
7508     while true do -- for each subtext block
7509
7510         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7511
7512         if Babel.debug then
7513             print()
7514             print((mode == 0) and '@@@@<' or '@@@@>', w)
7515         end
7516
7517         if nw == nil and w == '' then break end
7518
7519         if not lang then goto next end
7520         if not lbkr[lang] then goto next end
7521
7522         -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7523         -- loops are nested.
7524         for k=1, #lbkr[lang] do
7525             local p = lbkr[lang][k].pattern
7526             local r = lbkr[lang][k].replace
7527             local attr = lbkr[lang][k].attr or -1
7528
7529             if Babel.debug then
7530                 print('*****', p, mode)
7531             end
7532
7533             -- This variable is set in some cases below to the first *byte*
7534             -- after the match, either as found by u.match (faster) or the

```

```

7535     -- computed position based on sc if w has changed.
7536     local last_match = 0
7537     local step = 0
7538
7539     -- For every match.
7540     while true do
7541         if Babel.debug then
7542             print('====')
7543         end
7544         local new -- used when inserting and removing nodes
7545         local dummy_node -- used by after
7546
7547         local matches = { u.match(w, p, last_match) }
7548
7549         if #matches < 2 then break end
7550
7551         -- Get and remove empty captures (with ()'s, which return a
7552         -- number with the position), and keep actual captures
7553         -- (from (...)), if any, in matches.
7554         local first = table.remove(matches, 1)
7555         local last = table.remove(matches, #matches)
7556         -- Non re-fetched substrings may contain \31, which separates
7557         -- subsubstrings.
7558         if string.find(w:sub(first, last-1), Babel.us_char) then break end
7559
7560         local save_last = last -- with A()BC()D, points to D
7561
7562         -- Fix offsets, from bytes to unicode. Explained above.
7563         first = u.len(w:sub(1, first-1)) + 1
7564         last = u.len(w:sub(1, last-1)) -- now last points to C
7565
7566         -- This loop stores in a small table the nodes
7567         -- corresponding to the pattern. Used by 'data' to provide a
7568         -- predictable behavior with 'insert' (w_nodes is modified on
7569         -- the fly), and also access to 'remove'd nodes.
7570         local sc = first-1 -- Used below, too
7571         local data_nodes = {}
7572
7573         local enabled = true
7574         for q = 1, last-first+1 do
7575             data_nodes[q] = w_nodes[sc+q]
7576             if enabled
7577                 and attr > -1
7578                 and not node.has_attribute(data_nodes[q], attr)
7579             then
7580                 enabled = false
7581             end
7582         end
7583
7584         -- This loop traverses the matched substring and takes the
7585         -- corresponding action stored in the replacement list.
7586         -- sc = the position in substr nodes / string
7587         -- rc = the replacement table index
7588         local rc = 0
7589
7590         ----- TODO. dummy_node?
7591         while rc < last-first+1 or dummy_node do -- for each replacement
7592             if Babel.debug then
7593                 print('.....', rc + 1)
7594             end
7595             sc = sc + 1
7596             rc = rc + 1
7597

```

```

7598     if Babel.debug then
7599         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7600         local ss = ''
7601         for itt in node.traverse(head) do
7602             if itt.id == 29 then
7603                 ss = ss .. unicode.utf8.char(itt.char)
7604             else
7605                 ss = ss .. '{' .. itt.id .. '}'
7606             end
7607         end
7608         print('*****', ss)
7609     end
7610 end
7611
7612 local crep = r[rc]
7613 local item = w_nodes[sc]
7614 local item_base = item
7615 local placeholder = Babel.us_char
7616 local d
7617
7618 if crep and crep.data then
7619     item_base = data_nodes[crep.data]
7620 end
7621
7622 if crep then
7623     step = crep.step or step
7624 end
7625
7626 if crep and crep.after then
7627     crep.insert = true
7628     if dummy_node then
7629         item = dummy_node
7630     else -- TODO. if there is a node after?
7631         d = node.copy(item_base)
7632         head, item = node.insert_after(head, item, d)
7633         dummy_node = item
7634     end
7635 end
7636
7637 if crep and not crep.after and dummy_node then
7638     node.remove(head, dummy_node)
7639     dummy_node = nil
7640 end
7641
7642 if not enabled then
7643     last_match = save_last
7644     goto next
7645
7646 elseif crep and next(crep) == nil then -- = {}
7647     if step == 0 then
7648         last_match = save_last -- Optimization
7649     else
7650         last_match = utf8.offset(w, sc+step)
7651     end
7652     goto next
7653
7654 elseif crep == nil or crep.remove then
7655     node.remove(head, item)
7656     table.remove(w_nodes, sc)
7657     w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7658     sc = sc - 1 -- Nothing has been inserted.
7659     last_match = utf8.offset(w, sc+1+step)
7660     goto next

```

```

7661
7662     elseif crep and crep.kashida then
7663         node.set_attribute(item,
7664             Babel.attr_kashida,
7665             crep.kashida)
7666         last_match = utf8.offset(w, sc+1+step)
7667         goto next
7668
7669     elseif crep and crep.string then
7670         local str = crep.string(matches)
7671         if str == '' then -- Gather with nil
7672             node.remove(head, item)
7673             table.remove(w_nodes, sc)
7674             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7675             sc = sc - 1 -- Nothing has been inserted.
7676         else
7677             local loop_first = true
7678             for s in string.utfvalues(str) do
7679                 d = node.copy(item_base)
7680                 d.char = s
7681                 if loop_first then
7682                     loop_first = false
7683                     head, new = node.insert_before(head, item, d)
7684                     if sc == 1 then
7685                         word_head = head
7686                     end
7687                     w_nodes[sc] = d
7688                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7689                 else
7690                     sc = sc + 1
7691                     head, new = node.insert_before(head, item, d)
7692                     table.insert(w_nodes, sc, new)
7693                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7694                 end
7695                 if Babel.debug then
7696                     print('.....', 'str')
7697                     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7698                 end
7699             end -- for
7700             node.remove(head, item)
7701         end -- if ''
7702         last_match = utf8.offset(w, sc+1+step)
7703         goto next
7704
7705     elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7706         d = node.new(7, 3) -- (disc, regular)
7707         d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7708         d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7709         d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7710         d.attr = item_base.attr
7711         if crep.pre == nil then -- TeXbook p96
7712             d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7713         else
7714             d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7715         end
7716         placeholder = '|'
7717         head, new = node.insert_before(head, item, d)
7718
7719     elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7720         -- ERROR
7721
7722     elseif crep and crep.penalty then
7723         d = node.new(14, 0) -- (penalty, userpenalty)

```

```

7724         d.attr = item_base.attr
7725         d.penalty = tovalue(crep.penalty)
7726         head, new = node.insert_before(head, item, d)
7727
7728     elseif crep and crep.space then
7729         -- 655360 = 10 pt = 10 * 65536 sp
7730         d = node.new(12, 13)      -- (glue, spaceskip)
7731         local quad = font.getfont(item_base.font).size or 655360
7732         node.setglue(d, tovalue(crep.space[1]) * quad,
7733                       tovalue(crep.space[2]) * quad,
7734                       tovalue(crep.space[3]) * quad)
7735         if mode == 0 then
7736             placeholder = ' '
7737         end
7738         head, new = node.insert_before(head, item, d)
7739
7740     elseif crep and crep.norule then
7741         -- 655360 = 10 pt = 10 * 65536 sp
7742         d = node.new(2, 3)        -- (rule, empty) = \no*rule
7743         local quad = font.getfont(item_base.font).size or 655360
7744         d.width  = tovalue(crep.norule[1]) * quad
7745         d.height = tovalue(crep.norule[2]) * quad
7746         d.depth  = tovalue(crep.norule[3]) * quad
7747         head, new = node.insert_before(head, item, d)
7748
7749     elseif crep and crep.spacefactor then
7750         d = node.new(12, 13)      -- (glue, spaceskip)
7751         local base_font = font.getfont(item_base.font)
7752         node.setglue(d,
7753                       tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7754                       tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7755                       tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7756         if mode == 0 then
7757             placeholder = ' '
7758         end
7759         head, new = node.insert_before(head, item, d)
7760
7761     elseif mode == 0 and crep and crep.space then
7762         -- ERROR
7763
7764     elseif crep and crep.kern then
7765         d = node.new(13, 1)      -- (kern, user)
7766         local quad = font.getfont(item_base.font).size or 655360
7767         d.attr = item_base.attr
7768         d.kern = tovalue(crep.kern) * quad
7769         head, new = node.insert_before(head, item, d)
7770
7771     elseif crep and crep.node then
7772         d = node.new(crep.node[1], crep.node[2])
7773         d.attr = item_base.attr
7774         head, new = node.insert_before(head, item, d)
7775
7776     end -- i.e., replacement cases
7777
7778     -- Shared by disc, space(factor), kern, node and penalty.
7779     if sc == 1 then
7780         word_head = head
7781     end
7782     if crep.insert then
7783         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7784         table.insert(w_nodes, sc, new)
7785         last = last + 1
7786     else

```

```

7787         w_nodes[sc] = d
7788         node.remove(head, item)
7789         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7790     end
7791
7792     last_match = utf8.offset(w, sc+1+step)
7793
7794     ::next::
7795
7796     end -- for each replacement
7797
7798     if Babel.show_transforms then texio.write_nl('> ' .. w) end
7799     if Babel.debug then
7800         print('.....', '/')
7801         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7802     end
7803
7804     if dummy_node then
7805         node.remove(head, dummy_node)
7806         dummy_node = nil
7807     end
7808
7809     end -- for match
7810
7811     end -- for patterns
7812
7813     ::next::
7814     word_head = nw
7815 end -- for substring
7816
7817 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7818 return head
7819 end
7820
7821 -- This table stores capture maps, numbered consecutively
7822 Babel.capture_maps = {}
7823
7824 function Babel.esc_hex_to_char(h)
7825     if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7826         tex.getcatcode(tonumber(h, 16)) ~= 12 then
7827         return string.format(["\Uchar%X"], tonumber(h,16))
7828     else
7829         return unicode.utf8.char(tonumber(h, 16))
7830     end
7831 end
7832
7833 -- The following functions belong to the next macro
7834 function Babel.capture_func(key, cap)
7835     local ret = "[" .. cap:gsub('{{([0-9])}}', ")]..m[%1]..["] .. "]"
7836     local cnt
7837     local u = unicode.utf8
7838     ret = u.gsub(ret, '{(%x%x%x%x+)}', '\x01\x04')
7839     ret, cnt = ret:gsub('{{([0-9])|([^\^|]+)|(.-)}', Babel.capture_func_map)
7840     ret = u.gsub(ret, '\x01(%x%x%x%x+)\x04', Babel.esc_hex_to_char)
7841     ret = ret:gsub("%[%[%]]%.", '')
7842     ret = ret:gsub("%.%[%[%]]%", '')
7843     return key .. [[=function(m) return ]] .. ret .. [[ end]]
7844 end
7845
7846 function Babel.capt_map(from, mapno)
7847     return Babel.capture_maps[mapno][from] or from
7848 end
7849

```

```

7850 -- Handle the {n|abc|ABC} syntax in captures
7851 function Babel.capture_func_map(capno, from, to)
7852     local u = unicode.utf8
7853     from = u.gsub(from, '\x01(%x%x%x%x+)\x04',
7854         function (n)
7855             return u.char(tonumber(n, 16))
7856         end)
7857     to = u.gsub(to, '\x01(%x%x%x%x+)\x04',
7858         function (n)
7859             return u.char(tonumber(n, 16))
7860         end)
7861     local froms = {}
7862     for s in string.utfcharacters(from) do
7863         table.insert(froms, s)
7864     end
7865     local cnt = 1
7866     table.insert(Babel.capture_maps, {})
7867     local mlen = table.getn(Babel.capture_maps)
7868     for s in string.utfcharacters(to) do
7869         Babel.capture_maps[mlen][froms[cnt]] = s
7870         cnt = cnt + 1
7871     end
7872     return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7873         (mlen) .. ").." .. "[["
7874 end
7875
7876 -- Create/Extend reversed sorted list of kashida weights:
7877 function Babel.capture_kashida(key, wt)
7878     wt = tonumber(wt)
7879     if Babel.kashida_wts then
7880         for p, q in ipairs(Babel.kashida_wts) do
7881             if wt == q then
7882                 break
7883             elseif wt > q then
7884                 table.insert(Babel.kashida_wts, p, wt)
7885                 break
7886             elseif table.getn(Babel.kashida_wts) == p then
7887                 table.insert(Babel.kashida_wts, wt)
7888             end
7889         end
7890     else
7891         Babel.kashida_wts = { wt }
7892     end
7893     return 'kashida = ' .. wt
7894 end
7895
7896 function Babel.capture_node(id, subtype)
7897     local sbt = 0
7898     for k, v in pairs(node.subtypes(id)) do
7899         if v == subtype then sbt = k end
7900     end
7901     return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7902 end
7903
7904 -- Experimental: applies prehyphenation transforms to a string (letters
7905 -- and spaces).
7906 function Babel.string_prehyphenation(str, locale)
7907     local n, head, last, res
7908     head = node.new(8, 0) -- dummy (hack just to start)
7909     last = head
7910     for s in string.utfvalues(str) do
7911         if s == 20 then
7912             n = node.new(12, 0)

```

```

7913     else
7914         n = node.new(29, 0)
7915         n.char = s
7916     end
7917     node.set_attribute(n, Babel.attr_locale, locale)
7918     last.next = n
7919     last = n
7920 end
7921 head = Babel.hyphenate_replace(head, 0)
7922 res = ''
7923 for n in node.traverse(head) do
7924     if n.id == 12 then
7925         res = res .. ' '
7926     elseif n.id == 29 then
7927         res = res .. unicode.utf8.char(n.char)
7928     end
7929 end
7930 tex.print(res)
7931 end
7932 </transforms>

```

10.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%

```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In babel the dir is set by a higher protocol based on the language/script, which in turn sets the correct dir (`<l>`, `<r>` or `<al>`).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don’t think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where `luatex` excels, because everything related to bidi writing is under our control.

```

7933 (*basic-r)
7934 Babel.bidi_enabled = true
7935

```



```

7936 require('babel-data-bidi.lua')
7937
7938 local characters = Babel.characters
7939 local ranges = Babel.ranges
7940
7941 local DIR = node.id("dir")
7942
7943 local function dir_mark(head, from, to, outer)
7944   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7945   local d = node.new(DIR)
7946   d.dir = '+' .. dir
7947   node.insert_before(head, from, d)
7948   d = node.new(DIR)
7949   d.dir = '-' .. dir
7950   node.insert_after(head, to, d)
7951 end
7952
7953 function Babel.bidi(head, ispar)
7954   local first_n, last_n          -- first and last char with nums
7955   local last_es                  -- an auxiliary 'last' used with nums
7956   local first_d, last_d          -- first and last char in L/R block
7957   local dir, dir_real

   Next also depends on script/lang (<al>/<r>). To be set by babel.tex.pardir is dangerous, could be
   (re)set but it should be changed only in vmode. There are two strong's – strong = l/al/r and
   strong_lr = l/r (there must be a better way):

7958   local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7959   local strong_lr = (strong == 'l') and 'l' or 'r'
7960   local outer = strong
7961
7962   local new_dir = false
7963   local first_dir = false
7964   local inmath = false
7965
7966   local last_lr
7967
7968   local type_n = ''
7969
7970   for item in node.traverse(head) do
7971
7972     -- three cases: glyph, dir, otherwise
7973     if item.id == node.id'glyph'
7974       or (item.id == 7 and item.subtype == 2) then
7975
7976       local itemchar
7977       if item.id == 7 and item.subtype == 2 then
7978         itemchar = item.replace.char
7979       else
7980         itemchar = item.char
7981       end
7982       local chardata = characters[itemchar]
7983       dir = chardata and chardata.d or nil
7984       if not dir then
7985         for nn, et in ipairs(ranges) do
7986           if itemchar < et[1] then
7987             break
7988           elseif itemchar <= et[2] then
7989             dir = et[3]
7990             break
7991           end
7992         end
7993       end
7994       dir = dir or 'l'

```

```
7995     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end
```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```
7996     if new_dir then
7997         attr_dir = 0
7998         for at in node.traverse(item.attr) do
7999             if at.number == Babel.attr_dir then
8000                 attr_dir = at.value & 0x3
8001             end
8002         end
8003         if attr_dir == 1 then
8004             strong = 'r'
8005         elseif attr_dir == 2 then
8006             strong = 'al'
8007         else
8008             strong = 'l'
8009         end
8010         strong_lr = (strong == 'l') and 'l' or 'r'
8011         outer = strong_lr
8012         new_dir = false
8013     end
8014
8015     if dir == 'nsm' then dir = strong end          -- W1
```

Numbers. The dual <al>/<r> system for R is somewhat cumbersome.

```
8016     dir_real = dir          -- We need dir_real to set strong below
8017     if dir == 'al' then dir = 'r' end -- W3
```

By W2, there are no <en> <et> <es> if strong == <al>, only <an>. Therefore, there are not <et en> nor <en et>, W5 can be ignored, and W6 applied:

```
8018     if strong == 'al' then
8019         if dir == 'en' then dir = 'an' end          -- W2
8020         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8021         strong_lr = 'r'          -- W3
8022     end
```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```
8023     elseif item.id == node.id'dir' and not inmath then
8024         new_dir = true
8025         dir = nil
8026     elseif item.id == node.id'math' then
8027         inmath = (item.subtype == 0)
8028     else
8029         dir = nil          -- Not a char
8030     end
```

Numbers in R mode. A sequence of <en>, <et>, <an>, <es> and <cs> is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only <an> is relevant if <al>.

```
8031     if dir == 'en' or dir == 'an' or dir == 'et' then
8032         if dir ~= 'et' then
8033             type_n = dir
8034         end
8035         first_n = first_n or item
8036         last_n = last_es or item
8037         last_es = nil
8038     elseif dir == 'es' and last_n then -- W3+W6
8039         last_es = item
```

```

8040 elseif dir == 'cs' then          -- it's right - do nothing
8041 elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8042   if strong_lr == 'r' and type_n ~= '' then
8043     dir_mark(head, first_n, last_n, 'r')
8044   elseif strong_lr == 'l' and first_d and type_n == 'an' then
8045     dir_mark(head, first_n, last_n, 'r')
8046     dir_mark(head, first_d, last_d, outer)
8047     first_d, last_d = nil, nil
8048   elseif strong_lr == 'l' and type_n ~= '' then
8049     last_d = last_n
8050   end
8051   type_n = ''
8052   first_n, last_n = nil, nil
8053 end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8054 if dir == 'l' or dir == 'r' then
8055   if dir ~= outer then
8056     first_d = first_d or item
8057     last_d = item
8058   elseif first_d and dir ~= strong_lr then
8059     dir_mark(head, first_d, last_d, outer)
8060     first_d, last_d = nil, nil
8061   end
8062 end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it's clearly <r> and <l>, resptly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn't hurt.

```

8063 if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8064   item.char = characters[item.char] and
8065     characters[item.char].m or item.char
8066 elseif (dir or new_dir) and last_lr ~= item then
8067   local mir = outer .. strong_lr .. (dir or outer)
8068   if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8069     for ch in node.traverse(node.next(last_lr)) do
8070       if ch == item then break end
8071       if ch.id == node.id'glyph' and characters[ch.char] then
8072         ch.char = characters[ch.char].m or ch.char
8073       end
8074     end
8075   end
8076 end

```

Save some values for the next iteration. If the current node is 'dir', open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8077 if dir == 'l' or dir == 'r' then
8078   last_lr = item
8079   strong = dir_real          -- Don't search back - best save now
8080   strong_lr = (strong == 'l') and 'l' or 'r'
8081 elseif new_dir then
8082   last_lr = nil
8083 end
8084 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8085 if last_lr and outer == 'r' then
8086   for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8087     if characters[ch.char] then

```

```

8088         ch.char = characters[ch.char].m or ch.char
8089     end
8090 end
8091 end
8092 if first_n then
8093     dir_mark(head, first_n, last_n, outer)
8094 end
8095 if first_d then
8096     dir_mark(head, first_d, last_d, outer)
8097 end

    In boxes, the dir node could be added before the original head, so the actual head is the previous
    node.

8098 return node.prev(head) or head
8099 end
8100 </basic-r>

```

And here the Lua code for bidi=basic:

```

8101 (*basic)
8102 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8103
8104 Babel.fontmap = Babel.fontmap or {}
8105 Babel.fontmap[0] = {}          -- l
8106 Babel.fontmap[1] = {}          -- r
8107 Babel.fontmap[2] = {}          -- al/an
8108
8109 -- To cancel mirroring. Also OML, OMS, U?
8110 Babel.symbol_fonts = Babel.symbol_fonts or {}
8111 Babel.symbol_fonts[font.id('tenln')] = true
8112 Babel.symbol_fonts[font.id('tenlnw')] = true
8113 Babel.symbol_fonts[font.id('tencirc')] = true
8114 Babel.symbol_fonts[font.id('tencircw')] = true
8115
8116 Babel.bidi_enabled = true
8117 Babel.mirroring_enabled = true
8118
8119 require('babel-data-bidi.lua')
8120
8121 local characters = Babel.characters
8122 local ranges = Babel.ranges
8123
8124 local DIR = node.id('dir')
8125 local GLYPH = node.id('glyph')
8126
8127 local function insert_implicit(head, state, outer)
8128     local new_state = state
8129     if state.sim and state.eim and state.sim ~= state.eim then
8130         dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8131         local d = node.new(DIR)
8132         d.dir = '+' .. dir
8133         node.insert_before(head, state.sim, d)
8134         local d = node.new(DIR)
8135         d.dir = '-' .. dir
8136         node.insert_after(head, state.eim, d)
8137     end
8138     new_state.sim, new_state.eim = nil, nil
8139     return head, new_state
8140 end
8141
8142 local function insert_numeric(head, state)
8143     local new
8144     local new_state = state
8145     if state.san and state.ean and state.san ~= state.ean then
8146         local d = node.new(DIR)

```

```

8147     d.dir = '+TLT'
8148     _, new = node.insert_before(head, state.san, d)
8149     if state.san == state.sim then state.sim = new end
8150     local d = node.new(DIR)
8151     d.dir = '-TLT'
8152     _, new = node.insert_after(head, state.ean, d)
8153     if state.ean == state.eim then state.eim = new end
8154 end
8155 new_state.san, new_state.ean = nil, nil
8156 return head, new_state
8157 end
8158
8159 local function glyph_not_symbol_font(node)
8160 if node.id == GLYPH then
8161     return not Babel.symbol_fonts[node.font]
8162 else
8163     return false
8164 end
8165 end
8166
8167 -- TODO - \hbox with an explicit dir can lead to wrong results
8168 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8169 -- was made to improve the situation, but the problem is the 3-dir
8170 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8171 -- well.
8172
8173 function Babel.bidi(head, ispar, hdir)
8174     local d -- d is used mainly for computations in a loop
8175     local prev_d = ''
8176     local new_d = false
8177
8178     local nodes = {}
8179     local outer_first = nil
8180     local inmath = false
8181
8182     local glue_d = nil
8183     local glue_i = nil
8184
8185     local has_en = false
8186     local first_et = nil
8187
8188     local has_hyperlink = false
8189
8190     local ATDIR = Babel.attr_dir
8191     local attr_d, temp
8192     local locale_d
8193
8194     local save_outer
8195     local locale_d = node.get_attribute(head, ATDIR)
8196     if locale_d then
8197         locale_d = locale_d & 0x3
8198         save_outer = (locale_d == 0 and 'l') or
8199                     (locale_d == 1 and 'r') or
8200                     (locale_d == 2 and 'al')
8201     elseif ispar then -- Or error? Shouldn't happen
8202         -- when the callback is called, we are just _after_ the box,
8203         -- and the textdir is that of the surrounding text
8204         save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8205     else -- Empty box
8206         save_outer = ('TRT' == hdir) and 'r' or 'l'
8207     end
8208     local outer = save_outer
8209     local last = outer

```

```

8210 -- 'al' is only taken into account in the first, current loop
8211 if save_outer == 'al' then save_outer = 'r' end
8212
8213 local fontmap = Babel.fontmap
8214
8215 for item in node.traverse(head) do
8216
8217     -- Mask: DxxxPPTT (Done, Paddir [0-2], Textdir [0-2])
8218     locale_d = node.get_attribute(item, ATDIR)
8219     node.set_attribute(item, ATDIR, 0x80)
8220
8221     -- In what follows, #node is the last (previous) node, because the
8222     -- current one is not added until we start processing the neutrals.
8223     -- three cases: glyph, dir, otherwise
8224     if glyph_not_symbol_font(item)
8225         or (item.id == 7 and item.subtype == 2) then
8226
8227         if inmath then goto nextnode end
8228
8229         if locale_d == 0x80 then goto nextnode end
8230         locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8231
8232         local d_font = nil
8233         local item_r
8234         if item.id == 7 and item.subtype == 2 then
8235             item_r = item.replace -- automatic discs have just 1 glyph
8236         else
8237             item_r = item
8238         end
8239
8240         local chardata = characters[item_r.char]
8241         d = chardata and chardata.d or nil
8242         if not d or d == 'nsm' then
8243             for nn, et in ipairs(ranges) do
8244                 if item_r.char < et[1] then
8245                     break
8246                 elseif item_r.char <= et[2] then
8247                     if not d then d = et[3]
8248                     elseif d == 'nsm' then d_font = et[3]
8249                     end
8250                     break
8251                 end
8252             end
8253         end
8254         d = d or 'l'
8255
8256         -- A short 'pause' in bidi for mapfont
8257         -- %%% TODO0. move if fontmap here
8258         d_font = d_font or d
8259         d_font = (d_font == 'l' and 0) or
8260             (d_font == 'nsm' and 0) or
8261             (d_font == 'r' and 1) or
8262             (d_font == 'al' and 2) or
8263             (d_font == 'an' and 2) or nil
8264         if d_font and fontmap and fontmap[d_font][item_r.font] then
8265             item_r.font = fontmap[d_font][item_r.font]
8266         end
8267
8268         if new_d then
8269             table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8270             if inmath then
8271                 attr_d = 0
8272             else

```

```

8273         attr_d = locale_d & 0x3
8274     end
8275     if attr_d == 1 then
8276         outer_first = 'r'
8277         last = 'r'
8278     elseif attr_d == 2 then
8279         outer_first = 'r'
8280         last = 'al'
8281     else
8282         outer_first = 'l'
8283         last = 'l'
8284     end
8285     outer = last
8286     has_en = false
8287     first_et = nil
8288     new_d = false
8289 end
8290
8291 if glue_d then
8292     if (d == 'l' and 'l' or 'r') ~= glue_d then
8293         table.insert(nodes, {glue_i, 'on', nil})
8294     end
8295     glue_d = nil
8296     glue_i = nil
8297 end
8298
8299 elseif item.id == DIR then
8300     if inmath then goto nextnode end
8301     d = nil
8302     new_d = true
8303
8304 elseif item.id == node.id'glue' and item.subtype == 13 then
8305     if inmath then goto nextnode end
8306     glue_d = d
8307     glue_i = item
8308     d = nil
8309
8310 elseif item.id == node.id'math' then
8311     if item.subtype == 0 then -- mathon
8312         inmath = true
8313         d = 'on'
8314         table.insert(nodes, {item, 'on', outer_first})
8315         outer_first = nil
8316         goto nextnode
8317     else -- mathoff
8318         inmath = false
8319     end
8320
8321 elseif item.id == 8 and item.subtype == 19 then
8322     has_hyperlink = true
8323
8324 else
8325     d = nil
8326 end
8327
8328 -- AL <= EN/ET/ES      -- W2 + W3 + W6
8329 if last == 'al' and d == 'en' then
8330     d = 'an'            -- W3
8331 elseif last == 'al' and (d == 'et' or d == 'es') then
8332     d = 'on'            -- W6
8333 end
8334
8335 -- EN + CS/ES + EN      -- W4

```

```

8336 if d == 'en' and #nodes >= 2 then
8337   if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8338     and nodes[#nodes-1][2] == 'en' then
8339     nodes[#nodes][2] = 'en'
8340   end
8341 end
8342
8343 -- AN + CS + AN      -- W4 too, because uax9 mixes both cases
8344 if d == 'an' and #nodes >= 2 then
8345   if (nodes[#nodes][2] == 'cs')
8346     and nodes[#nodes-1][2] == 'an' then
8347     nodes[#nodes][2] = 'an'
8348   end
8349 end
8350
8351 -- ET/EN            -- W5 + W7->l / W6->on
8352 if d == 'et' then
8353   first_et = first_et or (#nodes + 1)
8354 elseif d == 'en' then
8355   has_en = true
8356   first_et = first_et or (#nodes + 1)
8357 elseif first_et then -- d may be nil here !
8358   if has_en then
8359     if last == 'l' then
8360       temp = 'l' -- W7
8361     else
8362       temp = 'en' -- W5
8363     end
8364   else
8365     temp = 'on' -- W6
8366   end
8367   for e = first_et, #nodes do
8368     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8369   end
8370   first_et = nil
8371   has_en = false
8372 end
8373
8374 -- Force mathdir in math if ON (currently works as expected only
8375 -- with 'l')
8376
8377 if inmath and d == 'on' then
8378   d = ('TRT' == tex.mathdir) and 'r' or 'l'
8379 end
8380
8381 if d then
8382   if d == 'al' then
8383     d = 'r'
8384     last = 'al'
8385   elseif d == 'l' or d == 'r' then
8386     last = d
8387   end
8388   prev_d = d
8389   table.insert(nodes, {item, d, outer_first})
8390 end
8391
8392 outer_first = nil
8393
8394 ::nextnode::
8395
8396 end -- for each node
8397
8398 -- TODO -- repeated here in case EN/ET is the last node. Find a

```



```

8399 -- better way of doing things:
8400 if first_et then      -- dir may be nil here !
8401   if has_en then
8402     if last == 'l' then
8403       temp = 'l'      -- W7
8404     else
8405       temp = 'en'     -- W5
8406     end
8407   else
8408     temp = 'on'       -- W6
8409   end
8410   for e = first_et, #nodes do
8411     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8412   end
8413 end
8414
8415 -- dummy node, to close things
8416 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8417
8418 ----- NEUTRAL -----
8419
8420 outer = save_outer
8421 last = outer
8422
8423 local first_on = nil
8424
8425 for q = 1, #nodes do
8426   local item
8427
8428   local outer_first = nodes[q][3]
8429   outer = outer_first or outer
8430   last = outer_first or last
8431
8432   local d = nodes[q][2]
8433   if d == 'an' or d == 'en' then d = 'r' end
8434   if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8435
8436   if d == 'on' then
8437     first_on = first_on or q
8438   elseif first_on then
8439     if last == d then
8440       temp = d
8441     else
8442       temp = outer
8443     end
8444     for r = first_on, q - 1 do
8445       nodes[r][2] = temp
8446       item = nodes[r][1] -- MIRRORING
8447       if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8448         and temp == 'r' and characters[item.char] then
8449         local font_mode = ''
8450         if item.font > 0 and font.fonts[item.font].properties then
8451           font_mode = font.fonts[item.font].properties.mode
8452         end
8453         if font_mode ~= 'harf' and font_mode ~= 'plug' then
8454           item.char = characters[item.char].m or item.char
8455         end
8456       end
8457     end
8458     first_on = nil
8459   end
8460
8461   if d == 'r' or d == 'l' then last = d end

```

```

8462 end
8463
8464 ----- IMPLICIT, REORDER -----
8465
8466 outer = save_outer
8467 last = outer
8468
8469 local state = {}
8470 state.has_r = false
8471
8472 for q = 1, #nodes do
8473
8474     local item = nodes[q][1]
8475
8476     outer = nodes[q][3] or outer
8477
8478     local d = nodes[q][2]
8479
8480     if d == 'nsm' then d = last end          -- W1
8481     if d == 'en' then d = 'an' end
8482     local isdir = (d == 'r' or d == 'l')
8483
8484     if outer == 'l' and d == 'an' then
8485         state.san = state.san or item
8486         state.ean = item
8487     elseif state.san then
8488         head, state = insert_numeric(head, state)
8489     end
8490
8491     if outer == 'l' then
8492         if d == 'an' or d == 'r' then      -- im -> implicit
8493             if d == 'r' then state.has_r = true end
8494             state.sim = state.sim or item
8495             state.eim = item
8496         elseif d == 'l' and state.sim and state.has_r then
8497             head, state = insert_implicit(head, state, outer)
8498         elseif d == 'l' then
8499             state.sim, state.eim, state.has_r = nil, nil, false
8500         end
8501     else
8502         if d == 'an' or d == 'l' then
8503             if nodes[q][3] then -- nil except after an explicit dir
8504                 state.sim = item -- so we move sim 'inside' the group
8505             else
8506                 state.sim = state.sim or item
8507             end
8508             state.eim = item
8509         elseif d == 'r' and state.sim then
8510             head, state = insert_implicit(head, state, outer)
8511         elseif d == 'r' then
8512             state.sim, state.eim = nil, nil
8513         end
8514     end
8515
8516     if isdir then
8517         last = d          -- Don't search back - best save now
8518     elseif d == 'on' and state.san then
8519         state.san = state.san or item
8520         state.ean = item
8521     end
8522
8523 end
8524

```

```

8525 head = node.prev(head) or head
8526 % \end{macrocode}
8527 %
8528 % Now direction nodes has been distributed with relation to characters
8529 % and spaces, we need to take into account \TeX-specific elements in
8530 % the node list, to move them at an appropriate place. Firstly, with
8531 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8532 % that the latter are still discardable.
8533 %
8534 % \begin{macrocode}
8535 --- FIXES ---
8536 if has_hyperlink then
8537   local flag, linking = 0, 0
8538   for item in node.traverse(head) do
8539     if item.id == DIR then
8540       if item.dir == '+TRT' or item.dir == '+TLT' then
8541         flag = flag + 1
8542       elseif item.dir == '-TRT' or item.dir == '-TLT' then
8543         flag = flag - 1
8544       end
8545       elseif item.id == 8 and item.subtype == 19 then
8546         linking = flag
8547       elseif item.id == 8 and item.subtype == 20 then
8548         if linking > 0 then
8549           if item.prev.id == DIR and
8550             (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8551             d = node.new(DIR)
8552             d.dir = item.prev.dir
8553             node.remove(head, item.prev)
8554             node.insert_after(head, item, d)
8555           end
8556         end
8557         linking = 0
8558       end
8559     end
8560   end
8561
8562   for item in node.traverse_id(10, head) do
8563     local p = item
8564     local flag = false
8565     while p.prev and p.prev.id == 14 do
8566       flag = true
8567       p = p.prev
8568     end
8569     if flag then
8570       node.insert_before(head, p, node.copy(item))
8571       node.remove(head, item)
8572     end
8573   end
8574
8575   return head
8576 end
8577
8577 function Babel.unset_atdir(head)
8578   local ATDIR = Babel.attr_dir
8579   for item in node.traverse(head) do
8580     node.set_attribute(item, ATDIR, 0x80)
8581   end
8582   return head
8583 end
8584 /basic

```

11. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```
% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%
```

For the meaning of these codes, see the Unicode standard.

12. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```
8585 (*nil)
8586 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8587 \LdfInit{nil}{datenil}
```

When this file is read as an option, i.e., by the `\usepackage` command, nil could be an ‘unknown’ language in which case we have to make it known.

```
8588 \ifx\l@nil\undefined
8589 \newlanguage\l@nil
8590 \@namedef{bbl@hyphendata@the\l@nil}{}}}% Remove warning
8591 \let\bbl@elt\relax
8592 \edef\bbl@languages{% Add it to the list of languages
8593 \bbl@languages\bbl@elt{nil}{the\l@nil}{}}
8594 \fi
```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```
8595 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}
```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

`\captionnil`

`\datenil`

```
8596 \let\captionnil\@empty
8597 \let\datenil\@empty
```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```
8598 \def\bbl@inidata@nil{%
8599 \bbl@elt{identification}{tag.ini}{und}%
8600 \bbl@elt{identification}{load.level}{0}%
8601 \bbl@elt{identification}{charset}{utf8}%
8602 \bbl@elt{identification}{version}{1.0}%
8603 \bbl@elt{identification}{date}{2022-05-16}%
8604 \bbl@elt{identification}{name.local}{nil}%
8605 \bbl@elt{identification}{name.english}{nil}%
8606 \bbl@elt{identification}{name.babel}{nil}%
8607 \bbl@elt{identification}{tag.bcp47}{und}%
8608 \bbl@elt{identification}{language.tag.bcp47}{und}%
8609 \bbl@elt{identification}{tag.opentype}{dfLT}%
8610 \bbl@elt{identification}{script.name}{Latin}%
8611 \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8612 \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8613 \bbl@elt{identification}{level}{1}%
```

```

8614 \bbl@elt{identification}{encodings}{}%
8615 \bbl@elt{identification}{derivate}{no}}
8616 \@namedef{bbl@tbc@nil}{und}
8617 \@namedef{bbl@lbc@nil}{und}
8618 \@namedef{bbl@casing@nil}{und}
8619 \@namedef{bbl@lotf@nil}{dflt}
8620 \@namedef{bbl@elname@nil}{nil}
8621 \@namedef{bbl@lname@nil}{nil}
8622 \@namedef{bbl@esname@nil}{Latin}
8623 \@namedef{bbl@sname@nil}{Latin}
8624 \@namedef{bbl@sbc@nil}{Latn}
8625 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8626 \ldf@finish{nil}
8627 </nil>

```

13. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8628 <<*Compute Julian day>> ≡
8629 \def\bbl@fpmo#1#2{(#1-#2*floor(#1/#2))}
8630 \def\bbl@cs@gregleap#1{%
8631   (\bbl@fpmo{#1}{4} == 0) &&
8632   (!((\bbl@fpmo{#1}{100} == 0) && (\bbl@fpmo{#1}{400} != 0)))}
8633 \def\bbl@cs@jd#1#2#3{% year, month, day
8634   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8635     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8636     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8637     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3 } }
8638 <</Compute Julian day>>

```

13.1. Islamic

The code for the Civil calendar is based on it, too.

```

8639 <*ca-islamic>
8640 <@Compute Julian day@>
8641 % == islamic (default)
8642 % Not yet implemented
8643 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

```

The Civil calendar.

```

8644 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8645   ((#3 + ceil(29.5 * (#2 - 1)) +
8646     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8647     1948439.5) - 1) }
8648 \@namedef{bbl@ca@islamic-civil++}{\bbl@ca@islamicvl@x{+2}}
8649 \@namedef{bbl@ca@islamic-civil+}{\bbl@ca@islamicvl@x{+1}}
8650 \@namedef{bbl@ca@islamic-civil}{\bbl@ca@islamicvl@x{}}
8651 \@namedef{bbl@ca@islamic-civil-}{\bbl@ca@islamicvl@x{-1}}
8652 \@namedef{bbl@ca@islamic-civil--}{\bbl@ca@islamicvl@x{-2}}
8653 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8654   \edef\bbl@tempa{%
8655     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1 } }%
8656   \edef#5{%
8657     \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) } }%
8658   \edef#6{\fpeval{
8659     min(12,ceil((\bbl@tempa-(29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) } }%

```

```
8660 \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}
```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```
8661 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8662 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8663 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8664 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8665 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8666 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8667 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8668 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8669 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8670 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8671 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8672 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8673 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8674 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8675 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8676 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8677 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8678 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8679 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8680 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8681 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8682 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8683 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8684 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8685 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8686 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8687 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8688 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8689 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8690 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8691 65401,65431,65460,65490,65520}
8692 \@namedef{bbl@ca@islamic-umalqura+}{\bbl@ca@islamcuqr@x{+1}}
8693 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8694 \@namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{0}}
8695 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8696 \ifnum#2>2014 \ifnum#2<2038
8697 \bbl@afterfi\expandafter\@gobble
8698 \fi\fi
8699 {\bbl@error{year-out-range}{2014-2038}}{}}%
8700 \edef\bbl@tempd{\fpeval{ % (Julian) day
8701 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8702 \count@\@ne
8703 \bbl@foreach\bbl@cs@umalqura@data{%
8704 \advance\count@\@ne
8705 \ifnum##1>\bbl@tempd\else
8706 \edef\bbl@tempe{\the\count@}%
8707 \edef\bbl@tempb{##1}%
8708 \fi}%
8709 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8710 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1) / 12) }}% annus
8711 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8712 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8713 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8714 \bbl@add\bbl@precalendar{%
8715 \bbl@replace\bbl@ld@calendar{-civil}}}%
8716 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8717 \bbl@replace\bbl@ld@calendar{+}}}%

```

```

8718 \bbl@replace\bbl@ld@calendar{-}{}}
8719 </ca-islamic>

```

13.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaptations by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```

8720 <*ca-hebrew>
8721 \newcount\bbl@cntcommon
8722 \def\bbl@remainder#1#2#3{%
8723   #3=#1\relax
8724   \divide #3 by #2\relax
8725   \multiply #3 by -#2\relax
8726   \advance #3 by #1\relax}%
8727 \newif\ifbbl@divisible
8728 \def\bbl@checkifdivisible#1#2{%
8729   {\countdef\tmp=0
8730    \bbl@remainder{#1}{#2}{\tmp}%
8731    \ifnum \tmp=0
8732      \global\bbl@divisibletrue
8733    \else
8734      \global\bbl@divisiblefalse
8735    \fi}}
8736 \newif\ifbbl@gregleap
8737 \def\bbl@ifgregleap#1{%
8738   \bbl@checkifdivisible{#1}{4}%
8739   \ifbbl@divisible
8740     \bbl@checkifdivisible{#1}{100}%
8741     \ifbbl@divisible
8742       \bbl@checkifdivisible{#1}{400}%
8743       \ifbbl@divisible
8744         \bbl@gregleaptrue
8745       \else
8746         \bbl@gregleapfalse
8747       \fi
8748     \else
8749       \bbl@gregleaptrue
8750     \fi
8751   \else
8752     \bbl@gregleapfalse
8753   \fi
8754   \ifbbl@gregleap}
8755 \def\bbl@gregdayspriormonths#1#2#3{%
8756   {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8757     181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8758   \bbl@ifgregleap{#2}%
8759   \ifnum #1 > 2
8760     \advance #3 by 1
8761   \fi
8762   \fi
8763   \global\bbl@cntcommon=#3}%
8764   #3=\bbl@cntcommon}
8765 \def\bbl@gregdaysprioryears#1#2{%
8766   {\countdef\tmpc=4
8767    \countdef\tmpb=2
8768    \tmpb=#1\relax
8769    \advance \tmpb by -1
8770    \tmpc=\tmpb
8771    \multiply \tmpc by 365
8772    #2=\tmpc
8773    \tmpc=\tmpb
8774    \divide \tmpc by 4

```

```

8775 \advance #2 by \tmpc
8776 \tmpc=\tmpb
8777 \divide \tmpc by 100
8778 \advance #2 by -\tmpc
8779 \tmpc=\tmpb
8780 \divide \tmpc by 400
8781 \advance #2 by \tmpc
8782 \global\bbl@cntcommon=#2\relax}%
8783 #2=\bbl@cntcommon}
8784 \def\bbl@absfromgreg#1#2#3#4{%
8785 {\countdef\tmpd=0
8786 #4=#1\relax
8787 \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8788 \advance #4 by \tmpd
8789 \bbl@gregdaysprioryears{#3}{\tmpd}%
8790 \advance #4 by \tmpd
8791 \global\bbl@cntcommon=#4\relax}%
8792 #4=\bbl@cntcommon}
8793 \newif\ifbbl@hebrleap
8794 \def\bbl@checkleaphebryear#1{%
8795 {\countdef\tmpa=0
8796 \countdef\tmpb=1
8797 \tmpa=#1\relax
8798 \multiply \tmpa by 7
8799 \advance \tmpa by 1
8800 \bbl@remainder{\tmpa}{19}{\tmpb}%
8801 \ifnum \tmpb < 7
8802 \global\bbl@hebrleaptrue
8803 \else
8804 \global\bbl@hebrleapfalse
8805 \fi}}
8806 \def\bbl@hebreleapsedmonths#1#2{%
8807 {\countdef\tmpa=0
8808 \countdef\tmpb=1
8809 \countdef\tmpc=2
8810 \tmpa=#1\relax
8811 \advance \tmpa by -1
8812 #2=\tmpa
8813 \divide #2 by 19
8814 \multiply #2 by 235
8815 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8816 \tmpc=\tmpb
8817 \multiply \tmpb by 12
8818 \advance #2 by \tmpb
8819 \multiply \tmpc by 7
8820 \advance \tmpc by 1
8821 \divide \tmpc by 19
8822 \advance #2 by \tmpc
8823 \global\bbl@cntcommon=#2}%
8824 #2=\bbl@cntcommon}
8825 \def\bbl@hebreleapseddays#1#2{%
8826 {\countdef\tmpa=0
8827 \countdef\tmpb=1
8828 \countdef\tmpc=2
8829 \bbl@hebreleapsedmonths{#1}{#2}%
8830 \tmpa=#2\relax
8831 \multiply \tmpa by 13753
8832 \advance \tmpa by 5604
8833 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8834 \divide \tmpa by 25920
8835 \multiply #2 by 29
8836 \advance #2 by 1
8837 \advance #2 by \tmpa

```



```

8838 \bbl@remainder{#2}{7}{\tmpa}%
8839 \ifnum \tmpc < 19440
8840 \ifnum \tmpc < 9924
8841 \else
8842 \ifnum \tmpa=2
8843 \bbl@checkleaphebyear{#1}% of a common year
8844 \ifbbl@hebrleap
8845 \else
8846 \advance #2 by 1
8847 \fi
8848 \fi
8849 \fi
8850 \ifnum \tmpc < 16789
8851 \else
8852 \ifnum \tmpa=1
8853 \advance #1 by -1
8854 \bbl@checkleaphebyear{#1}% at the end of leap year
8855 \ifbbl@hebrleap
8856 \advance #2 by 1
8857 \fi
8858 \fi
8859 \fi
8860 \else
8861 \advance #2 by 1
8862 \fi
8863 \bbl@remainder{#2}{7}{\tmpa}%
8864 \ifnum \tmpa=0
8865 \advance #2 by 1
8866 \else
8867 \ifnum \tmpa=3
8868 \advance #2 by 1
8869 \else
8870 \ifnum \tmpa=5
8871 \advance #2 by 1
8872 \fi
8873 \fi
8874 \fi
8875 \global\bbl@cntcommon=#2\relax}%
8876 #2=\bbl@cntcommon}
8877 \def\bbl@daysinhebyear#1#2{%
8878 {\countdef\tmpe=12
8879 \bbl@hebreleaseddays{#1}{\tmpe}%
8880 \advance #1 by 1
8881 \bbl@hebreleaseddays{#1}{#2}%
8882 \advance #2 by -\tmpe
8883 \global\bbl@cntcommon=#2}%
8884 #2=\bbl@cntcommon}
8885 \def\bbl@hebrdayspriormonths#1#2#3{%
8886 {\countdef\tmpf= 14
8887 #3=\ifcase #1
8888 0 \or
8889 0 \or
8890 30 \or
8891 59 \or
8892 89 \or
8893 118 \or
8894 148 \or
8895 148 \or
8896 177 \or
8897 207 \or
8898 236 \or
8899 266 \or
8900 295 \or

```

```

8901         325 \or
8902         400
8903     \fi
8904     \bbl@checkleaphebrewyear{#2}%
8905     \ifbbl@hebrleap
8906         \ifnum #1 > 6
8907             \advance #3 by 30
8908         \fi
8909     \fi
8910     \bbl@daysinhebrewyear{#2}{\tmpf}%
8911     \ifnum #1 > 3
8912         \ifnum \tmpf=353
8913             \advance #3 by -1
8914         \fi
8915         \ifnum \tmpf=383
8916             \advance #3 by -1
8917         \fi
8918     \fi
8919     \ifnum #1 > 2
8920         \ifnum \tmpf=355
8921             \advance #3 by 1
8922         \fi
8923         \ifnum \tmpf=385
8924             \advance #3 by 1
8925         \fi
8926     \fi
8927     \global\bbl@cntcommon=#3\relax}%
8928     #3=\bbl@cntcommon}
8929 \def\bbl@absfromhebr#1#2#3#4{%
8930     {#4=#1\relax
8931     \bbl@hebrdayspriormonths{#2}{#3}{#1}%
8932     \advance #4 by #1\relax
8933     \bbl@hebreleapseddays{#3}{#1}%
8934     \advance #4 by #1\relax
8935     \advance #4 by -1373429
8936     \global\bbl@cntcommon=#4\relax}%
8937     #4=\bbl@cntcommon}
8938 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
8939     {\countdef\tmpx= 17
8940     \countdef\tmpy= 18
8941     \countdef\tmpz= 19
8942     #6=#3\relax
8943     \global\advance #6 by 3761
8944     \bbl@absfromgreg{#1}{#2}{#3}{#4}%
8945     \tmpz=1 \tmpy=1
8946     \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8947     \ifnum \tmpx > #4\relax
8948         \global\advance #6 by -1
8949         \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8950     \fi
8951     \advance #4 by -\tmpx
8952     \advance #4 by 1
8953     #5=#4\relax
8954     \divide #5 by 30
8955     \loop
8956         \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
8957         \ifnum \tmpx < #4\relax
8958             \advance #5 by 1
8959             \tmpy=\tmpx
8960         \repeat
8961         \global\advance #5 by -1
8962         \global\advance #4 by -\tmpy}}
8963 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebrewyear

```

```

8964 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
8965 \def\bbl@ca@hebrew#1-#2-#3\@@#4#5#6{%
8966 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
8967 \bbl@hebrfromgreg
8968 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
8969 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebyear}%
8970 \edef#4{\the\bbl@hebyear}%
8971 \edef#5{\the\bbl@hebrmonth}%
8972 \edef#6{\the\bbl@hebrday}}
8973 </ca-hebrew>

```

13.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

8974 <*ca-persian>
8975 <@Compute Julian day@>
8976 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
8977 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
8978 \def\bbl@ca@persian#1-#2-#3\@@#4#5#6{%
8979 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
8980 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
8981 \bbl@afterfi\expandafter\@gobble
8982 \fi\fi
8983 {\bbl@error{year-out-range}{2013-2050}{}}}%
8984 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8985 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8986 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
8987 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
8988 \ifnum\bbl@tempc<\bbl@tempb
8989 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
8990 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8991 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8992 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
8993 \fi
8994 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
8995 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
8996 \edef#5{\fpeval{% set Jalali month
8997 (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
8998 \edef#6{\fpeval{% set Jalali day
8999 (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : (((#5 - 1) * 30) + 6))}}}%
9000 </ca-persian>

```

13.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

9001 <*ca-coptic>
9002 <@Compute Julian day@>
9003 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
9004 \edef\bbl@tempd{\fpeval{\floor{\bbl@cs@jd{#1}{#2}{#3}} + 0.5}}%
9005 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
9006 \edef#4{\fpeval{%
9007 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9008 \edef\bbl@tempc{\fpeval{%
9009 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
9010 \edef#5{\fpeval{\floor{\bbl@tempc / 30} + 1}}%
9011 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9012 </ca-coptic>

```

```

9013 < *ca-ethiopic>
9014 <@Compute Julian day@>
9015 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%
9016 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9017 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
9018 \edef#4{\fpeval{%
9019 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9020 \edef\bbl@tempc{\fpeval{%
9021 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
9022 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9023 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9024 </ca-ethiopic>

```

13.5. Julian

Based on [ReinDersh].

```

9025 < *ca-julian>
9026 <@Compute Julian day@>
9027 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
9028 \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9029 \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9030 \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9031 \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9032 \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9033 \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9034 \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9035 \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}%
9036 </ca-julian>

```

13.6. Buddhist

That's very simple.

```

9037 < *ca-buddhist>
9038 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9039 \edef#4{\number\numexpr#1+543\relax}%
9040 \edef#5{#2}%
9041 \edef#6{#3}%
9042 </ca-buddhist>
9043 %
9044 % \subsection{Chinese}
9045 %
9046 % Brute force, with the Julian day of first day of each month. The
9047 % table has been computed with the help of \textsf{python-lunardate} by
9048 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9049 % is 2015-2044.
9050 %
9051 % \begin{macrocode}
9052 < *ca-chinese>
9053 \ExplSyntaxOn
9054 <@Compute Julian day@>
9055 \def\bbl@ca@chinese#1-#2-#3\@@#4#5#6{%
9056 \edef\bbl@tempd{\fpeval{%
9057 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9058 \count@ \z@
9059 \@tempcnta=2015
9060 \bbl@foreach\bbl@cs@chinese@data{%
9061 \ifnum##1>\bbl@tempd\else
9062 \advance\count@\@ne
9063 \ifnum\count@>12
9064 \count@\@ne
9065 \advance\@tempcnta\@ne\fi
9066 \bbl@xin@{,##1,}{,\bbl@cs@chinese@leap,}%

```

```

9067 \ifin@
9068 \advance\count@m@ne
9069 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9070 \else
9071 \edef\bbl@tempe{\the\count@}%
9072 \fi
9073 \edef\bbl@tempb{##1}%
9074 \fi}%
9075 \edef#4{\the@tempcnta}%
9076 \edef#5{\bbl@tempe}%
9077 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9078 \def\bbl@cs@chinese@leap{%
9079 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9080 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9081 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9082 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9083 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9084 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9085 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9086 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9087 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9088 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9089 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9090 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9091 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9092 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9093 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9094 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9095 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9096 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9097 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9098 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9099 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9100 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9101 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9102 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9103 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9104 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9105 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9106 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9107 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9108 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9109 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9110 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9111 10896,10926,10956,10986,11015,11045,11074,11103}
9112 \ExplSyntaxOff
9113 </ca-chinese>

```

14. Support for Plain T_EX (plain.def)

14.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `locallyhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `lplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them

with $\text{\texttt{iniTeX}}$, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing $\text{\texttt{iniTeX}}$ sees, we need to set some category codes just to be able to change the definition of `\input`.

```
9114 <*\bplain | blplain>
9115 \catcode`\{=1 % left brace is begin-group character
9116 \catcode`\}=2 % right brace is end-group character
9117 \catcode`\#=6 % hash mark is macro parameter character
```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```
9118 \openin 0 hyphen.cfg
9119 \ifeof0
9120 \else
9121 \let\input
```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that's done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```
9122 \def\input #1 {%
9123 \let\input\a
9124 \a hyphen.cfg
9125 \let\a\undefined
9126 }
9127 \fi
9128 </bplain | blplain>
```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```
9129 <bplain>\a plain.tex
9130 <blplain>\a lplain.tex
```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```
9131 <bplain>\def\fmtname{babel-plain}
9132 <blplain>\def\fmtname{babel-lplain}
```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2. Emulating some $\text{\texttt{L}^A\text{T}_{\text{E}}\text{X}}$ features

The file `babel.def` expects some definitions made in the $\text{\texttt{L}^A\text{T}_{\text{E}}\text{X}} 2_{\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9133 <<*\Emulate LaTeX>> ≡
9134 \def\@empty{}
9135 \def\loadlocalcfg#1{%
9136 \openin0#1.cfg
9137 \ifeof0
9138 \closein0
9139 \else
9140 \closein0
9141 {\immediate\writel6{*****}%
9142 \immediate\writel6{* Local config file #1.cfg used}%
9143 \immediate\writel6{*}%
9144 }
9145 \input #1.cfg\relax
9146 \fi
9147 \@endofldef}
```

14.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```

9148 \long\def\@firstofone#1{#1}
9149 \long\def\@firstoftwo#1#2{#1}
9150 \long\def\@secondoftwo#1#2{#2}
9151 \def\@nnil{\@nil}
9152 \def\@gobbletwo#1#2{}
9153 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9154 \def\@star@or@long#1{%
9155   \@ifstar
9156   {\let\l@ngrel@x\relax#1}%
9157   {\let\l@ngrel@x\long#1}}
9158 \let\l@ngrel@x\relax
9159 \def\@car#1#2\@nil{#1}
9160 \def\@cdr#1#2\@nil{#2}
9161 \let\@typeset@protect\relax
9162 \let\protected@edef\edef
9163 \long\def\@gobble#1{}
9164 \edef\@backslashchar{\expandafter\@gobble\string\}
9165 \def\strip@prefix#1>{}
9166 \def\g@addto@macro#1#2{{%
9167   \toks@\expandafter{#1#2}%
9168   \xdef#1{\the\toks@}}}
9169 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9170 \def\@nameuse#1{\csname #1\endcsname}
9171 \def\@ifundefined#1{%
9172   \expandafter\ifx\csname#1\endcsname\relax
9173     \expandafter\@firstoftwo
9174     \else
9175     \expandafter\@secondoftwo
9176     \fi}
9177 \def\@expandtwoargs#1#2#3{%
9178   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9179 \def\zap@space#1 #2{%
9180   #1%
9181   \ifx#2\@empty\else\expandafter\zap@space\fi
9182   #2}
9183 \let\bbl@trace\@gobble
9184 \def\bbl@error#1{% Implicit #2#3#4
9185   \begingroup
9186     \catcode`\=0 \catcode`\==12 \catcode`\`=12
9187     \catcode`\^M=5 \catcode`\%=14
9188     \input errbabel.def
9189   \endgroup
9190   \bbl@error{#1}}
9191 \def\bbl@warning#1{%
9192   \begingroup
9193     \newlinechar=`^^J
9194     \def\{^^J(babel) }%
9195     \message{\{#1}%
9196     \endgroup}
9197 \let\bbl@infowarn\bbl@warning
9198 \def\bbl@info#1{%
9199   \begingroup
9200     \newlinechar=`^^J
9201     \def\{^^J}%
9202     \wlog{#1}%
9203   \endgroup}

```

$\text{\LaTeX 2}_\varepsilon$ has the command `\onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9204 \ifx\@preamblecmds\undefined

```

```

9205 \def\@preamblecmds{}
9206 \fi
9207 \def\@onlypreamble#1{%
9208 \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9209 \@preamblecmds\do#1}}
9210 \@onlypreamble\@onlypreamble

```

Mimic L^AT_EX's \AtBeginDocument; for this to work the user needs to add \begindocument to his file.

```

9211 \def\begindocument{%
9212 \@begindocumenthook
9213 \global\let\@begindocumenthook\@undefined
9214 \def\do##1{\global\let##1\@undefined}%
9215 \@preamblecmds
9216 \global\let\do\noexpand}

9217 \ifx\@begindocumenthook\@undefined
9218 \def\@begindocumenthook{}
9219 \fi
9220 \@onlypreamble\@begindocumenthook
9221 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic L^AT_EX's \AtEndOfPackage. Our replacement macro is much simpler; it stores its argument in \@endofldf.

```

9222 \def\AtEndOfPackage#1{\g@addto@macro\@endofldf{#1}}
9223 \@onlypreamble\AtEndOfPackage
9224 \def\@endofldf{}
9225 \@onlypreamble\@endofldf
9226 \let\bbl@afterlang\@empty
9227 \chardef\bbl@opt@hyphenmap\z@

```

L^AT_EX needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer \ifx. The same trick is applied below.

```

9228 \catcode`\&=\z@
9229 \ifx&\if@filesw\@undefined
9230 \expandafter\let\csname if@filesw\expandafter\endcsname
9231 \csname iffalse\endcsname
9232 \fi
9233 \catcode`\&=4

```

Mimic L^AT_EX's commands to define control sequences.

```

9234 \def\newcommand{\@star@or@long\new@command}
9235 \def\new@command#1{%
9236 \@testopt{\@newcommand#1}0}
9237 \def\@newcommand#1[#2]{%
9238 \@ifnextchar [{\@xargdef#1[#2]}%
9239 {\@argdef#1[#2]}}
9240 \long\def\@argdef#1[#2]#3{%
9241 \@yargdef#1\@ne{#2}{#3}}
9242 \long\def\@xargdef#1[#2][#3]#4{%
9243 \expandafter\def\expandafter#1\expandafter{%
9244 \expandafter\@protected@testopt\expandafter #1%
9245 \csname\string#1\expandafter\endcsname{#3}}}%
9246 \expandafter\@yargdef \csname\string#1\endcsname
9247 \tw@{#2}{#4}}
9248 \long\def\@yargdef#1#2#3{%
9249 \@tempcnta#3\relax
9250 \advance \@tempcnta \@ne
9251 \let\@hash@\relax
9252 \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9253 \@tempcntb #2%
9254 \@whilenum \@tempcntb <\@tempcnta
9255 \do{%
9256 \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%

```



```

9257 \advance\@tempcntb \@ne}%
9258 \let\@hash@##%
9259 \l@ngrel@x\expandafter\def\expandafter#1\reserved@a}
9260 \def\providecommand{\@star@or@long\provide@command}
9261 \def\provide@command#1{%
9262 \begingroup
9263 \escapechar\m@ne\xdef\@gtempa{\string#1}%
9264 \endgroup
9265 \expandafter\ifundefined\@gtempa
9266 {\def\reserved@a{\new@command#1}}%
9267 {\let\reserved@a\relax
9268 \def\reserved@a{\new@command\reserved@a}}%
9269 \reserved@a}%

9270 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9271 \def\declare@robustcommand#1{%
9272 \edef\reserved@a{\string#1}%
9273 \def\reserved@b{#1}%
9274 \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9275 \edef#1{%
9276 \ifx\reserved@a\reserved@b
9277 \noexpand\x@protect
9278 \noexpand#1%
9279 \fi
9280 \noexpand\protect
9281 \expandafter\noexpand\csname
9282 \expandafter\@gobble\string#1 \endcsname
9283 }%
9284 \expandafter\new@command\csname
9285 \expandafter\@gobble\string#1 \endcsname
9286 }
9287 \def\x@protect#1{%
9288 \ifx\protect\@typeset@protect\else
9289 \@x@protect#1%
9290 \fi
9291 }
9292 \catcode`\&=\z@ % Trick to hide conditionals
9293 \def\@x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9294 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9295 \catcode`\&=4
9296 \ifx\in@\@undefined
9297 \def\in@#1#2{%
9298 \def\in@##1#1##2##3\in@{%
9299 \ifx\in@##2\in@false\else\in@true\fi}%
9300 \in@##2#1\in@\in@}
9301 \else
9302 \let\bbl@tempa\@empty
9303 \fi
9304 \bbl@tempa

```

$\TeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (activegrave and activeacute). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9305 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\TeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9306 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\TeX$ versions; just enough to make things work in plain $\TeX$ environments.

```
9307 \ifx\@tempcnta\@undefined
9308   \csname newcount\endcsname\@tempcnta\relax
9309 \fi
9310 \ifx\@tempcntb\@undefined
9311   \csname newcount\endcsname\@tempcntb\relax
9312 \fi
```

To prevent wasting two counters in $\TeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```
9313 \ifx\bye\@undefined
9314   \advance\count10 by -2\relax
9315 \fi
9316 \ifx\@ifnextchar\@undefined
9317   \def\@ifnextchar#1#2#3{%
9318     \let\reserved@d=#1%
9319     \def\reserved@a{#2}\def\reserved@b{#3}%
9320     \futurelet\@let@token\@ifnch}
9321   \def\@ifnch{%
9322     \ifx\@let@token\@sptoken
9323       \let\reserved@c\@xifnch
9324     \else
9325       \ifx\@let@token\reserved@d
9326         \let\reserved@c\reserved@a
9327       \else
9328         \let\reserved@c\reserved@b
9329       \fi
9330     \fi
9331     \reserved@c}
9332   \def\:{\let\@sptoken= }\: % this makes \@sptoken a space token
9333   \def\:{\@xifnch} \expandafter\def\:{\futurelet\@let@token\@ifnch}
9334 \fi
9335 \def\@testopt#1#2{%
9336   \@ifnextchar[#{1}{#1[#2]}}
9337 \def\@protected@testopt#1{%
9338   \ifx\protect\@typeset@protect
9339     \expandafter\@testopt
9340   \else
9341     \@x@protect#1%
9342   \fi}
9343 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9344   #2\relax}\fi}
9345 \long\def\@iwhilenum#1{\ifnum #1\relax\expandafter\@iwhilenum
9346   \else\expandafter\@gobble\fi{#1}}
```

14.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```
9347 \def\DeclareTextCommand{%
9348   \@dec@text@cmd\providecommand
9349 }
9350 \def\ProvideTextCommand{%
9351   \@dec@text@cmd\providecommand
9352 }
9353 \def\DeclareTextSymbol#1#2#3{%
9354   \@dec@text@cmd\chardef#1{#2}#3\relax
9355 }
9356 \def\@dec@text@cmd#1#2#3{%
9357   \expandafter\def\expandafter#2%
9358     \expandafter{%
```

```

9359         \csname#3-cmd\expandafter\endcsname
9360         \expandafter#2%
9361         \csname#3\string#2\endcsname
9362     }%
9363 % \let\@ifdefinable\@rc@ifdefinable
9364 \expandafter#1\csname#3\string#2\endcsname
9365 }
9366 \def\@current@cmd#1{%
9367 \ifx\protect\@typeset@protect\else
9368     \noexpand#1\expandafter\@gobble
9369 \fi
9370 }
9371 \def\@changed@cmd#1#2{%
9372 \ifx\protect\@typeset@protect
9373     \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9374     \expandafter\ifx\csname ?\string#1\endcsname\relax
9375     \expandafter\def\csname ?\string#1\endcsname{%
9376         \@changed@x@err{#1}%
9377     }%
9378 \fi
9379 \global\expandafter\let
9380     \csname\cf@encoding \string#1\expandafter\endcsname
9381     \csname ?\string#1\endcsname
9382 \fi
9383 \csname\cf@encoding\string#1%
9384 \expandafter\endcsname
9385 \else
9386     \noexpand#1%
9387 \fi
9388 }
9389 \def\@changed@x@err#1{%
9390     \errhelp{Your command will be ignored, type <return> to proceed}%
9391     \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9392 \def\DeclareTextCommandDefault#1{%
9393     \DeclareTextCommand#1?%
9394 }
9395 \def\ProvideTextCommandDefault#1{%
9396     \ProvideTextCommand#1?%
9397 }
9398 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9399 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9400 \def\DeclareTextAccent#1#2#3{%
9401     \DeclareTextCommand#1{#2}[1]{\accent#3 #1}
9402 }
9403 \def\DeclareTextCompositeCommand#1#2#3#4{%
9404     \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9405     \edef\reserved@b{\string##1}%
9406     \edef\reserved@c{%
9407         \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9408     \ifx\reserved@b\reserved@c
9409         \expandafter\expandafter\expandafter\ifx
9410             \expandafter\@car\reserved@a\relax\relax\@nil
9411             \@text@composite
9412     \else
9413         \edef\reserved@b##1{%
9414             \def\expandafter\noexpand
9415                 \csname#2\string#1\endcsname###1{%
9416                 \noexpand\@text@composite
9417                     \expandafter\noexpand\csname#2\string#1\endcsname
9418                     ###1\noexpand\@empty\noexpand\@text@composite
9419                     {##1}%
9420             }%
9421         }%

```

```

9422     \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9423     \fi
9424     \expandafter\def\csname\expandafter\string\csname
9425       #2\endcsname\string#1-\string#3\endcsname{#4}
9426   \else
9427     \errhelp{Your command will be ignored, type <return> to proceed}%
9428     \errmessage{\string\DeclareTextCompositeCommand\space used on
9429       inappropriate command \protect#1}
9430   \fi
9431 }
9432 \def\@text@composite#1#2#3\@text@composite{%
9433   \expandafter\@text@composite@x
9434     \csname\string#1-\string#2\endcsname
9435 }
9436 \def\@text@composite@x#1#2{%
9437   \ifx#1\relax
9438     #2%
9439   \else
9440     #1%
9441   \fi
9442 }
9443 %
9444 \def\@strip@args#1:#2-#3\@strip@args{#2}
9445 \def\DeclareTextComposite#1#2#3#4{%
9446   \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9447   \bgroup
9448     \lccode`\@=#4%
9449     \lowercase{%
9450   \egroup
9451     \reserved@a \@%
9452   }%
9453 }
9454 %
9455 \def\UseTextSymbol#1#2{#2}
9456 \def\UseTextAccent#1#2#3{}
9457 \def\@use@text@encoding#1{}
9458 \def\DeclareTextSymbolDefault#1#2{%
9459   \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9460 }
9461 \def\DeclareTextAccentDefault#1#2{%
9462   \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9463 }
9464 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX} 2_{\epsilon}$ method for accents for those that are known to be made active in *some* language definition file.

```

9465 \DeclareTextAccent{"}{OT1}{127}
9466 \DeclareTextAccent{'}{OT1}{19}
9467 \DeclareTextAccent{^}{OT1}{94}
9468 \DeclareTextAccent{\`}{OT1}{18}
9469 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for PLAIN $\text{\TeX}$.

```

9470 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9471 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9472 \DeclareTextSymbol{\textquoteleft}{OT1}{``}
9473 \DeclareTextSymbol{\textquoteright}{OT1}{``'}
9474 \DeclareTextSymbol{\i}{OT1}{16}
9475 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because plain $\text{\TeX}$ doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9476 \ifx\scriptsize\undefined
9477   \let\scriptsize\sevenrm

```

```

9478 \fi

And a few more “dummy” definitions.

9479 \def\language{english}%
9480 \let\bbl@opt@shorthands\@nnil
9481 \def\bbl@ifshorthand#1#2#3{#2}%
9482 \let\bbl@language@opts\@empty
9483 \let\bbl@provide@locale\relax
9484 \ifx\babeloptionstrings\undefined
9485   \let\bbl@opt@strings\@nnil
9486 \else
9487   \let\bbl@opt@strings\babeloptionstrings
9488 \fi
9489 \def\BabelStringsDefault{generic}
9490 \def\bbl@tempa{normal}
9491 \ifx\babeloptionmath\bbl@tempa
9492   \def\bbl@mathnormal{\noexpand\textormath}
9493 \fi
9494 \def\AfterBabelLanguage#1#2{}
9495 \ifx\BabelModifiers\undefined\let\BabelModifiers\relax\fi
9496 \let\bbl@afterlang\relax
9497 \def\bbl@opt@safe{BR}
9498 \ifx\@uclclist\undefined\let\@uclclist\@empty\fi
9499 \ifx\bbl@trace\undefined\def\bbl@trace#1{}\fi
9500 \expandafter\newif\csname ifbbl@single\endcsname
9501 \chardef\bbl@bidimode\z@
9502 <</Emulate LaTeX>>

A proxy file:

9503 <*\plain>
9504 \input babel.def
9505 </\plain>

```

15. Acknowledgements

In the initial stages of the development of babel, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn’t exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\text{\LaTeX}$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O’Reilly, 2007.
- [4] Donald E. Knuth, *The $\text{\TeX}$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O’Reilly, 2006.
- [6] Leslie Lamport, *$\text{\LaTeX}$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\text{\TeX}$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O’Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018
- [10] Hubert Partl, *German $\text{\TeX}$* , *TUGboat* 9 (1988) #1, pp. 70–72.

- [11] Joachim Schrod, *International $\text{\LaTeX}$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\text{\LaTeX}$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer, een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).